

Deep Learning for Computer Vision

Lecture 3: CNNs

Constantin Pape

Institute of Computer Science, Georg August Universität Göttingen



@cppape.bsky.social

<https://user.informatik.uni-goettingen.de/~pape41/>

Agenda for today

Network Architectures

- Activation functions
- Deep vs. wide networks
- Architecture types

Convolutional neural networks

- Introduction to CNNs
- Batch normalization
- Modern CNN architectures (probably next week)

Next week:

- Practical aspects of CNN training
- Transfer learning

Organization

Tutorial organization

- Latest tutorial assignments are on Stud.IP: “Misc -> tutorial-assignments-final.xlsx”
- We created a chat room for each individual tutorial. The links are in the main chat room. Please join the room for your tutorial to coordinate with your tutor and fellow students in your tutorial.
- **Tutorials start this week, tutors will share details on the timetable with you.**

Exercise presentation

- Split exercise in 3 parts, decide which group member presents which part
- Prepare for the presentation!
 - Focus on high-level, the results and answers to questions in the exercise
- All group members should come to the tutorial!
 - If you can't come once: that's ok, but we will keep track of attendance!
 - If someone from the group does not show up and/or does not present without excuse repeatedly, they will not get the exam qualification.
 - We will warn you about this.

Next exercise

- Will be available on Stud.IP later today.
- You will design CNNs and train them on CIFAR
 - You will need to train quite a few models:
 - Start early!
 - Distribute trainings among group members
- The content of today's lecture is important for the exercise.

Student assistant positions

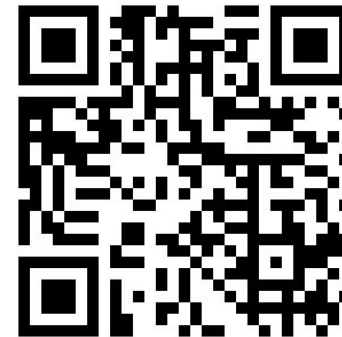
EKFZ: Student Assistant – IT Support for Regulatory Affairs

- Build up IT infrastructure for documenting gene therapy trials
- <https://ekfz.uni-goettingen.de/stellenausschreibungen/>



CryoET Facility: Scientific IT Infrastructure & DevOps Automation

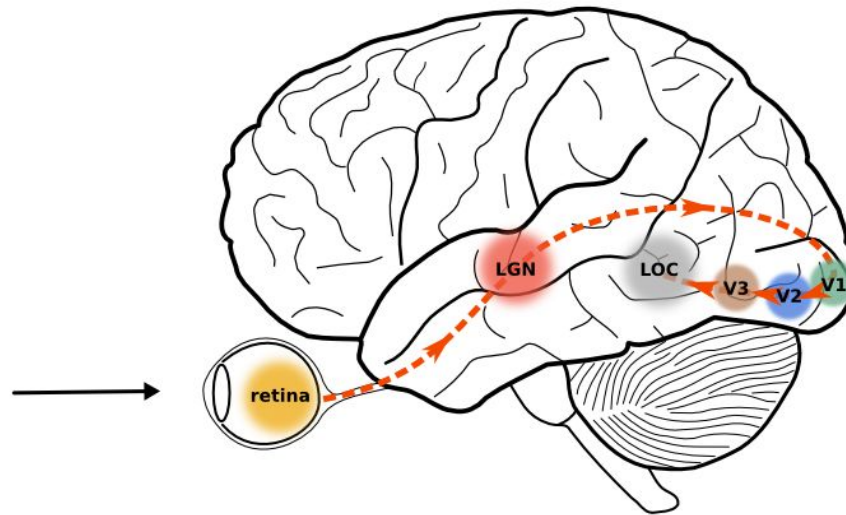
- Build up IT infrastructure for cryogenic electron tomography
- <https://owncloud.gwdg.de/index.php/s/WtIA9RPAEaPnPuv>



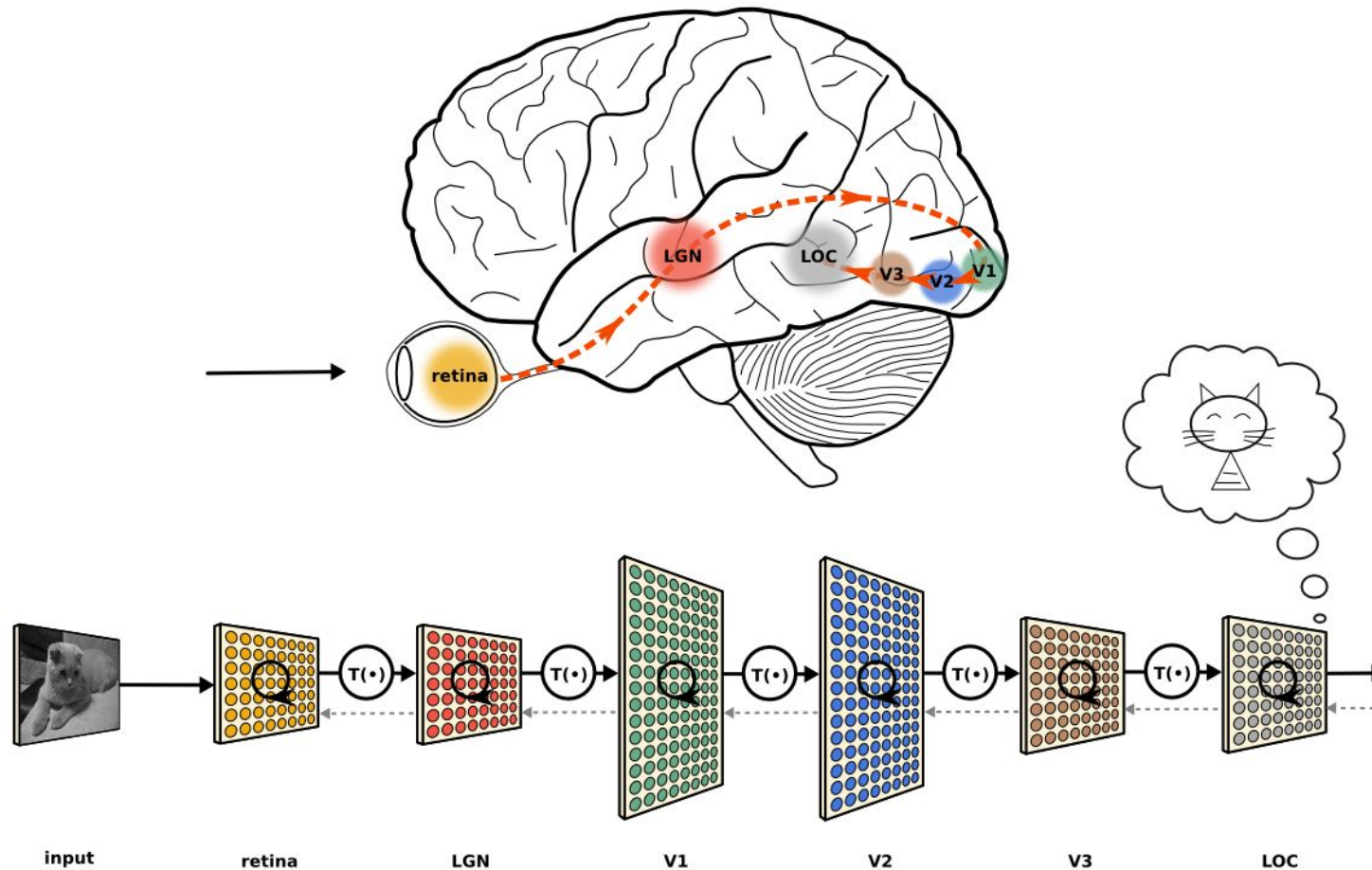
Convolutional Neural Networks (CNNs)

BACKGROUND & HISTORY: FROM PRIMARY VISUAL CORTEX TO CNNs

Hierarchical visual processing in the brain



Hierarchical visual processing in the brain



Hubel & Wiesel: Simple cells

Selective to orientation and location

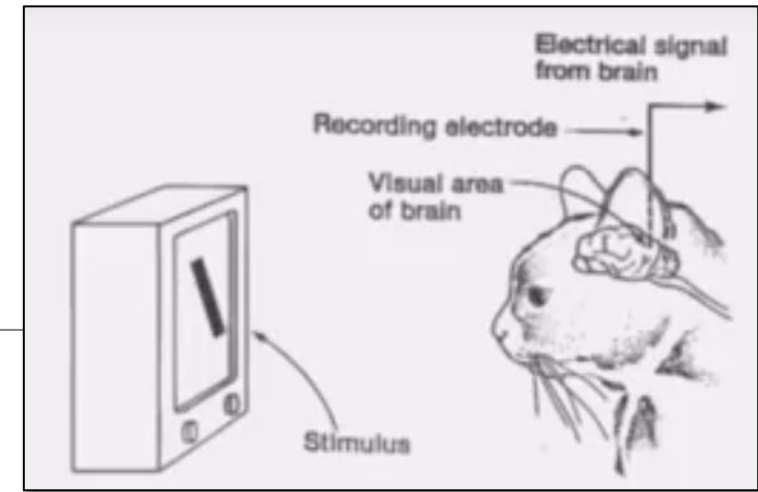
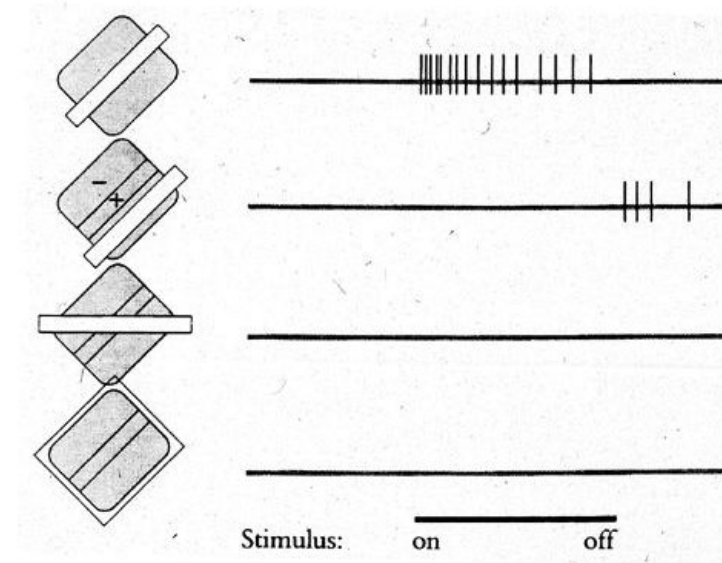
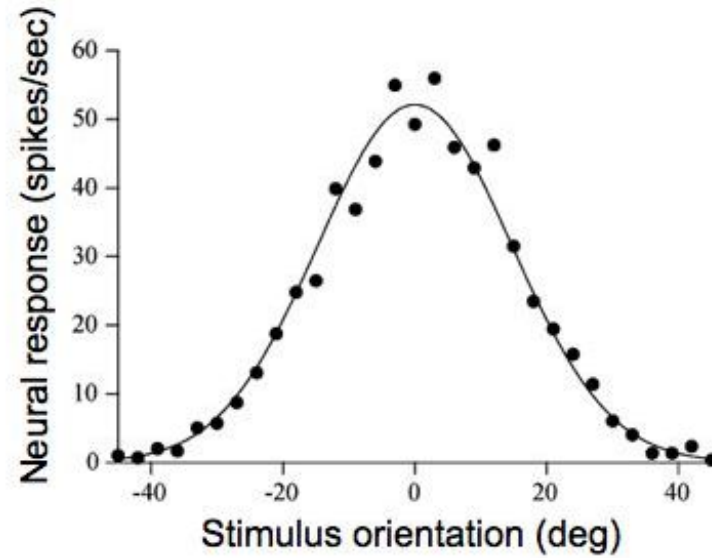
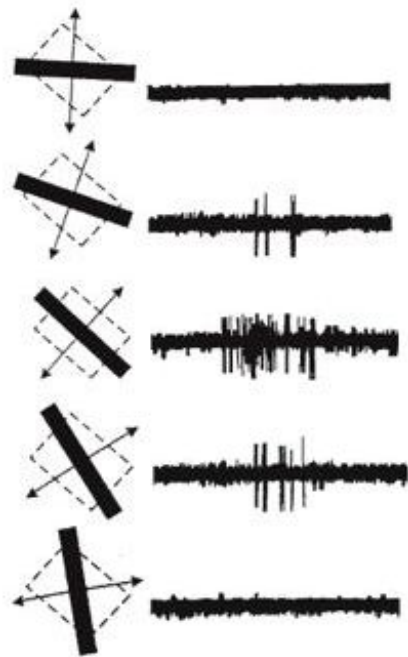
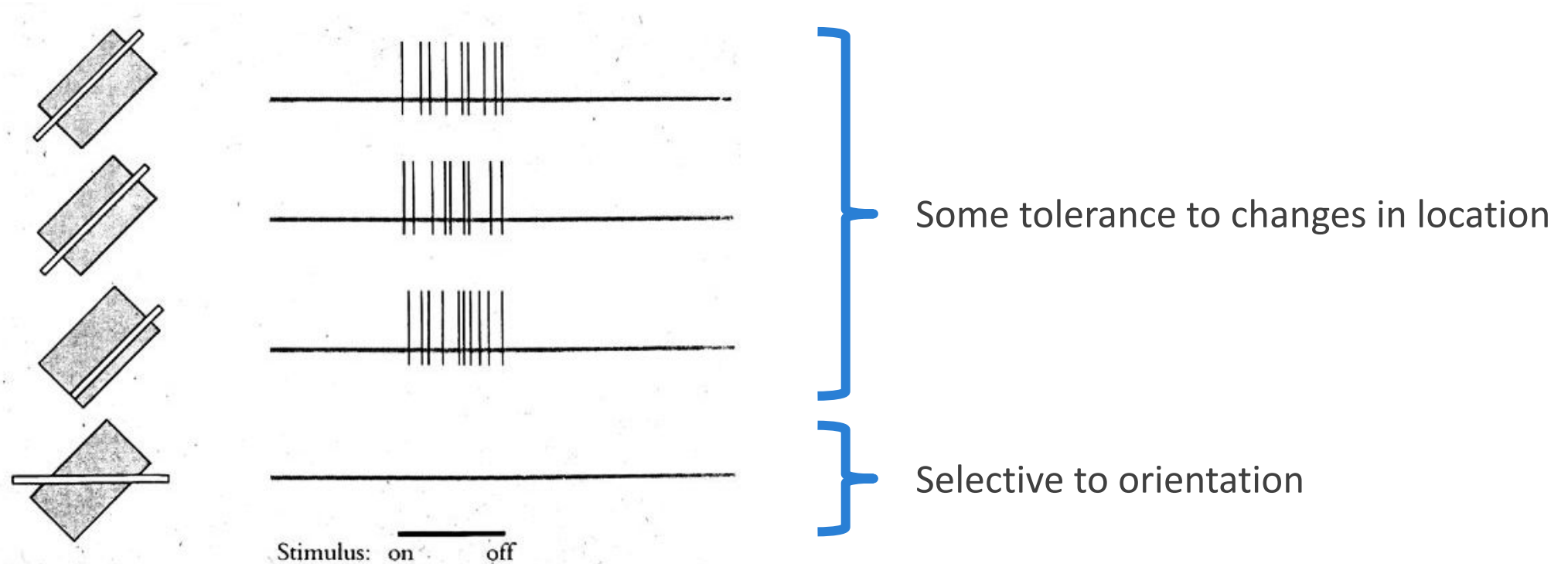


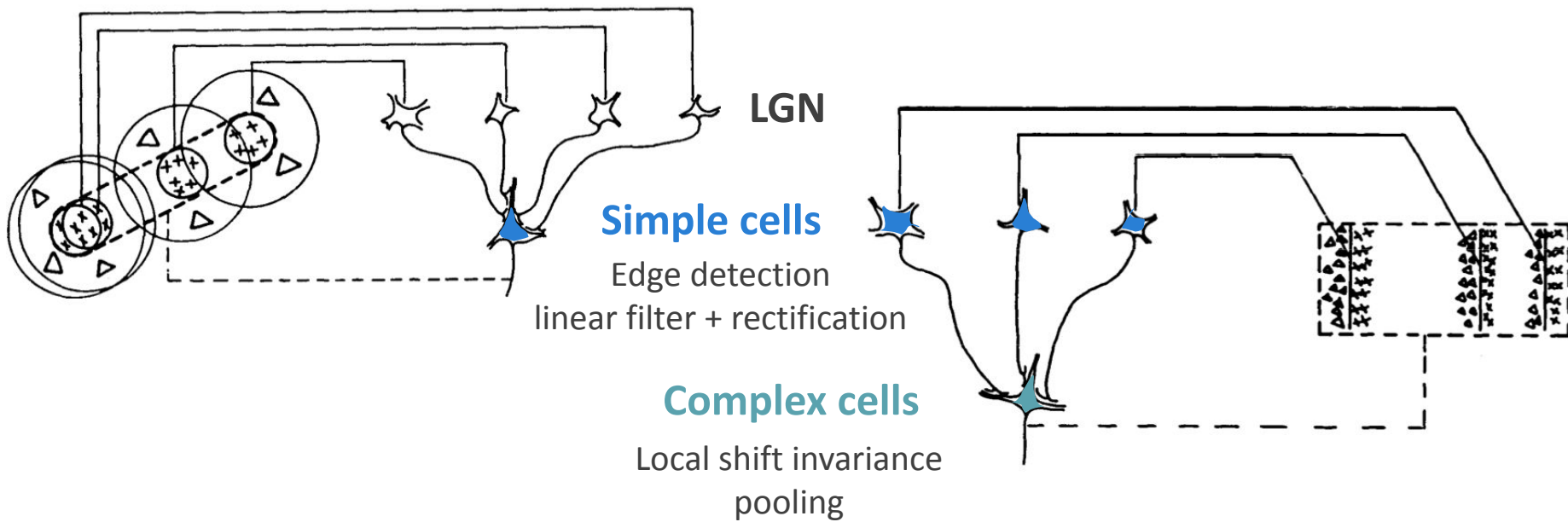
Figure credit: Andrej Karpathy



Hubel & Wiesel: Complex cells

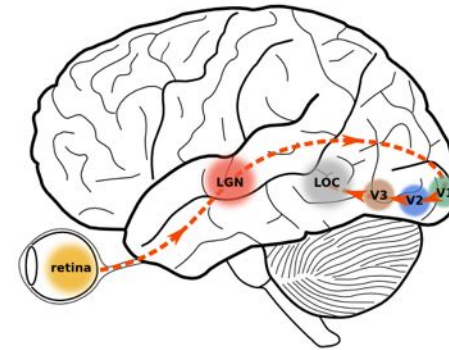
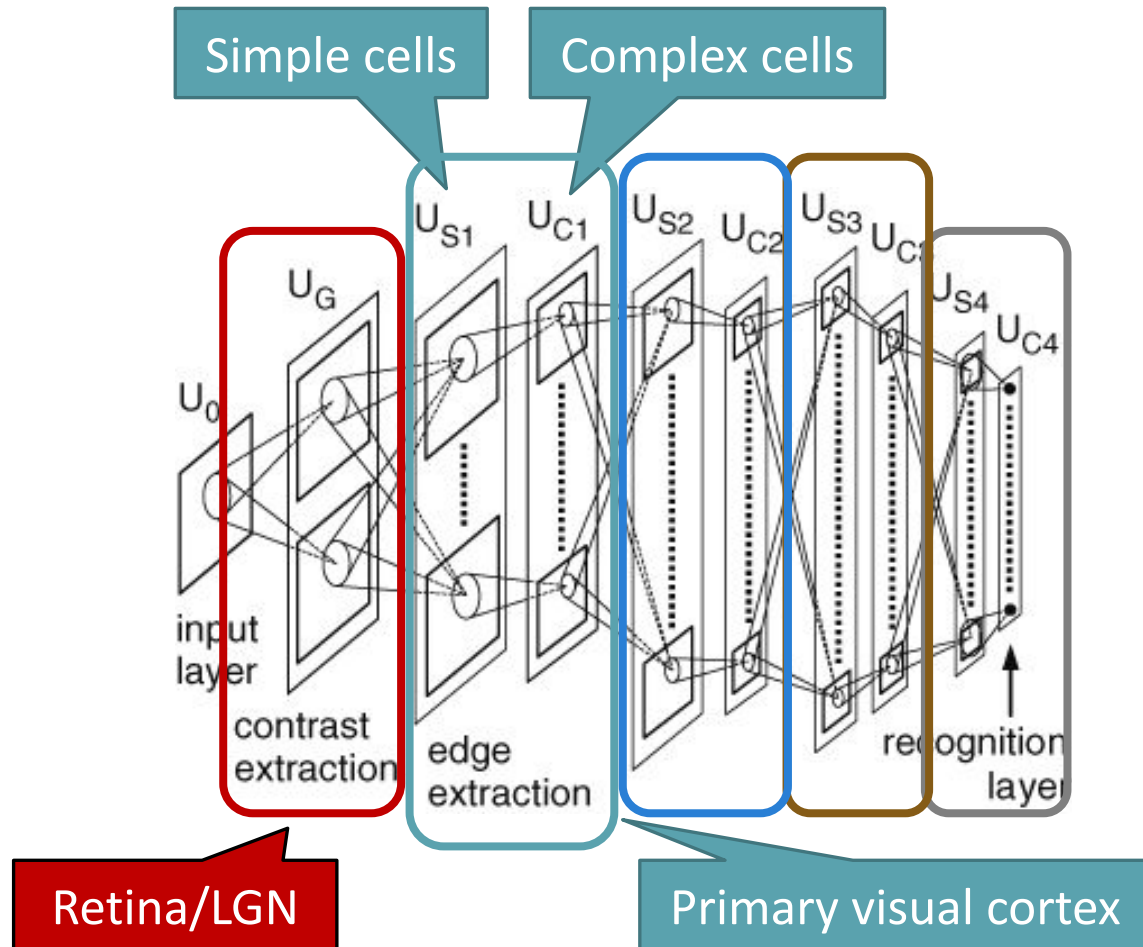


Hubel & Wiesel: Summary

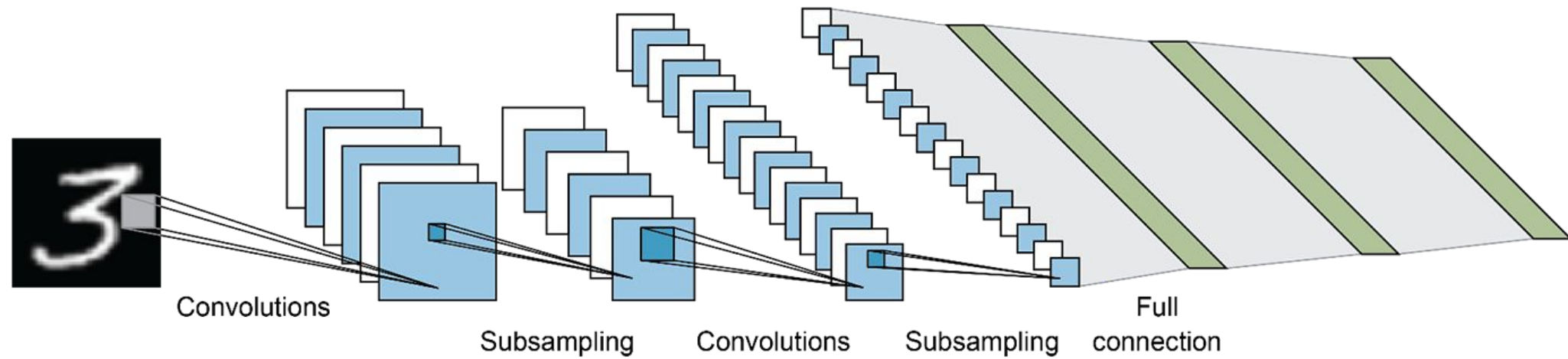


Retina +
LGN Primary visual cortex
(V1)

The Neocognitron (Fukushima 1980)



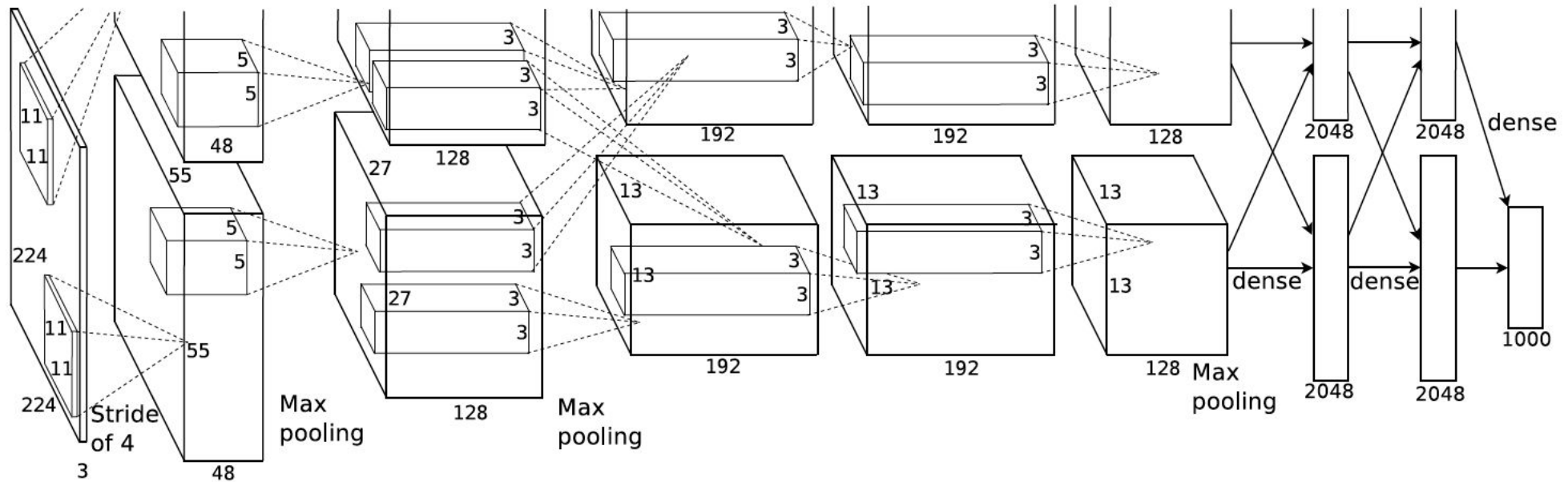
Convolutional neural networks (CNN)



Denker et al. 1989, LeCun et al. 1989, 1998

ImageNet Classification with Deep Convolutional Neural Networks

Krizhevsky, Sutskever, Hinton 2012



Today usually referred to as “**AlexNet**”

Fast-forward to today: ConvNets are everywhere

Classification



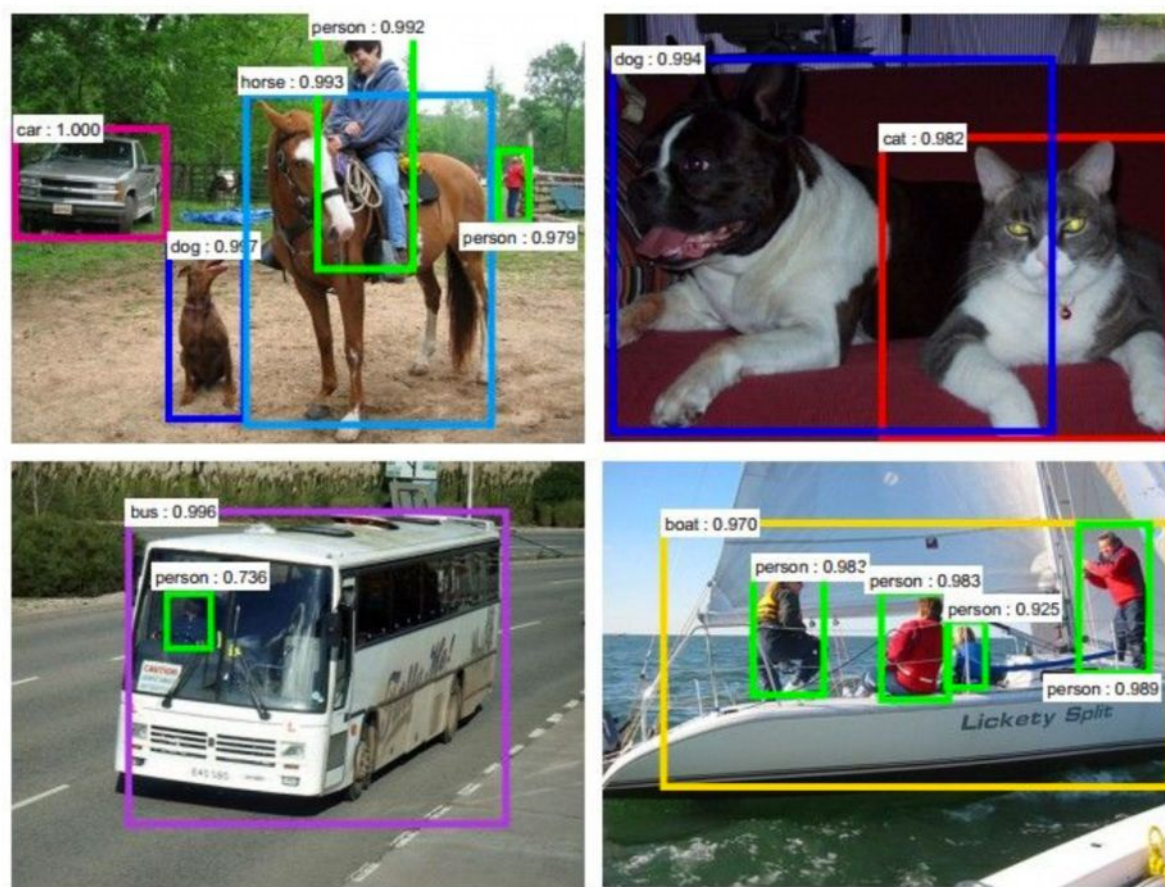
Retrieval



Figures copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

Fast-forward to today: ConvNets are everywhere

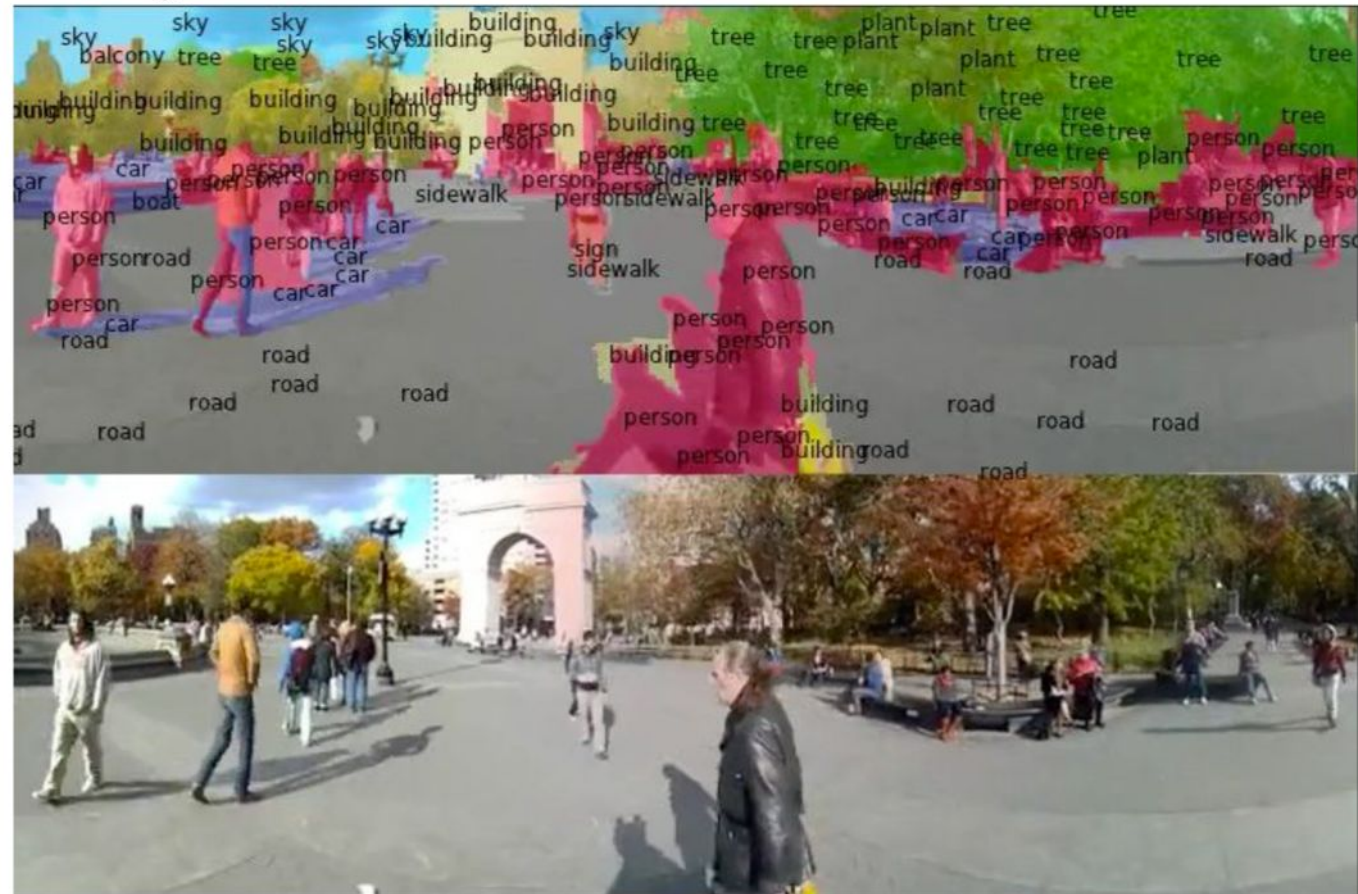
Detection



Figures copyright Shaoqing Ren, Kaiming He, Ross Girshick, Jian Sun, 2015. Reproduced with permission.

[Faster R-CNN: Ren, He, Girshick, Sun 2015]

Segmentation

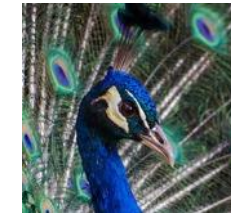
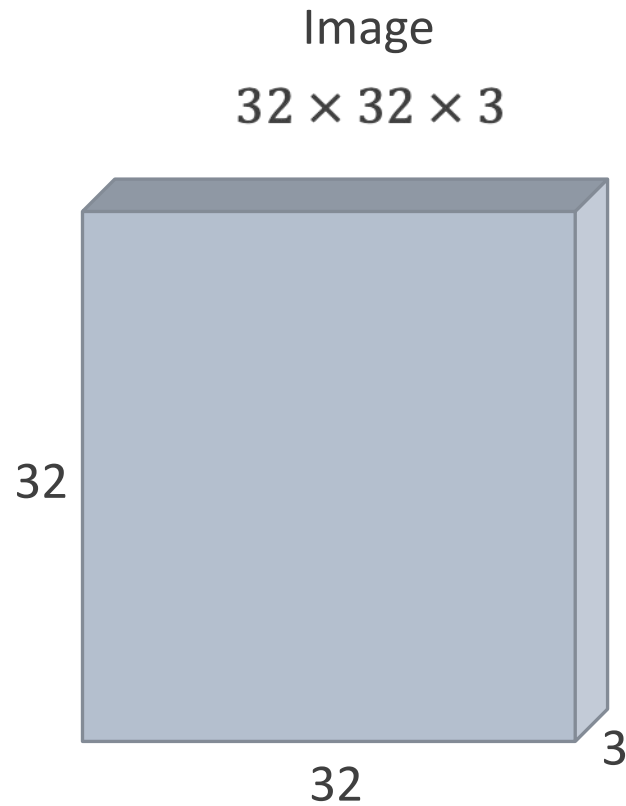


Figures copyright Clement Farabet, 2012.
Reproduced with permission.

[Farabet et al., 2012]

Convolution

Consider a 32×32 RGB image



Red



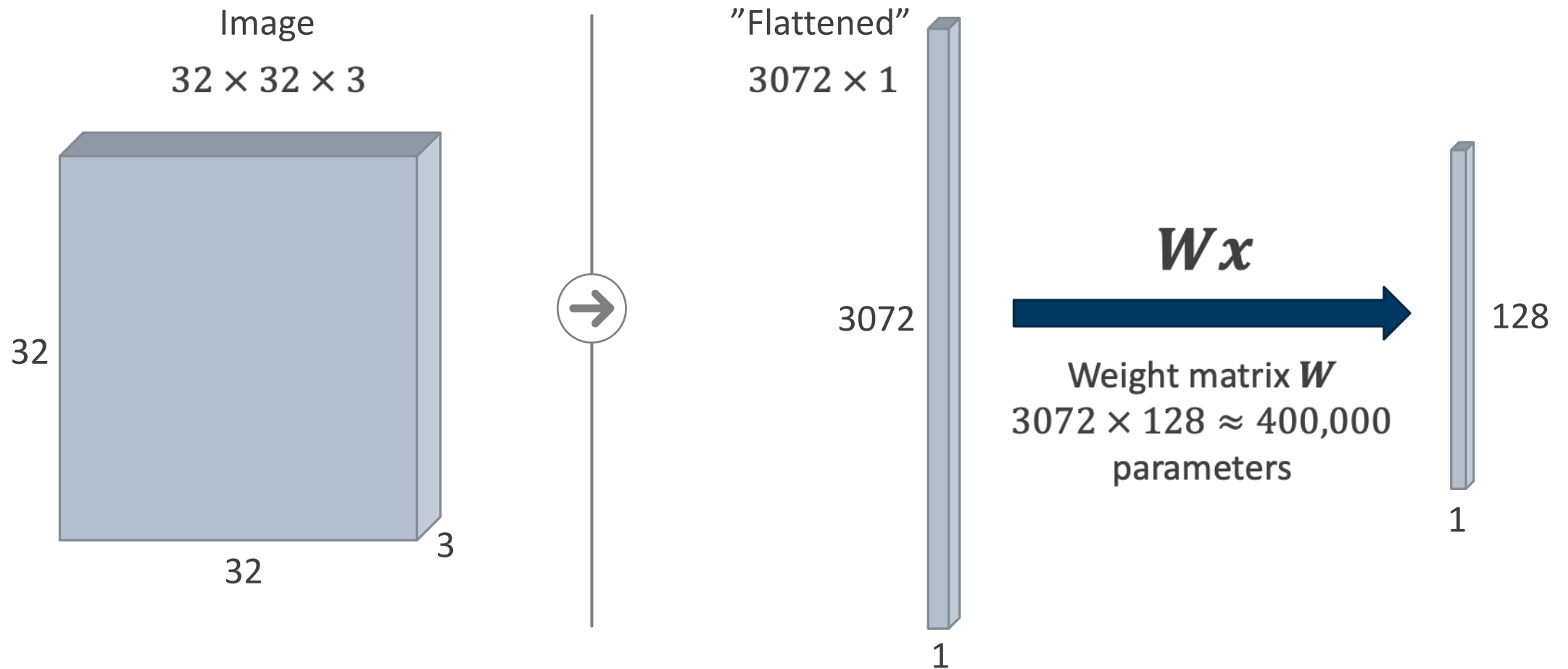
Green



Blue



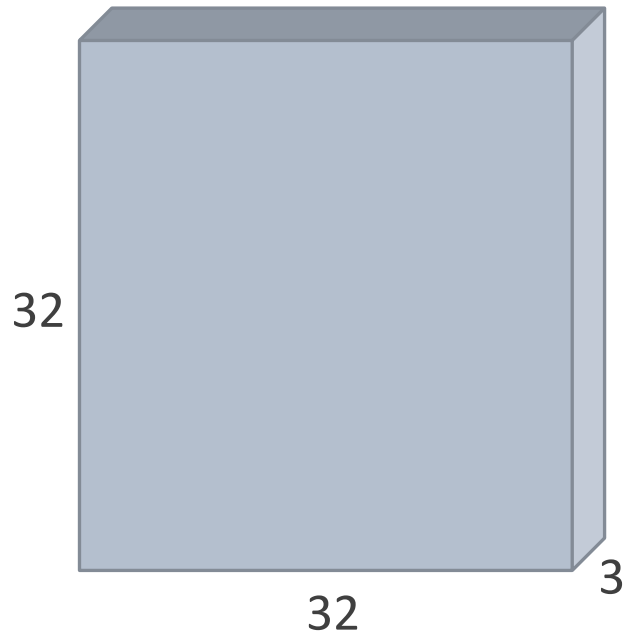
Fully-connected layer



Convolution

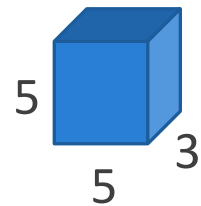
Image

$32 \times 32 \times 3$



Filter

$5 \times 5 \times 3$

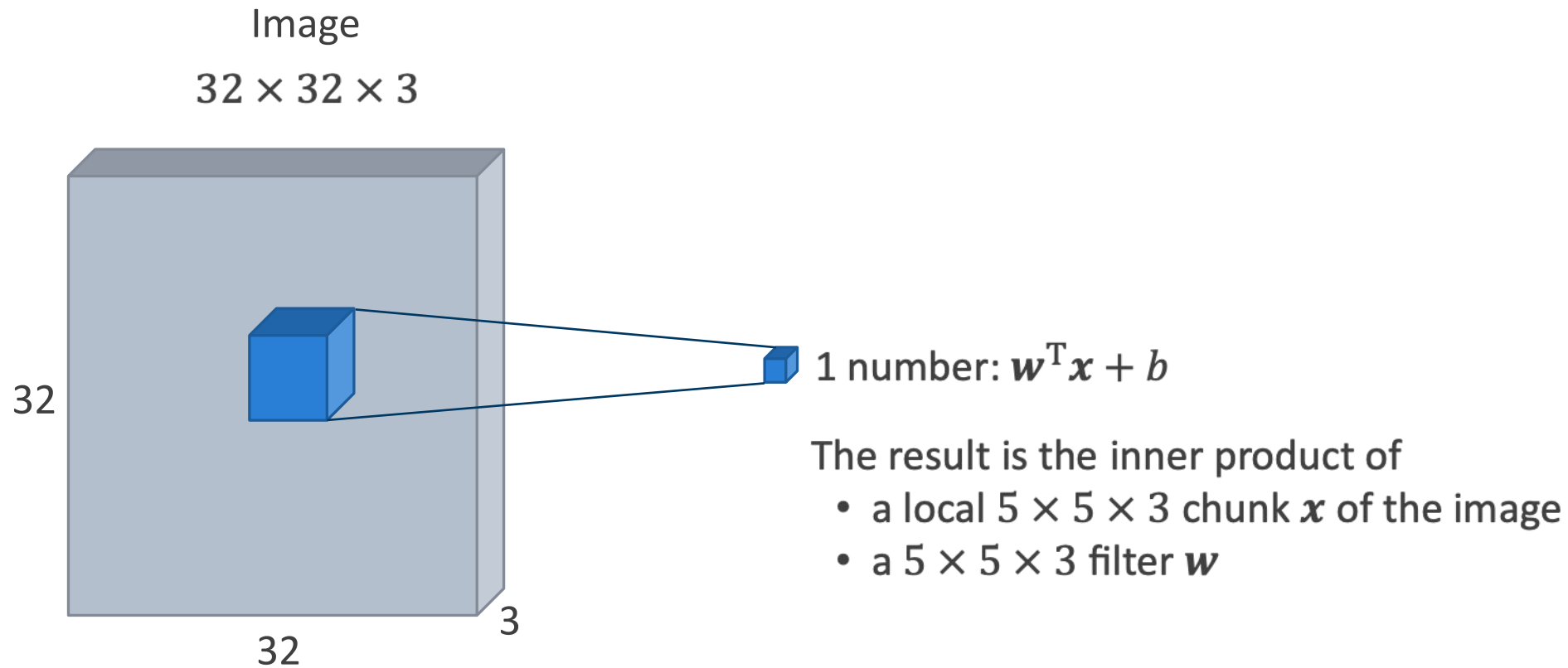


Convolve the filter with the image

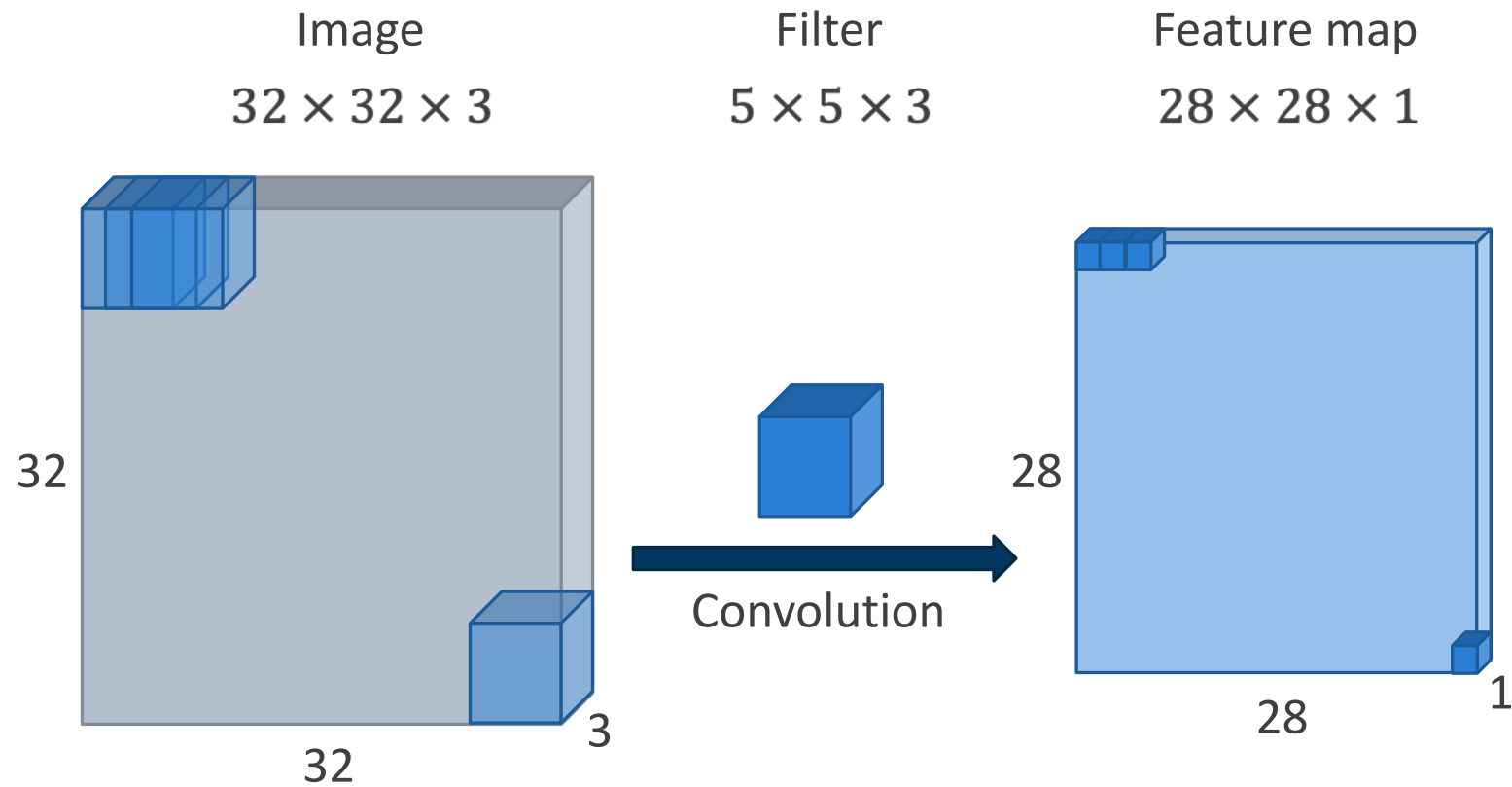


Slide over the image, computing dot products

Convolution

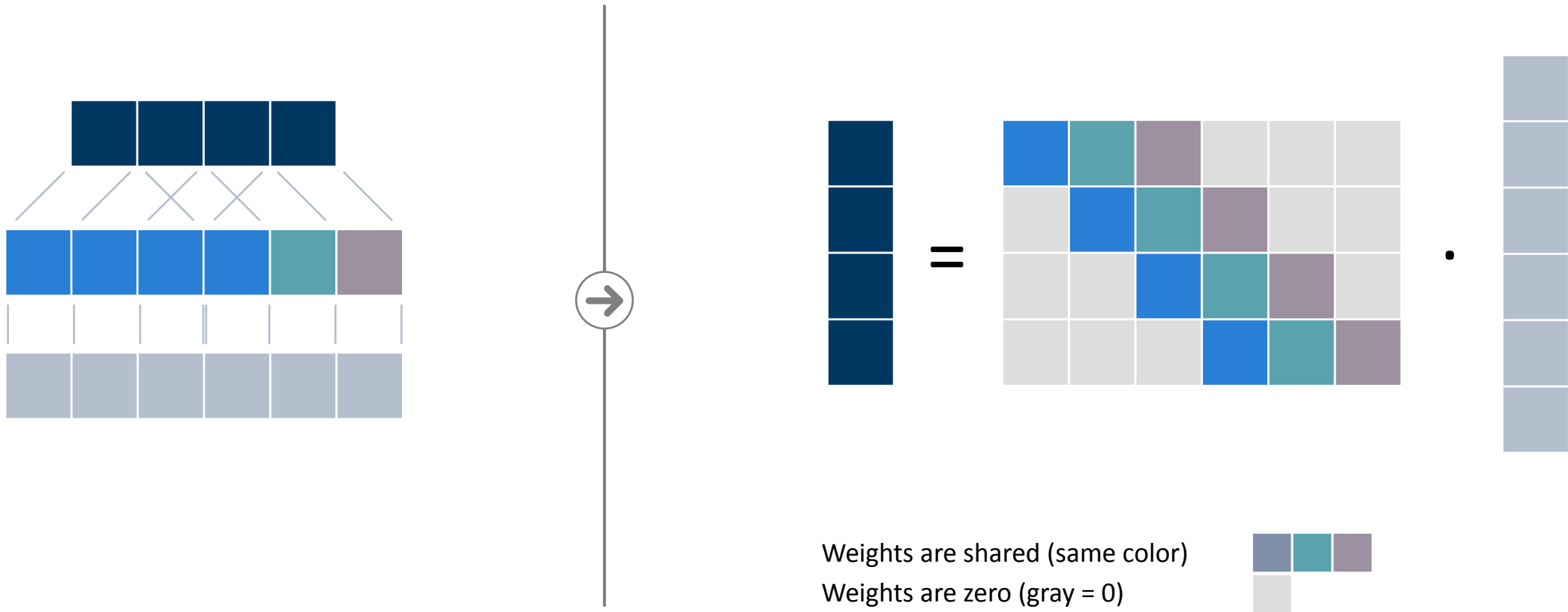


Convolution

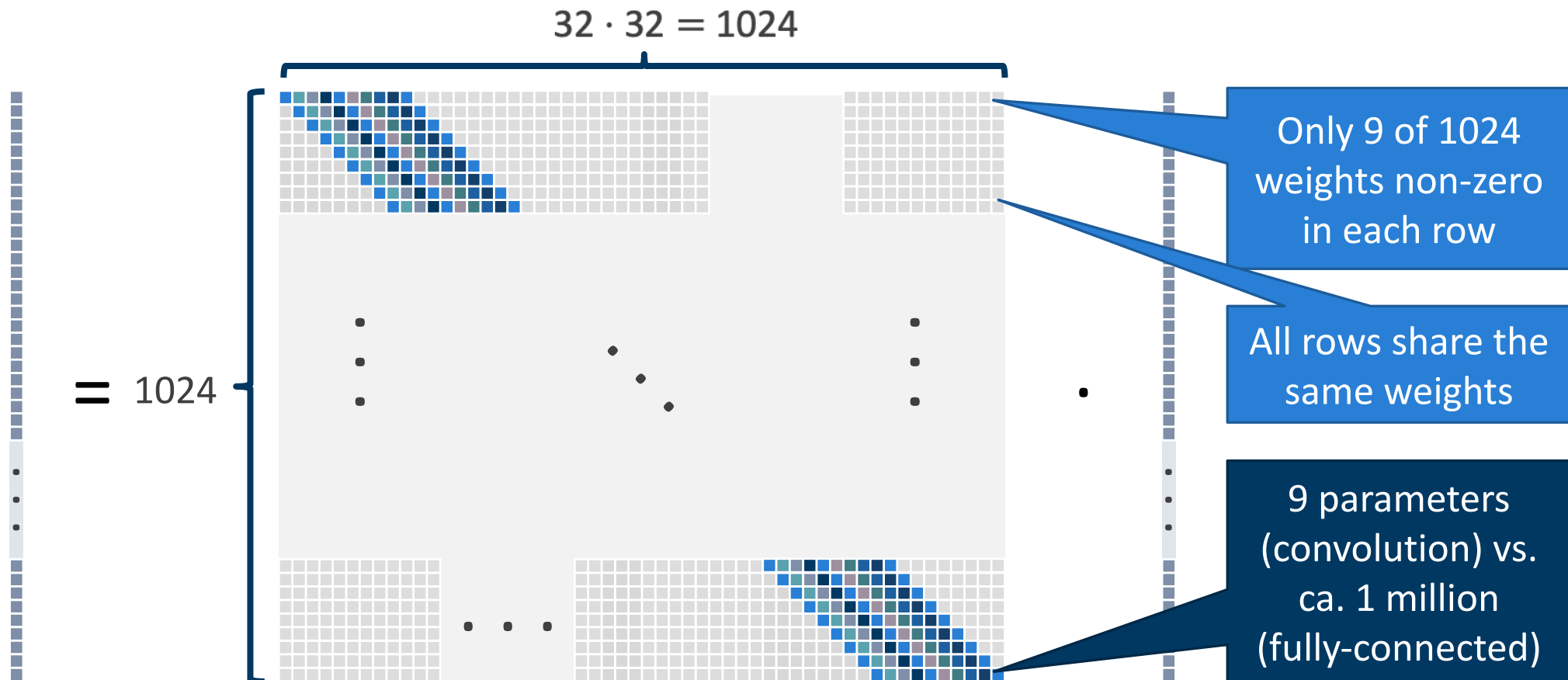


Weight sharing as the key advantage of CNNs

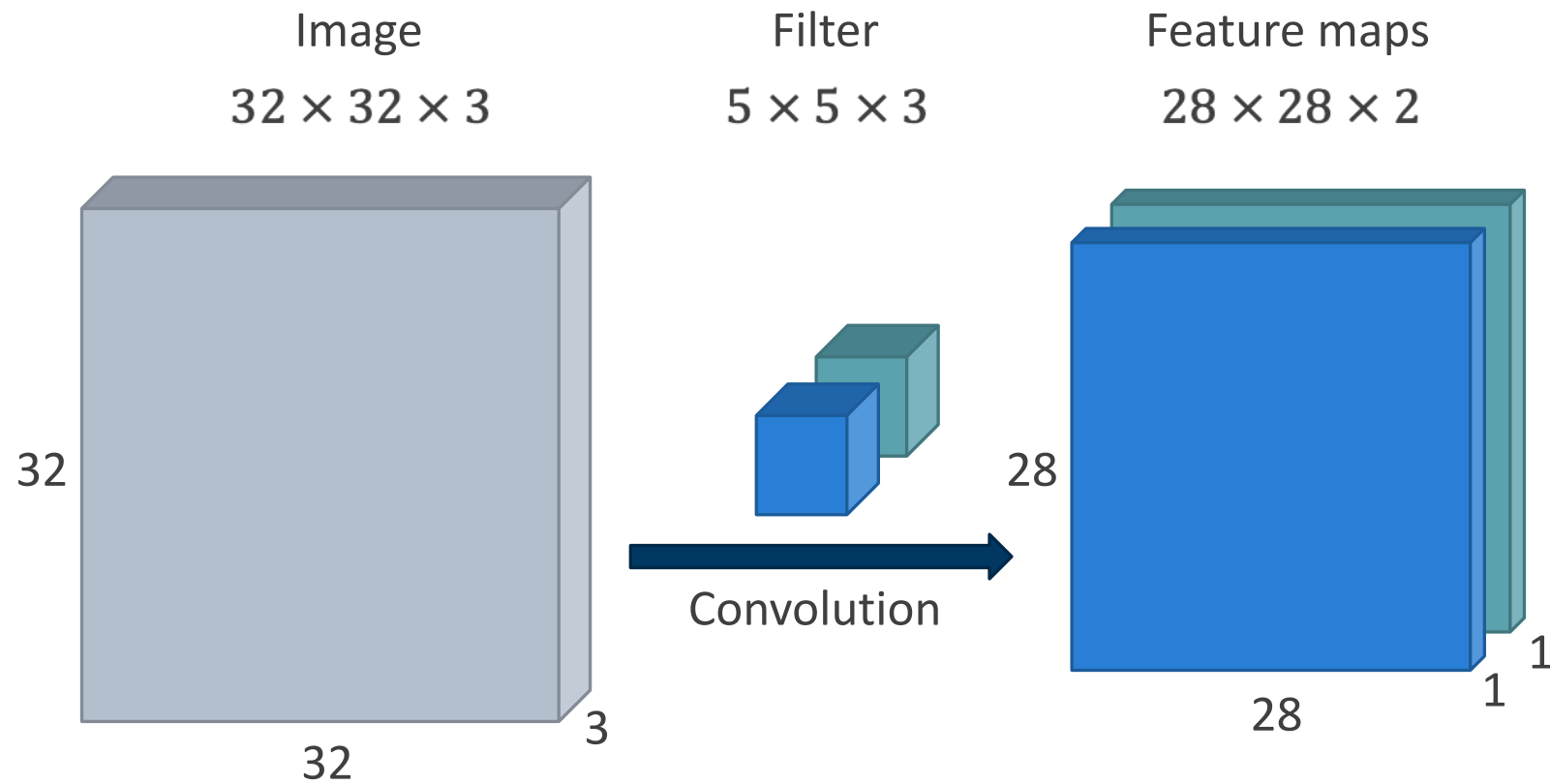
A convolution can be written as a matrix multiplication



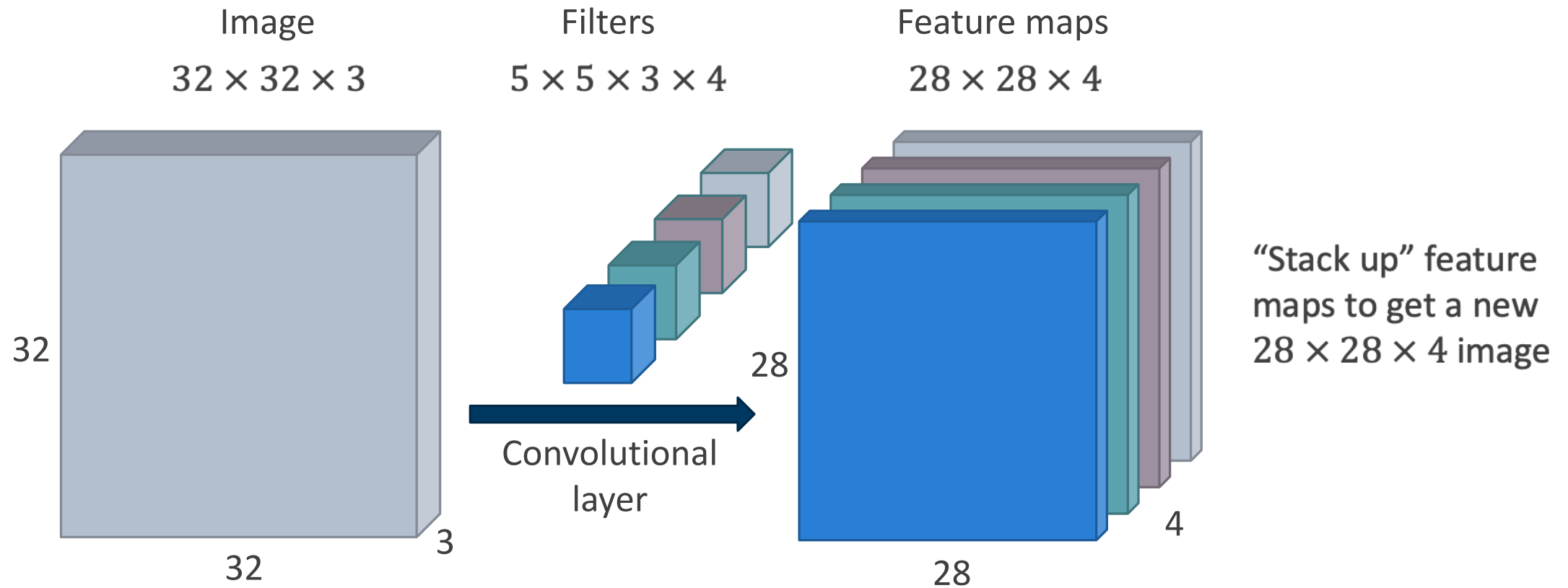
Example: 32×32 feature map / 3×3 kernel



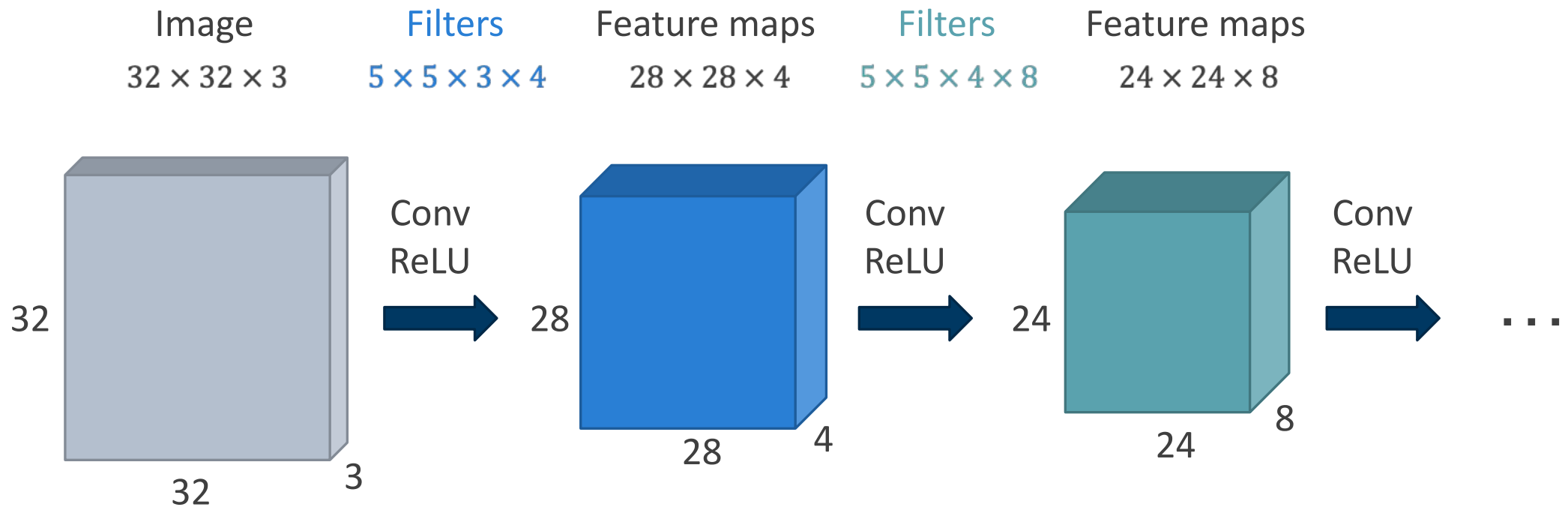
Convolution



Convolution



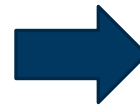
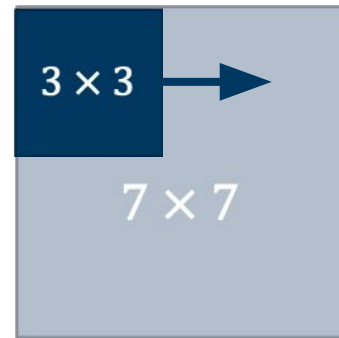
Convolutional neural network



A convnet is a sequence of convolution layers and activation functions

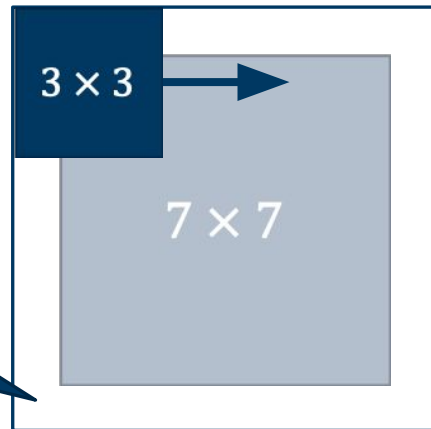
Zero padding

“valid”

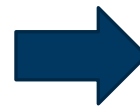


Feature maps “shrink” by $k - 1$ pixels after each convolution with $k \times k$ kernel

“same”



9 × 9



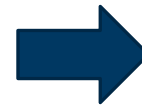
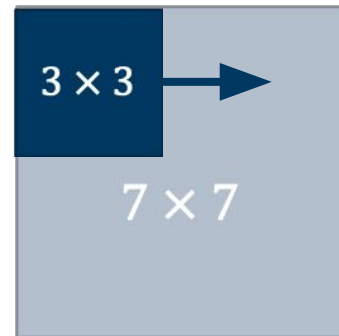
Feature maps keep the same size

Zero padding

Output size:

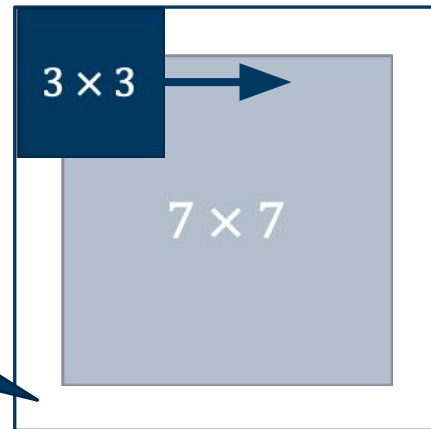
$$M = N + 2p - k + 1$$

“valid”



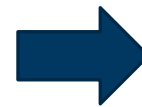
Feature maps
“shrink” by $k - 1$
pixels after each
convolution with
 $k \times k$ kernel

“same”



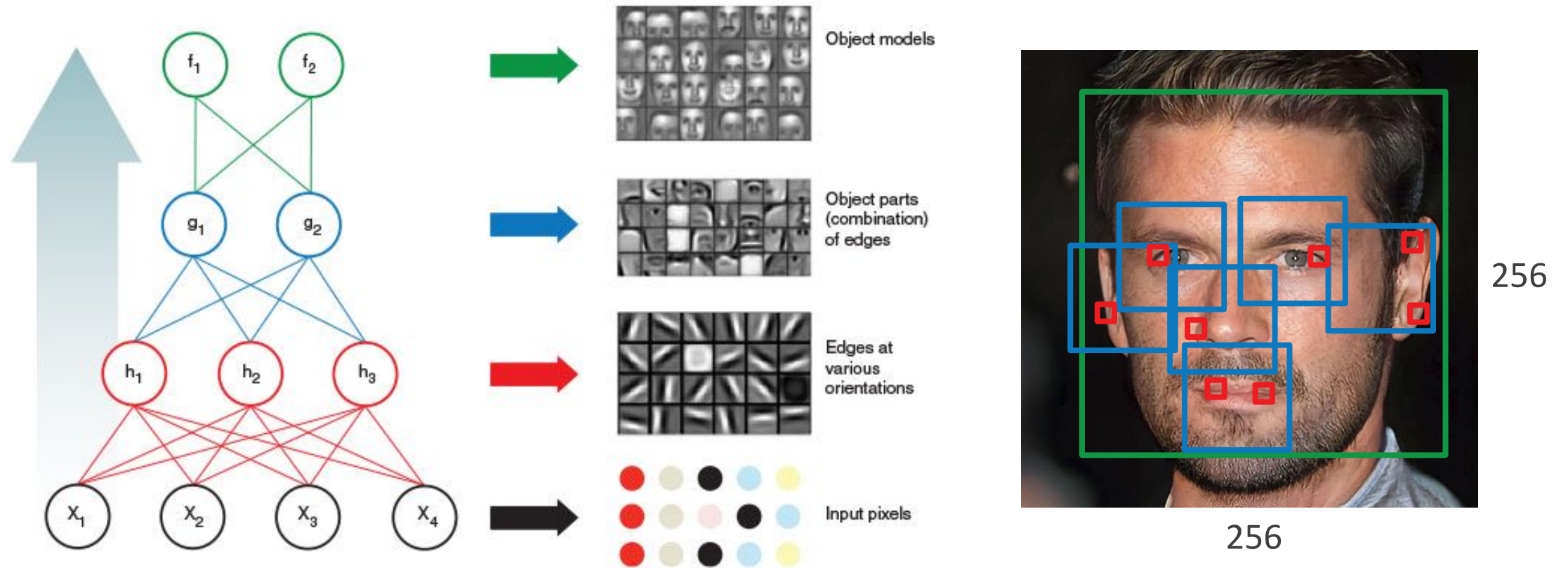
Adding zeros

9 × 9

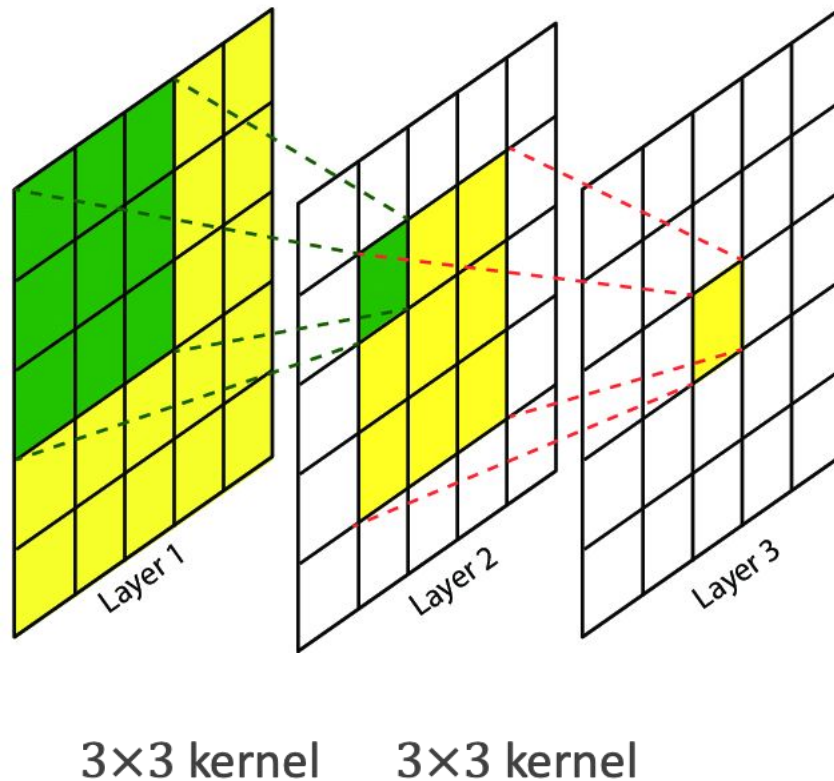


More general:
zero-padding by p
pixels on each side

Recall goal of NNs: repeated composition



Receptive fields



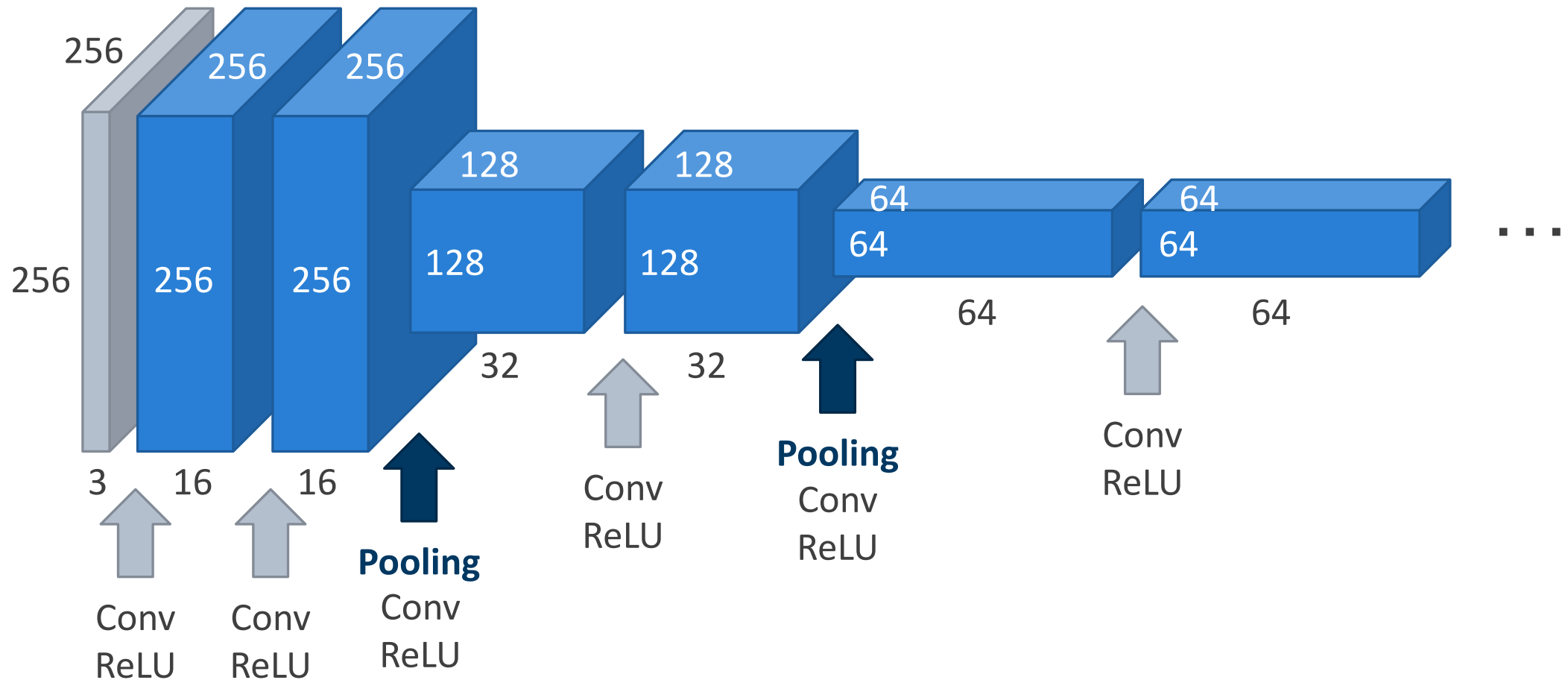
Receptive field sizes of a vanilla CNN:

$$\text{RF size} = 1 + \sum_{l=1}^L (k_l - 1)$$

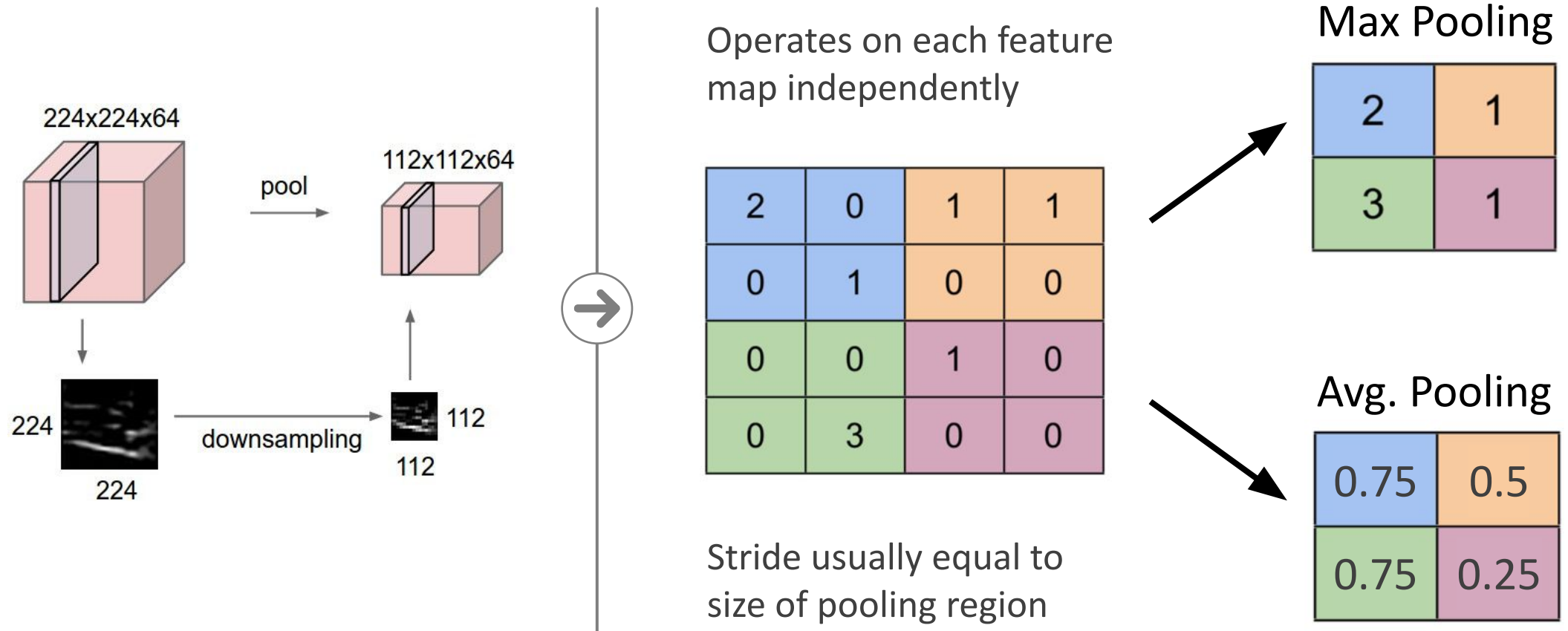
Suppose you want to achieve a 200×200 px receptive field with 5×5 kernels

-> **Would require 50 layers!**

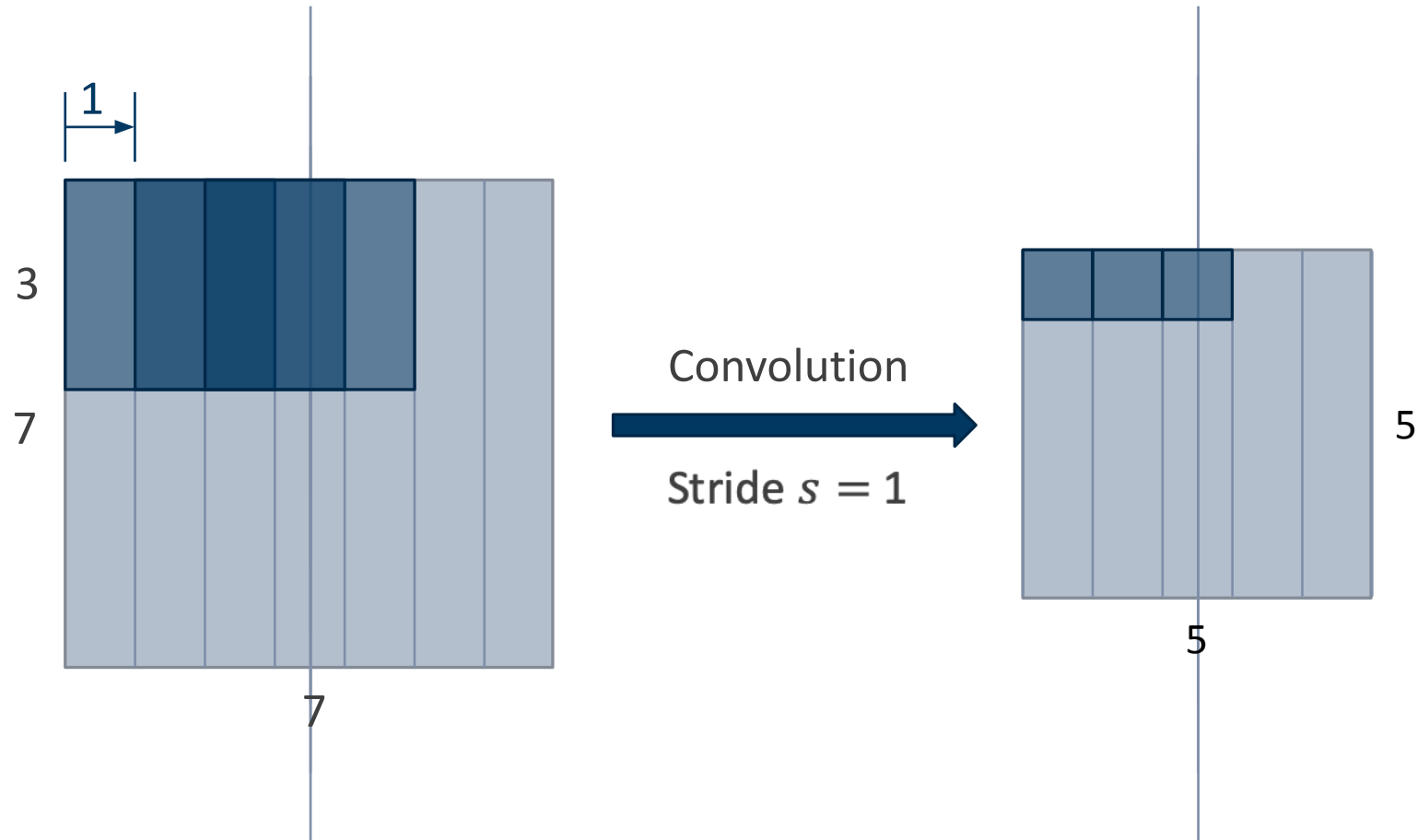
Convolutional neural network



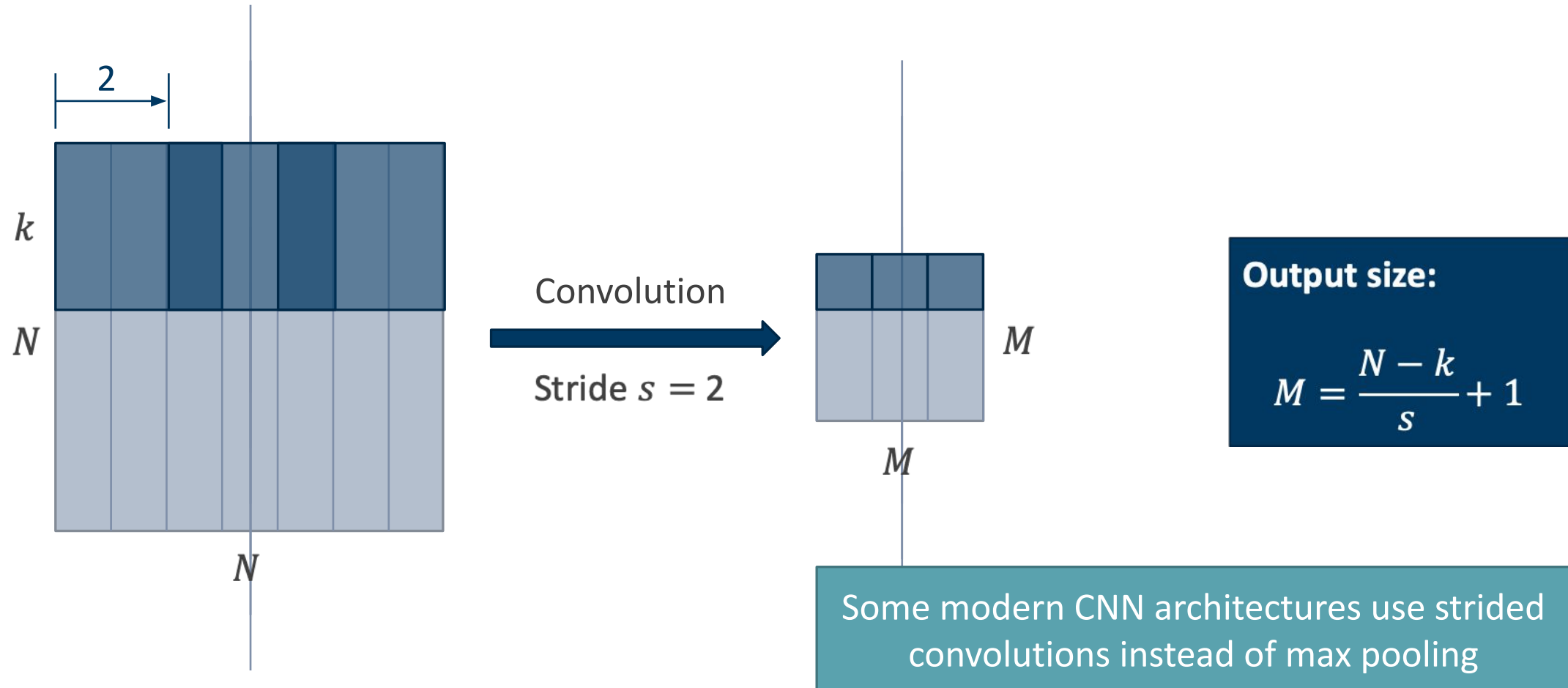
Reducing spatial dimensionality by pooling



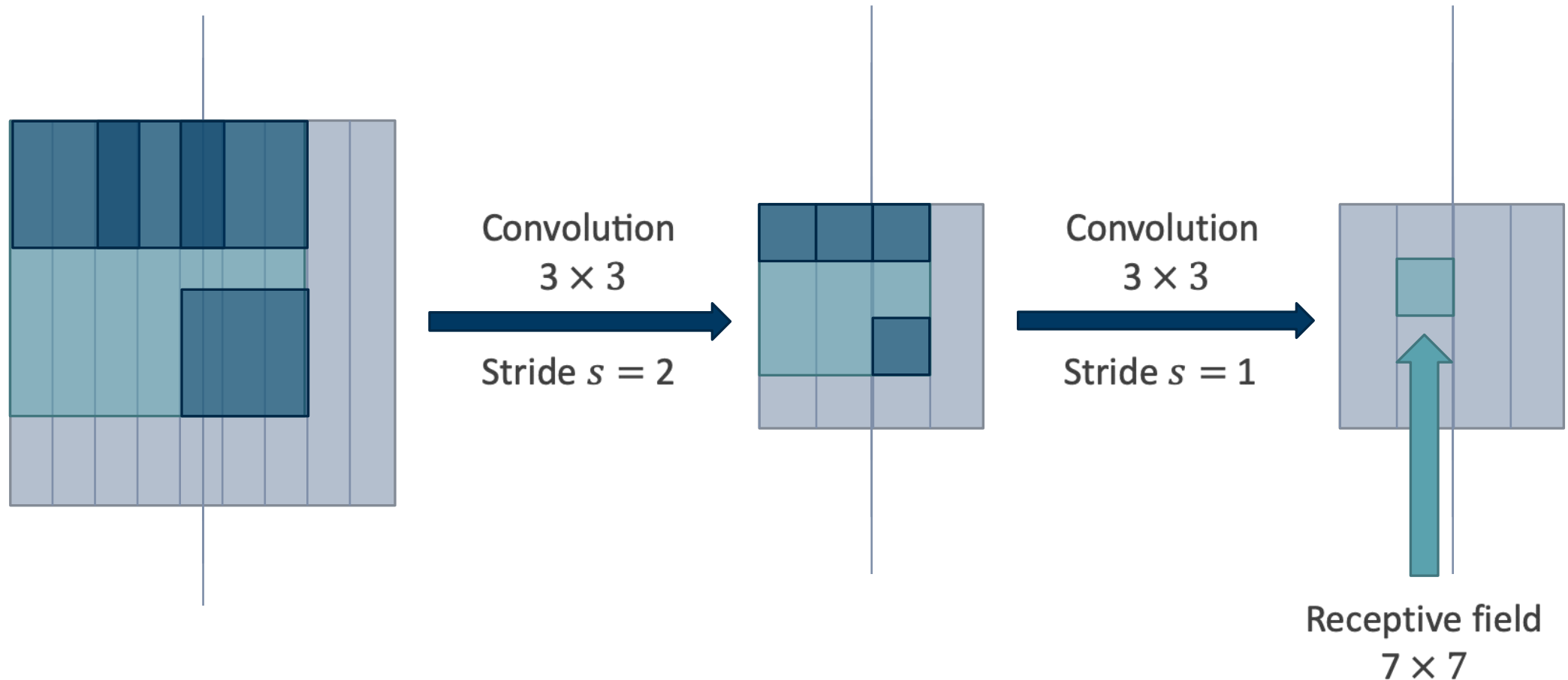
Reducing size by strided convolution



Reducing size by strided convolution



Receptive fields grow faster with pooling layers



Computing the receptive field

Single path networks

(simple feed forward networks
as we have discussed so far)

$$r_0 = \sum_{l=1}^L \left((k_l - 1) \prod_{i=1}^{l-1} s_i \right) + 1$$

k_l : kernel size of layer l

s_i : stride of layer i
(treat pooling and strided conv the same)

Computing the receptive field

Single path networks

(simple feed forward networks
as we have discussed so far)

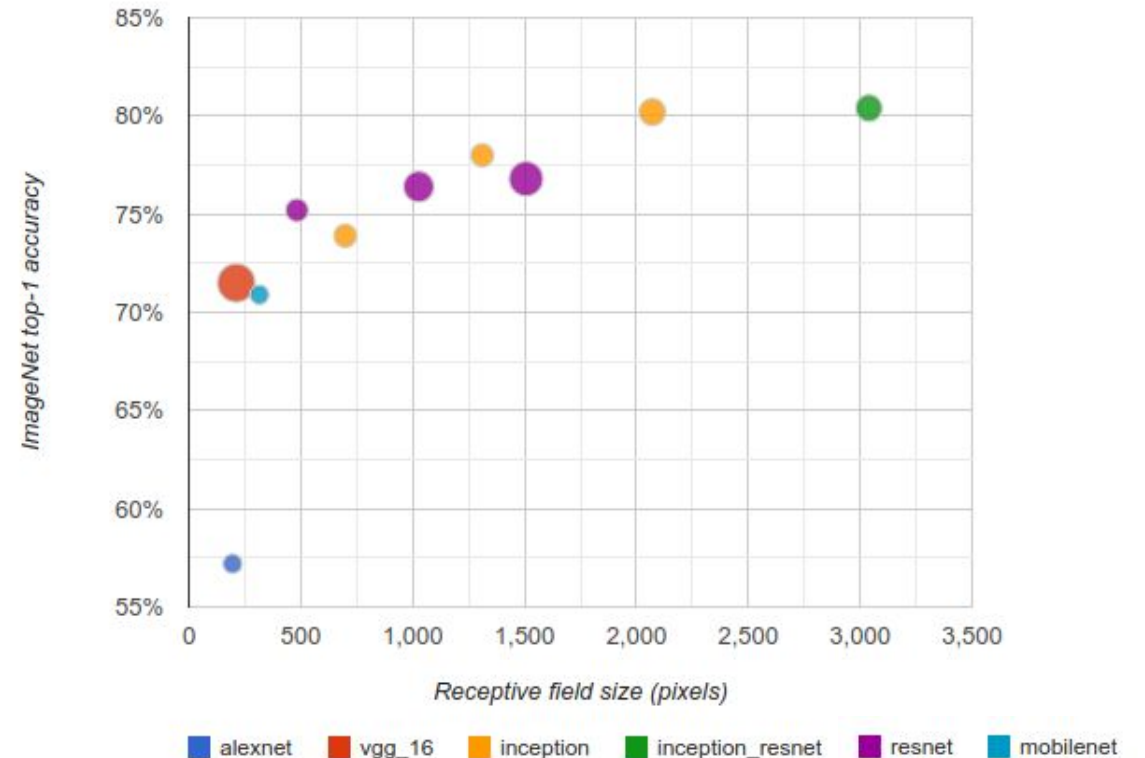
$$r_0 = \sum_{l=1}^L \left((k_l - 1) \prod_{i=1}^{l-1} s_i \right) + 1$$

k_l : kernel size of layer l

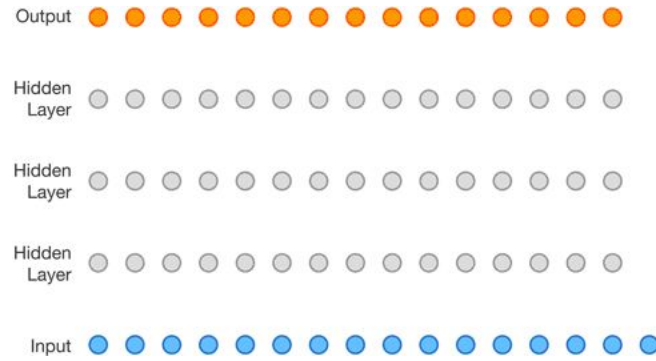
s_i : stride of layer i
(treat pooling and strided conv the same)

Modern architectures

multiple path, receptive field
= max(paths)

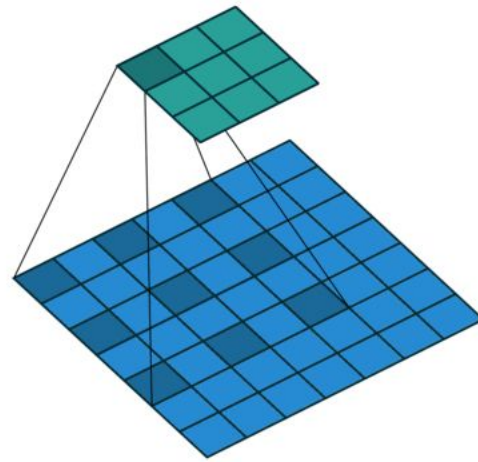


Dilated convolutions



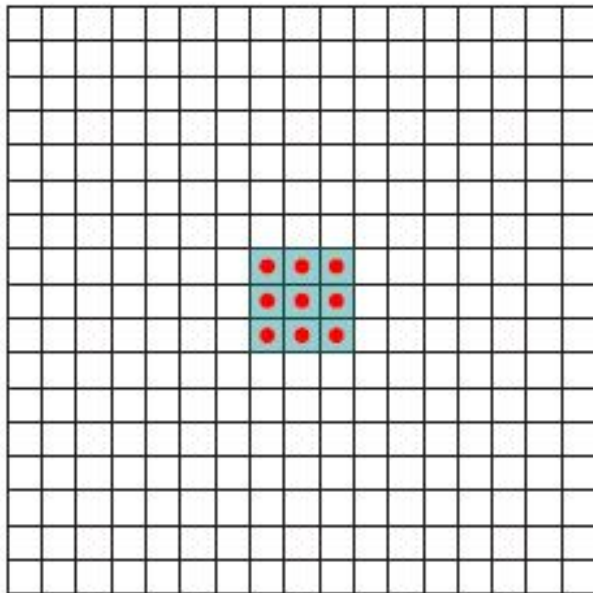
alternative name: “atrous” convolutions

Dilated convolutions in 2D

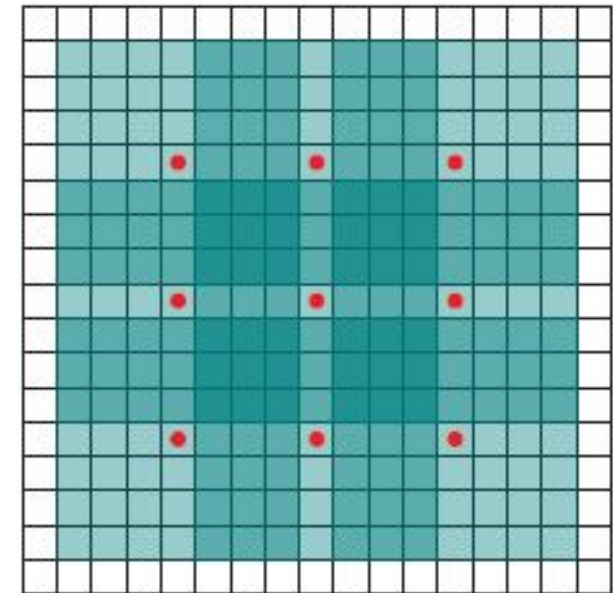
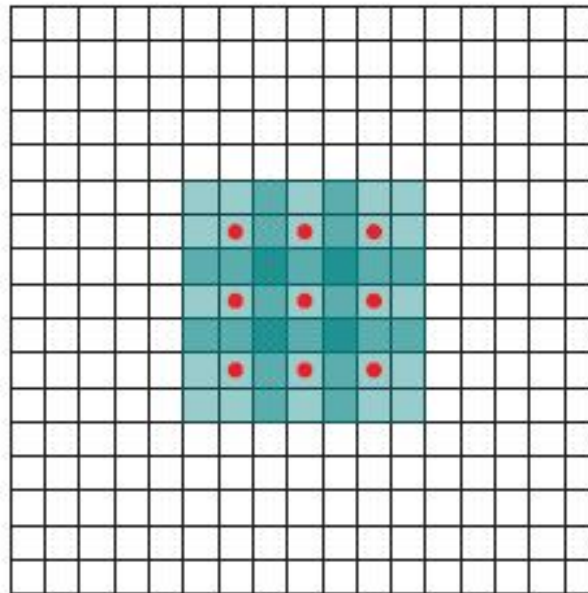


alternative name: “atrous” convolutions

Dilated convolutions increase receptive field sizes without reducing resolution

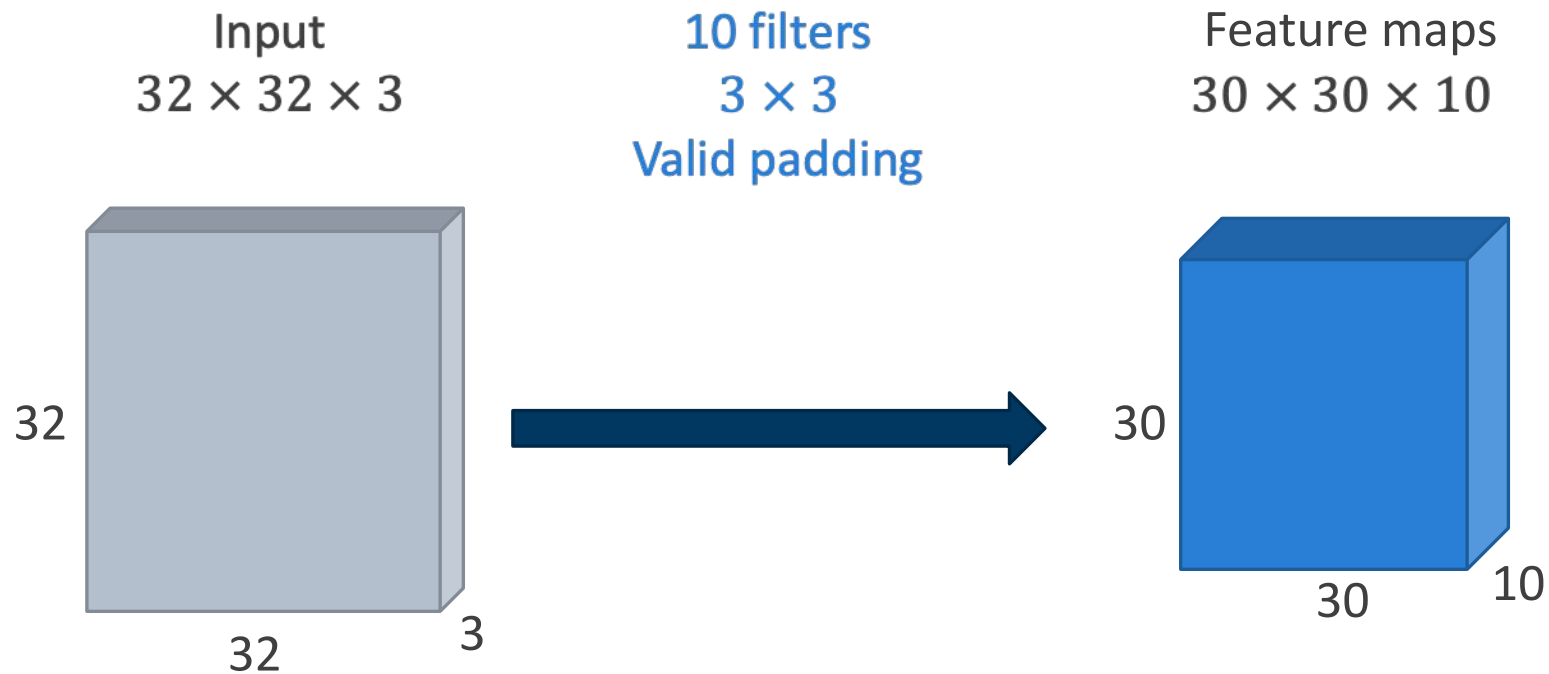


Convolution
 3×3
Dilation factor 2



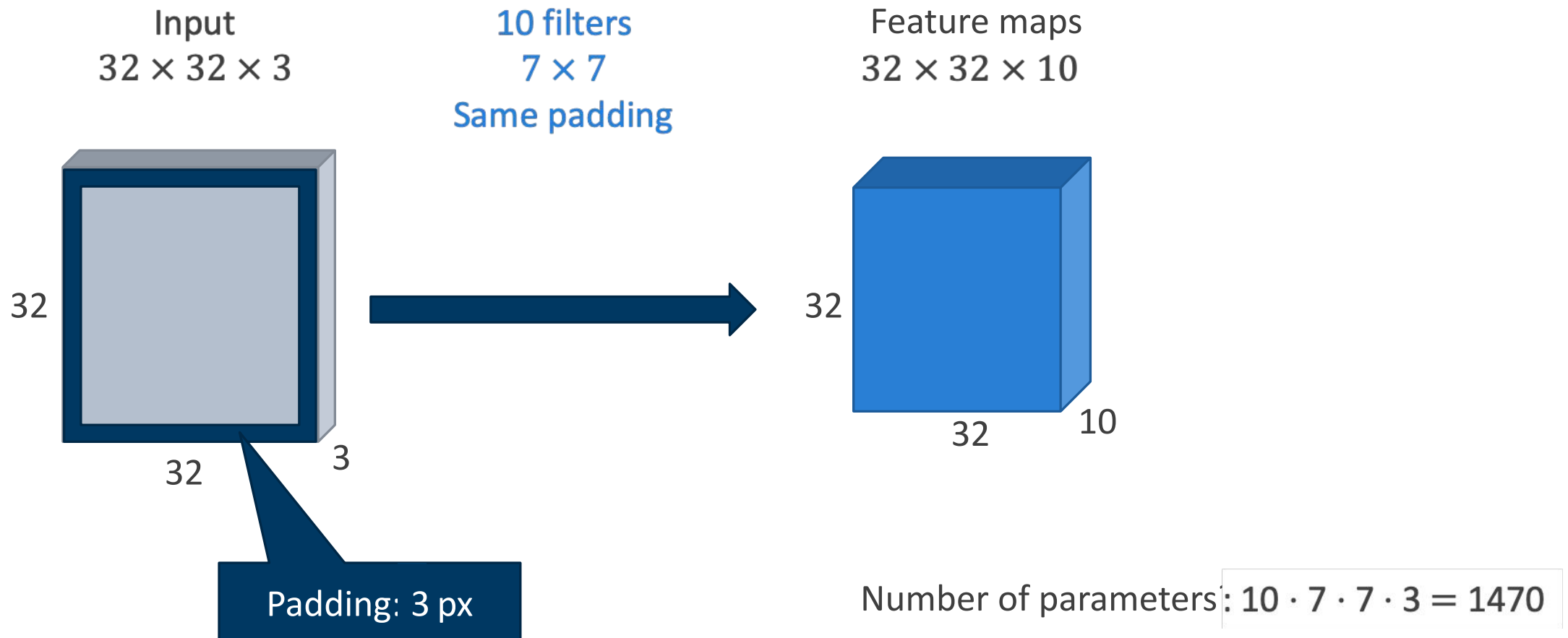
Convolution
 3×3
Dilation factor 4

Let's have a look at dimensions

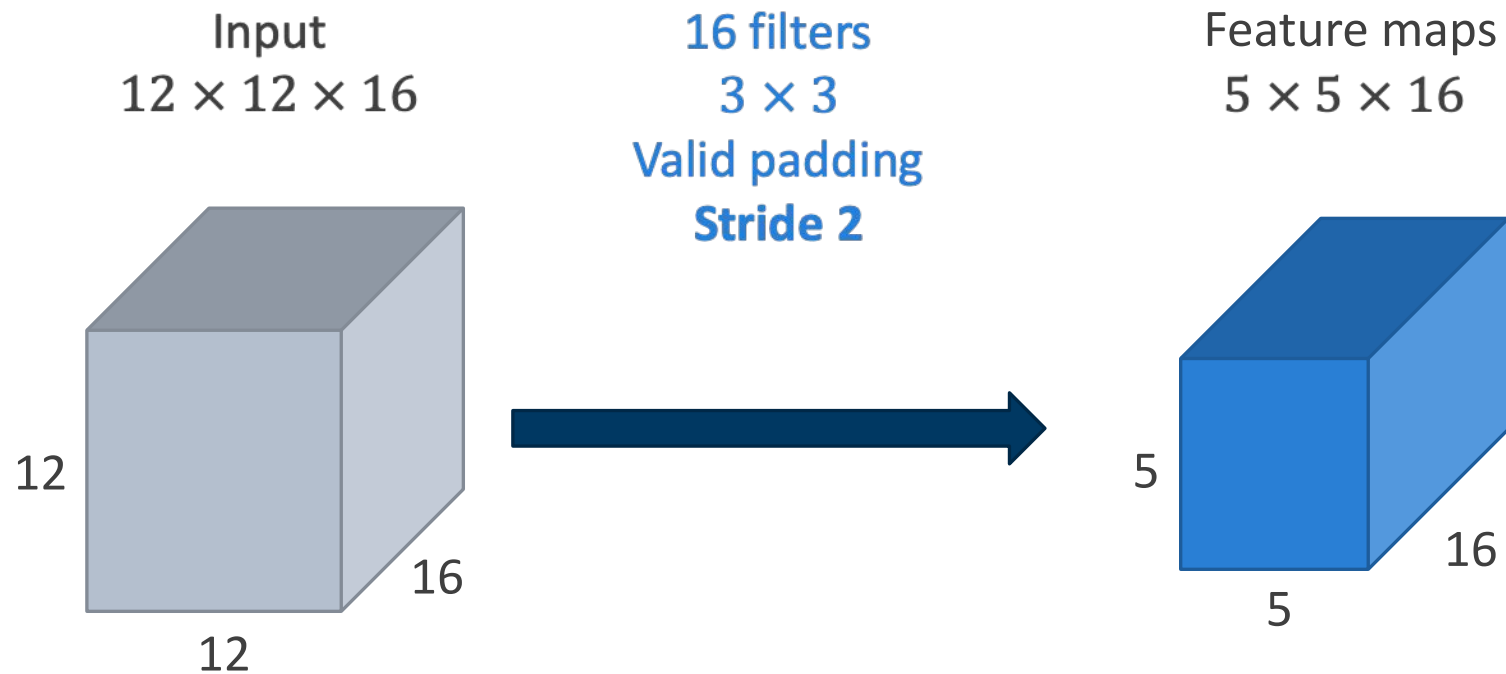


Number of parameters: $10 \cdot 3 \cdot 3 \cdot 3 = 270$

Let's have a look at dimensions

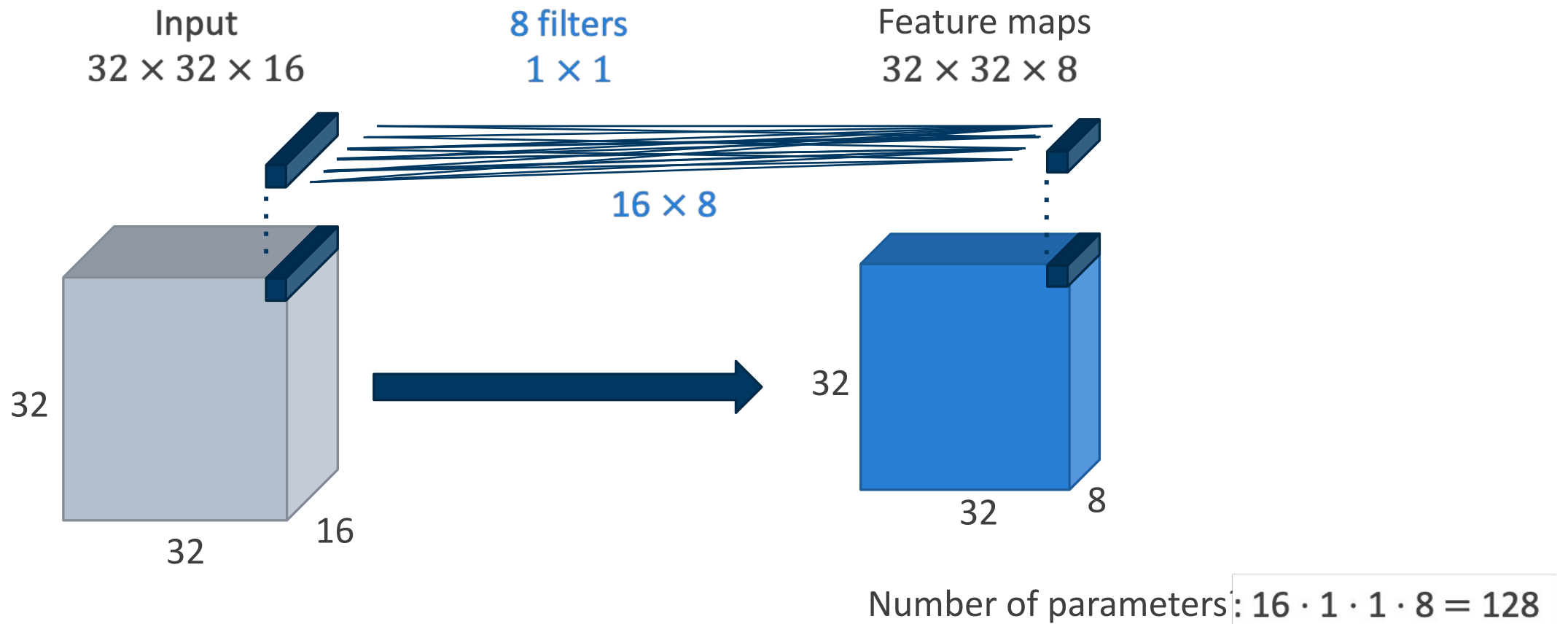


Let's have a look at dimensions

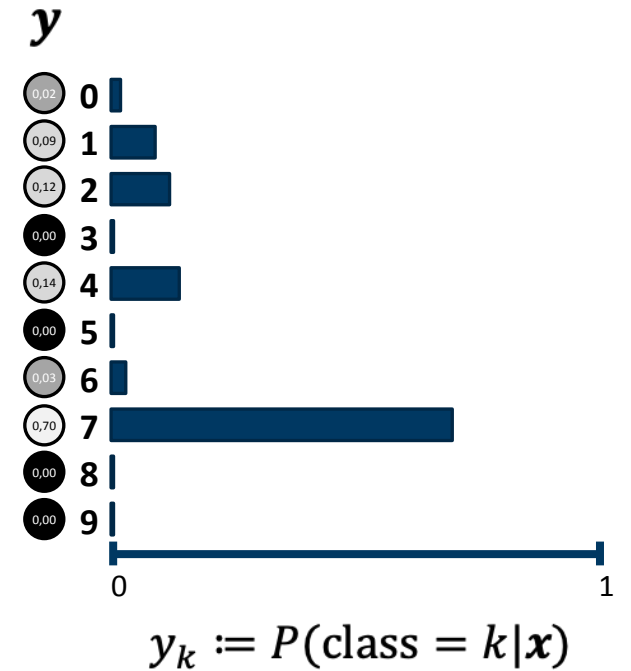
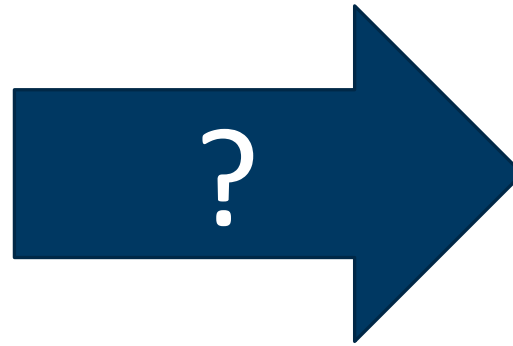
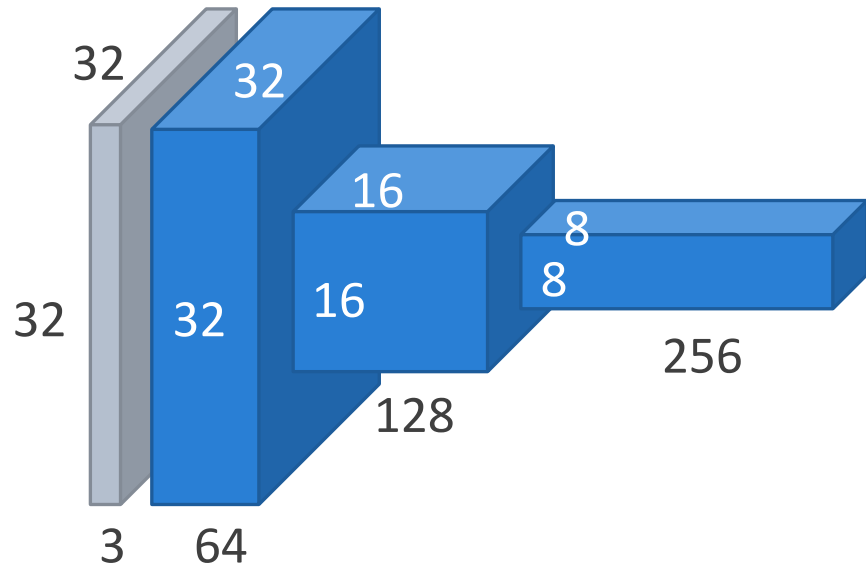


Number of parameters: $16 \cdot 3 \cdot 3 \cdot 16 = 2304$

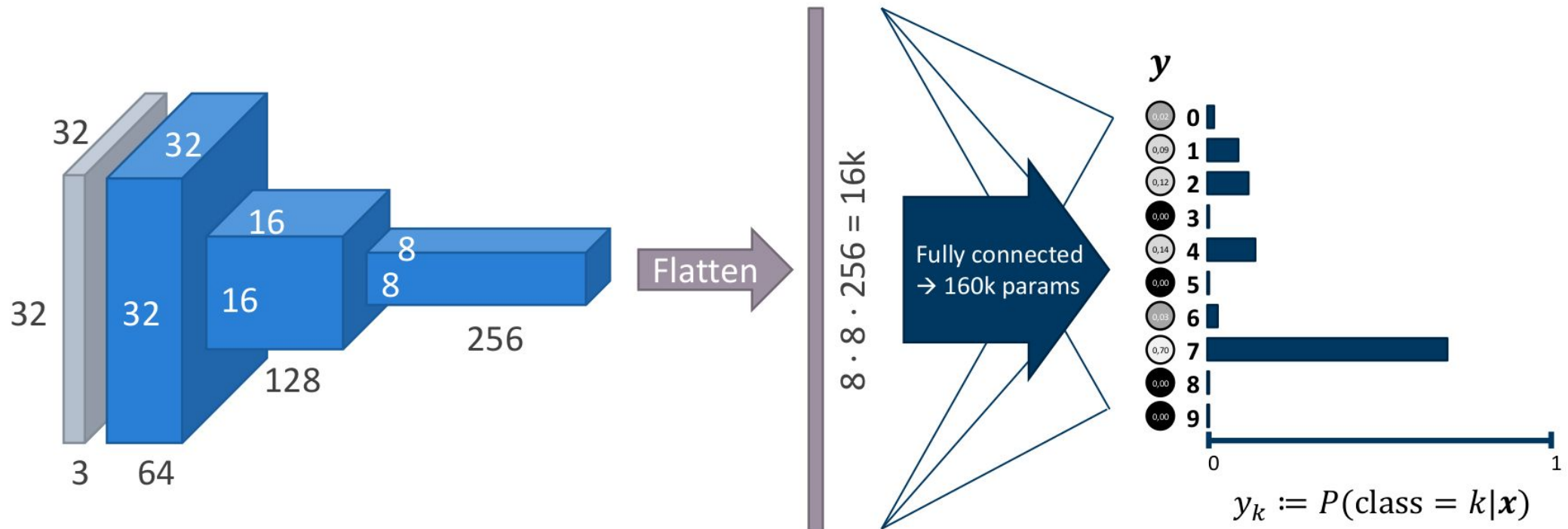
1 × 1 convolutions?



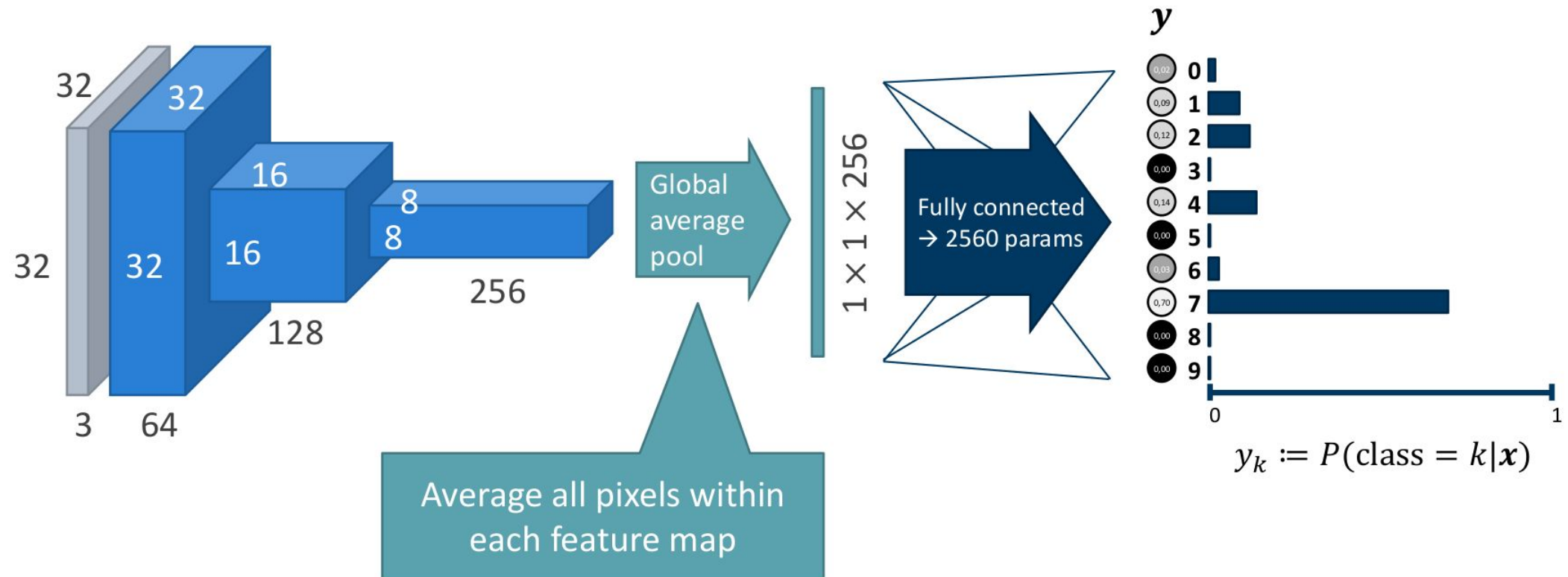
Putting it all together



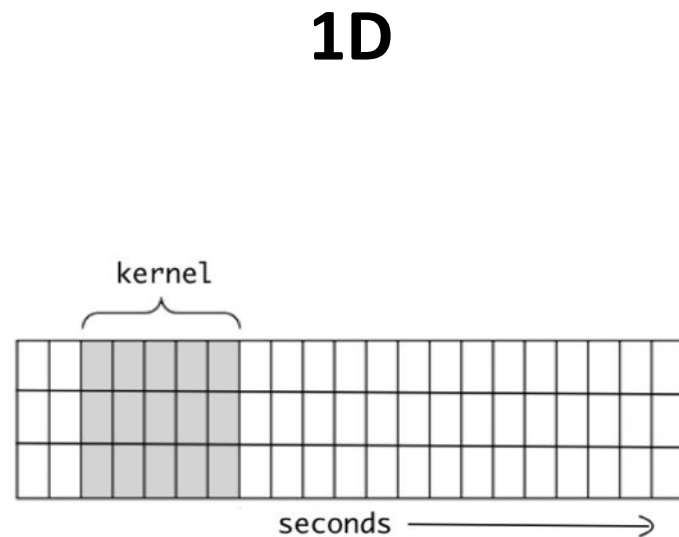
Flatten last convolutional feature maps before dense layer



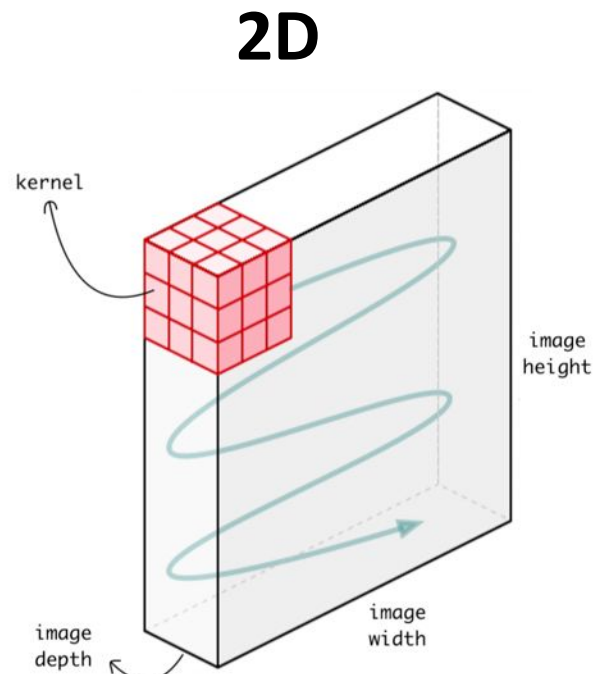
Global average pool of last convolutional feature maps before dense layer



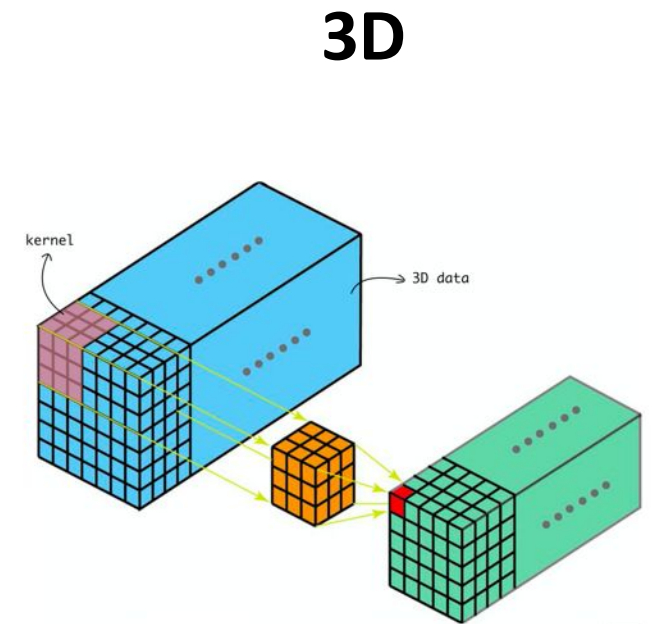
CNNs can be used for 1D, 2D as well as 3D data



Timeseries



Images



Volumetric images + movies

4D convolutions?



Figure 1: An example of 3D video: 3D scenes at different time steps. Best viewed on display.

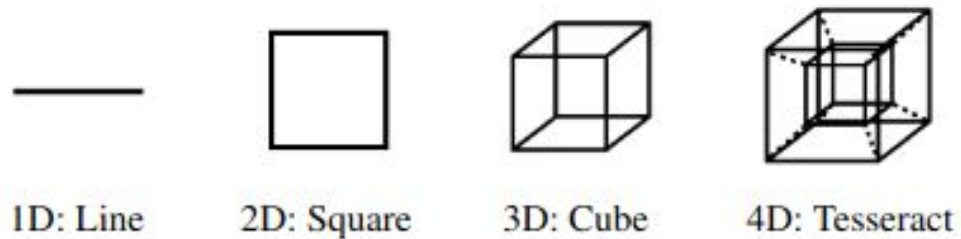


Figure 2: 2D projections of hypercubes in various dimensions

4D Spatio-Temporal ConvNets: Minkowski Convolutional Neural Networks

Christopher Choy
chrischoy@stanford.edu

JunYoung Gwak
jgwak@stanford.edu

Silvio Savarese
ssilvio@stanford.edu

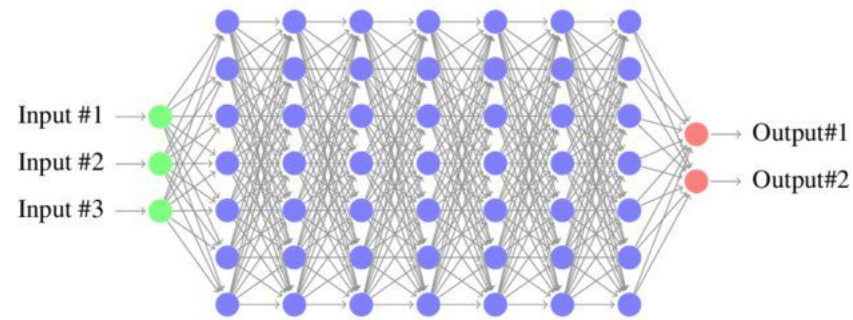
<https://arxiv.org/abs/1904.08755>

Batch normalization

Weight initialization

Remember last lecture:

- Sigmoid or Tanh: vanishing gradients
- ReLU: “dead ReLU”



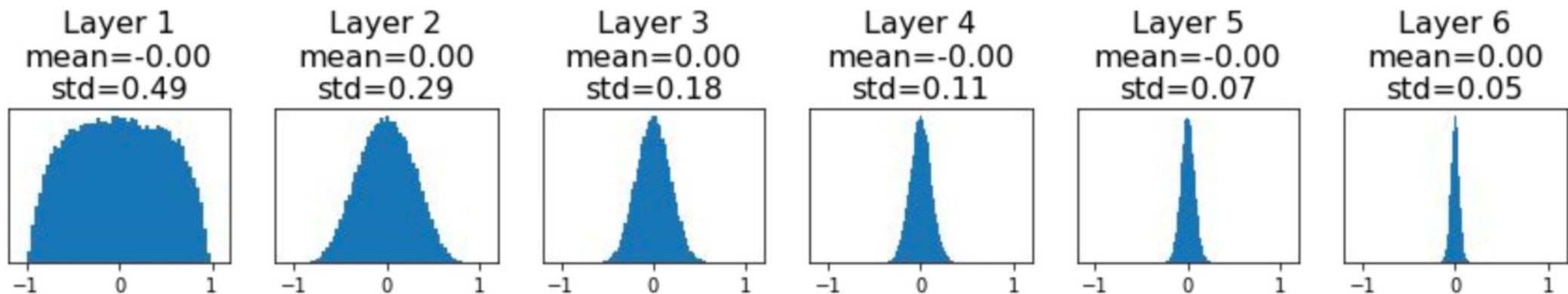
```
W = 0.01 * np.random.randn(D_in, D_out)
```

Weight initialization: activation statistics

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for D_in, D_out in zip(dims[:-1], dims[1:]):
    W = 0.01 * np.random.randn(D_in, D_out)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

For deeper networks:
All activations tend to
zero in deeper layers

-> No gradients
-> No learning

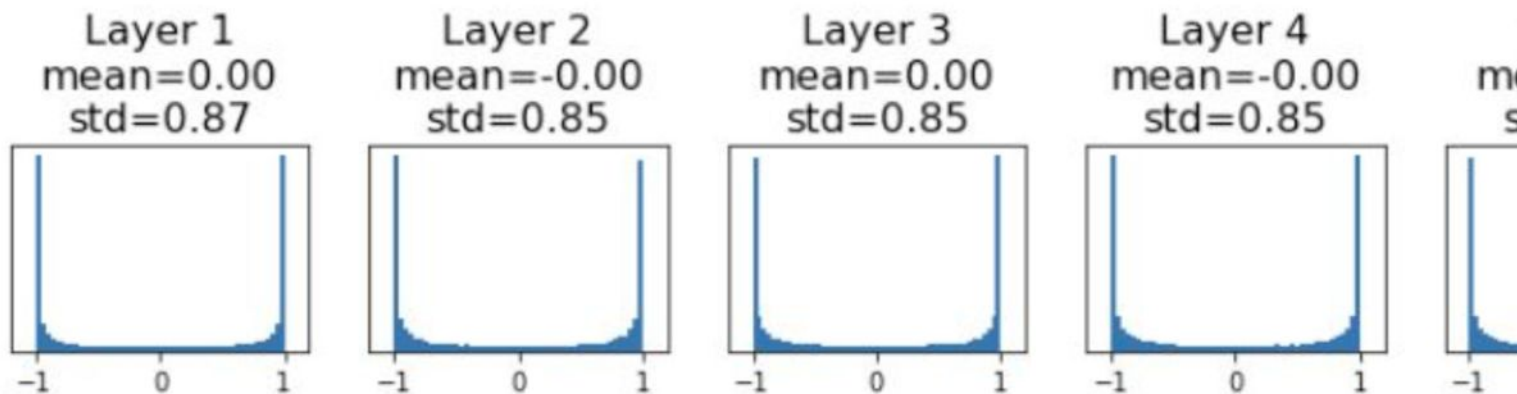


Weight initialization: activation statistics

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for D_in, D_out in zip(dims[:-1], dims[1:]):
    W = 0.05 * np.random.randn(D_in, D_out)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

For deeper networks:
All activations saturate

-> No gradients
-> No learning

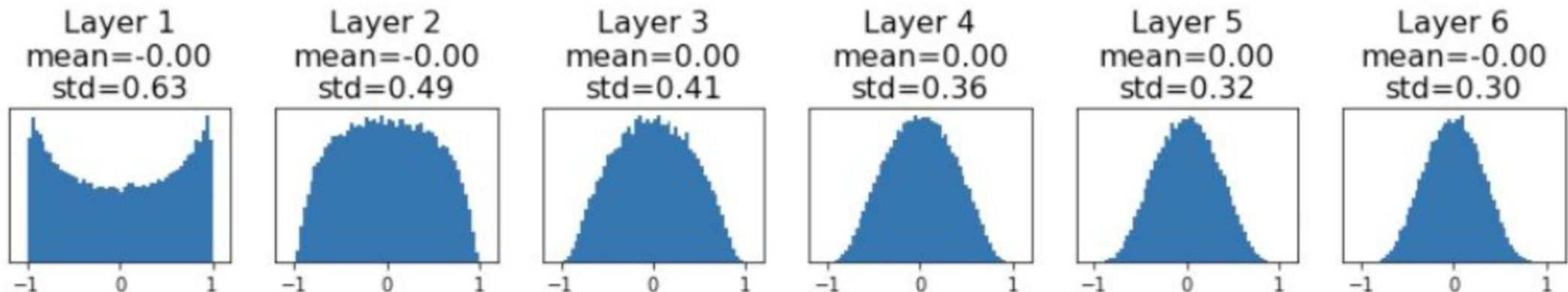


In case of ReLU
Activations/gradients
explode

Weight initialization: Xavier initialization

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for D_in, D_out in zip(dims[:-1], dims[1:]):
    W = np.random.randn(D_in, D_out) / np.sqrt(D_in)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

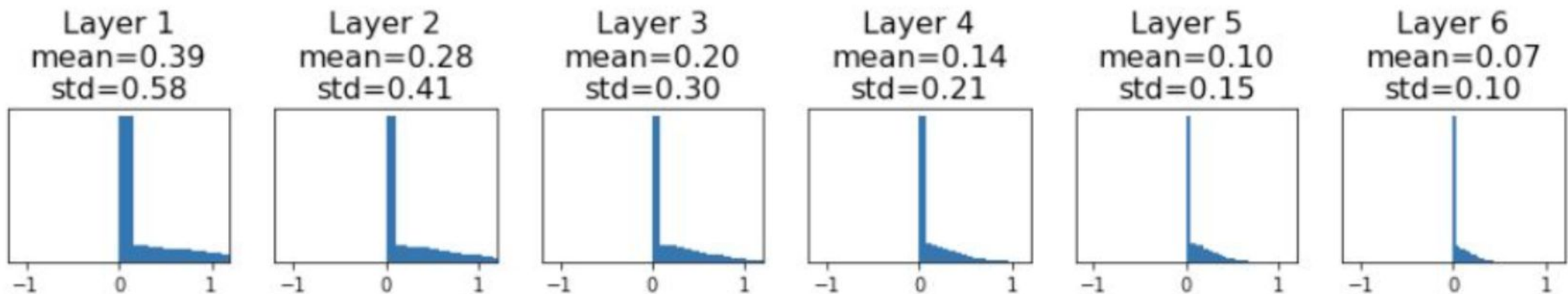
Glorot & Bengio, AISTATS 2010



Weight initialization: What about ReLU?

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for D_in, D_out in zip(dims[:-1], dims[1:]):
    W = np.random.randn(D_in, D_out) / np.sqrt(D_in)
    x = np.maximum(0, x.dot(W))
    hs.append(x)
```

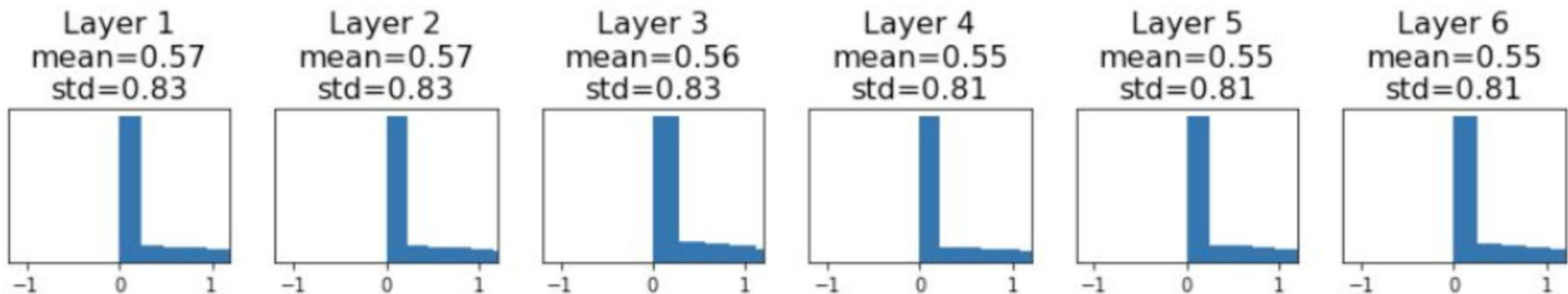
Glorot & Bengio, AISTATS 2010



Weight initialization: He / MSRA initialization

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for D_in, D_out in zip(dims[:-1], dims[1:]):
    W = np.random.randn(D_in, D_out) / np.sqrt(2 / D_in)
    x = np.maximum(0, x.dot(W))
    hs.append(x)
```

Glorot & Bengio, AISTATS 2010



Proper initialization is an active area of research...

Understanding the difficulty of training deep feedforward neural networks
by Glorot and Bengio, 2010

Exact solutions to the nonlinear dynamics of learning in deep linear neural networks
by Saxe et al, 2013

Random walk initialization for training very deep feedforward networks
by Sussillo and Abbott, 2014

Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification
by He et al., 2015

Data-dependent Initializations of Convolutional Neural Networks
by Krähenbühl et al., 2015

All you need is a good init,
by Mishkin and Matas, 2015

Fixup Initialization: Residual Learning Without Normalization
by Zhang et al, 2019

The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks
by Frankle and Carbin, 2019

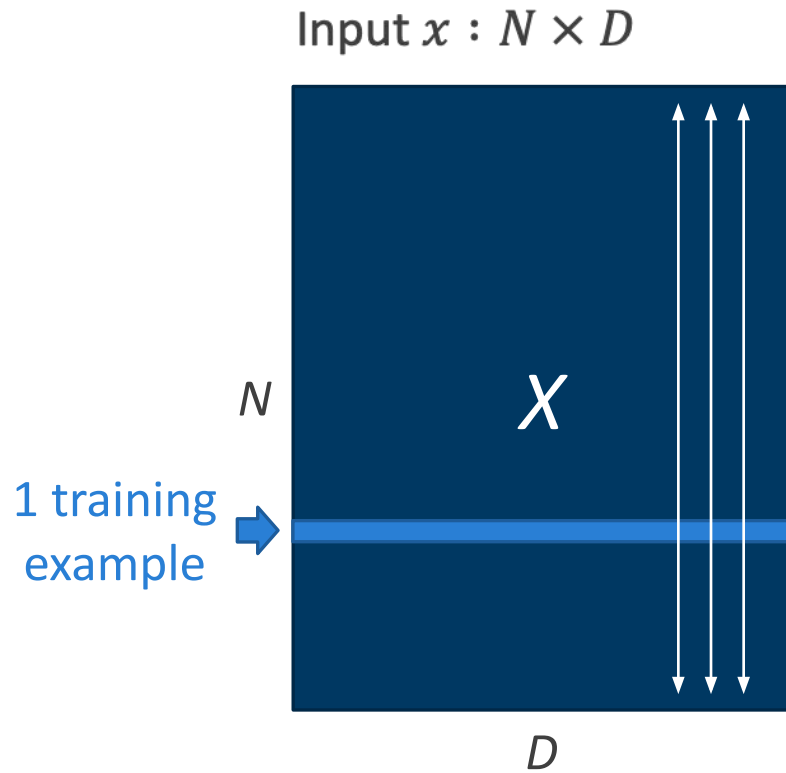
Batch normalization: motivation



It is common practice to normalize the inputs to have zero mean, unit variance
Do the same for intermediate layers' activations

$$\text{For each layer: } \hat{x} = \frac{x - E[x]}{\sqrt{\text{Var}[x]}}$$

Batch normalization



$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j}$$

Per-channel mean (D -dim)

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2$$

Per-channel variance (D -dim)

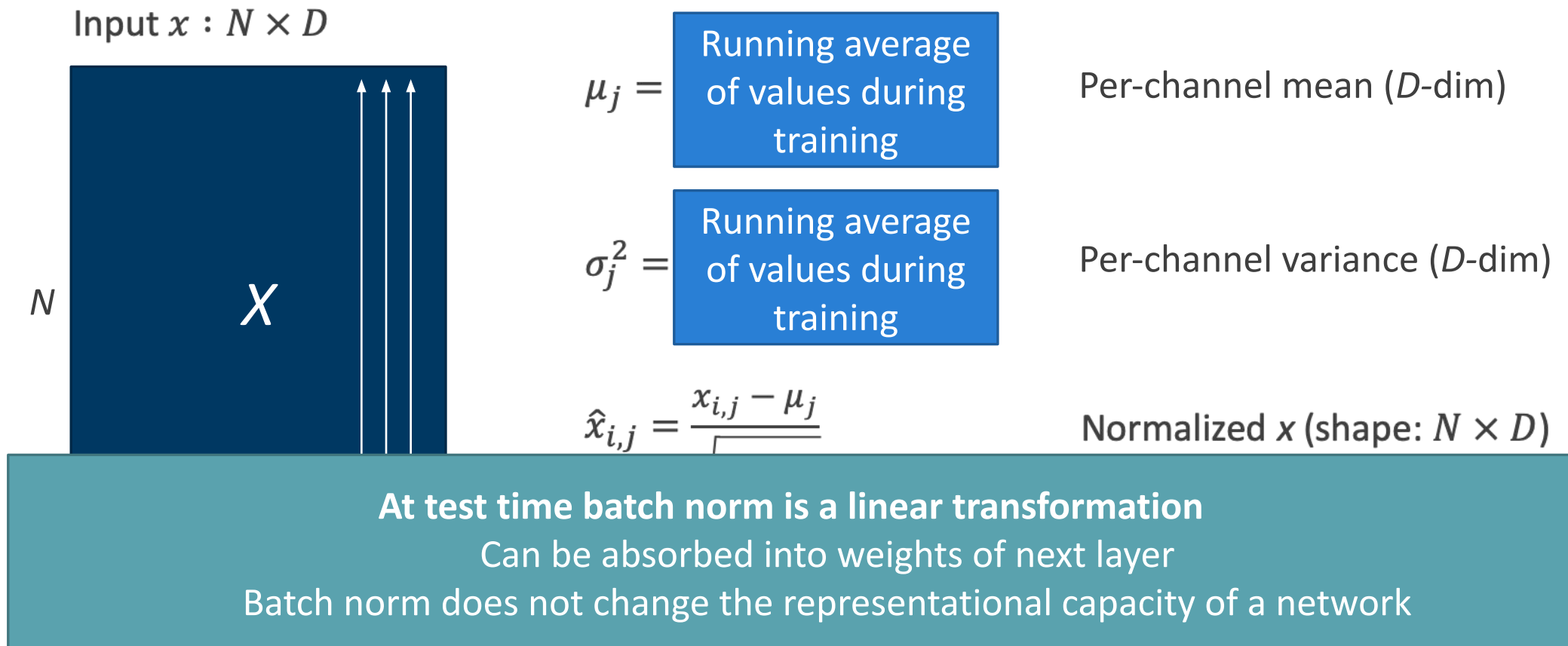
$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

Normalized x (shape: $N \times D$)

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$

β_j, γ_j learnable parameters

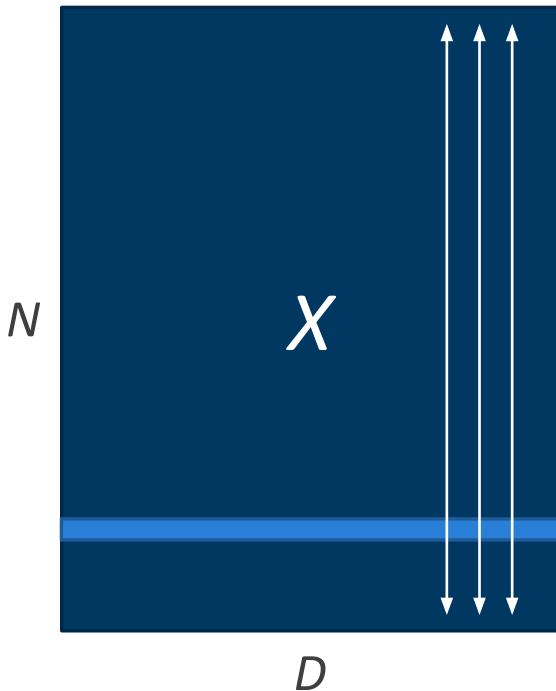
Batch normalization at test time



Batch norm: CNNs

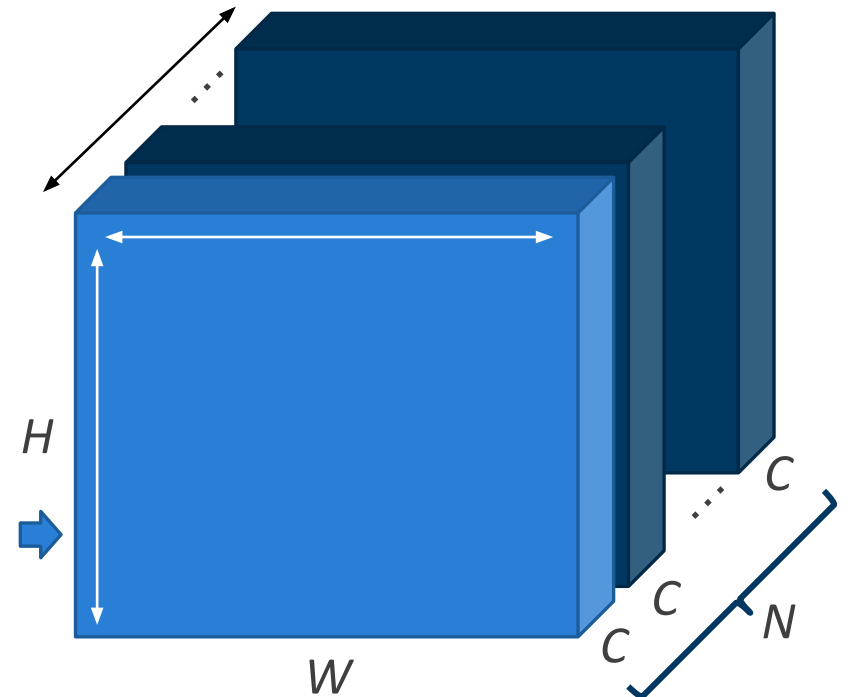
Input x Fully connected network

$N \times D$
↓ Normalize
 $\mu, \sigma, \beta, \gamma$ $1 \times D$



CNN

$N \times H \times W \times C$
↓ Normalize
 $1 \times 1 \times 1 \times C$

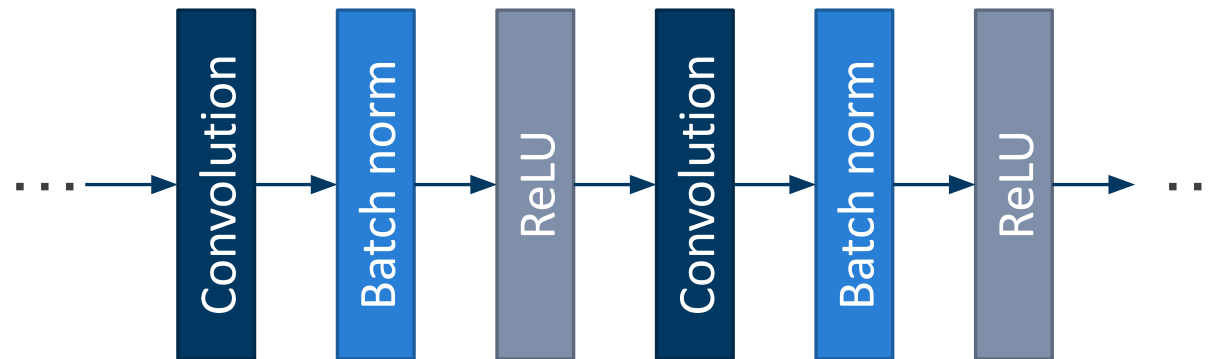


1 training example

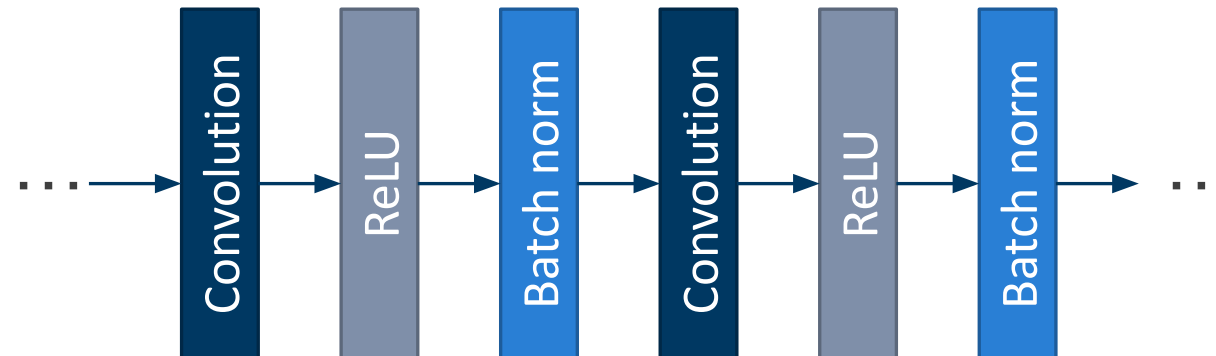
Where to place batch norm layer?

Original proposal

Ioffe & Szegedy 2015



Alternative placement



Layer normalization

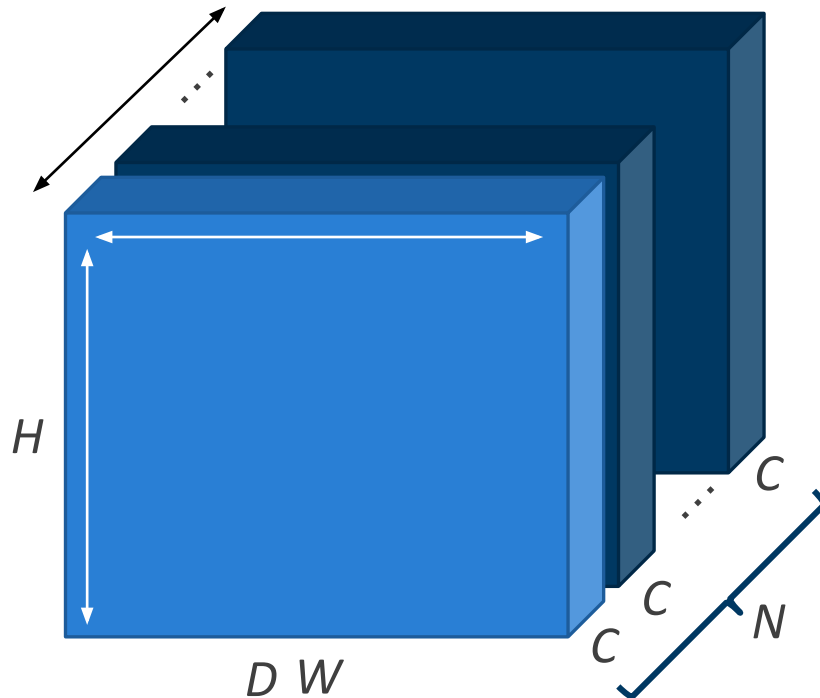
Same behavior at training and test time!

Batch norm

Input x $N \times H \times W \times C$

$\mu, \sigma, \beta, \gamma$ $1 \times 1 \times 1 \times C$

Normalize

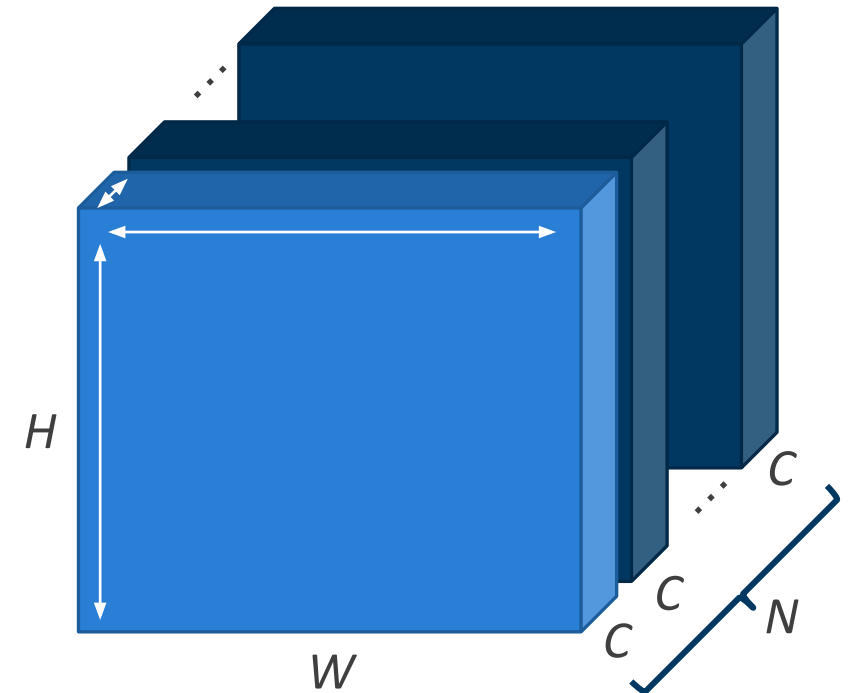


Layer norm

Input x $N \times H \times W \times C$

$N \times 1 \times 1 \times 1$

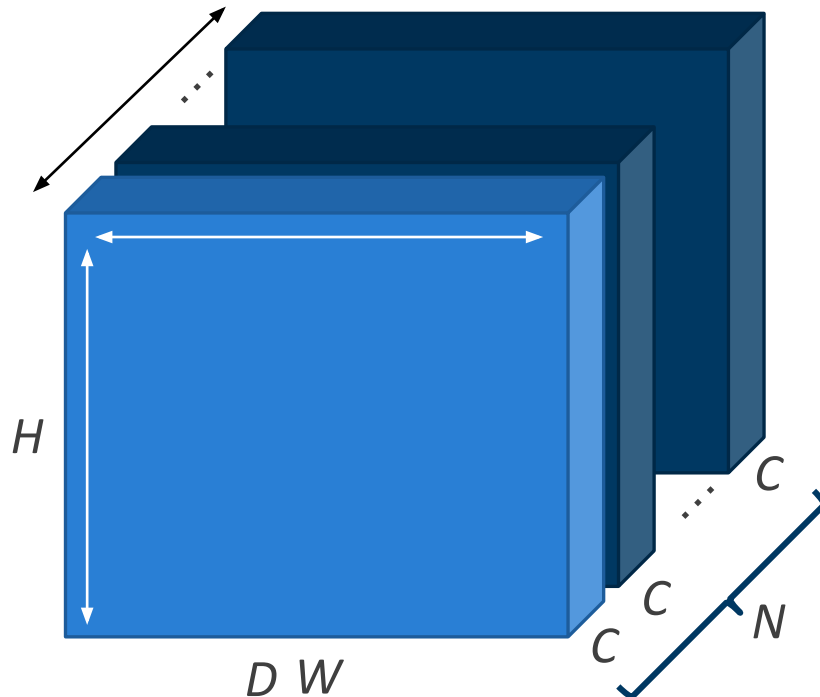
Normalize



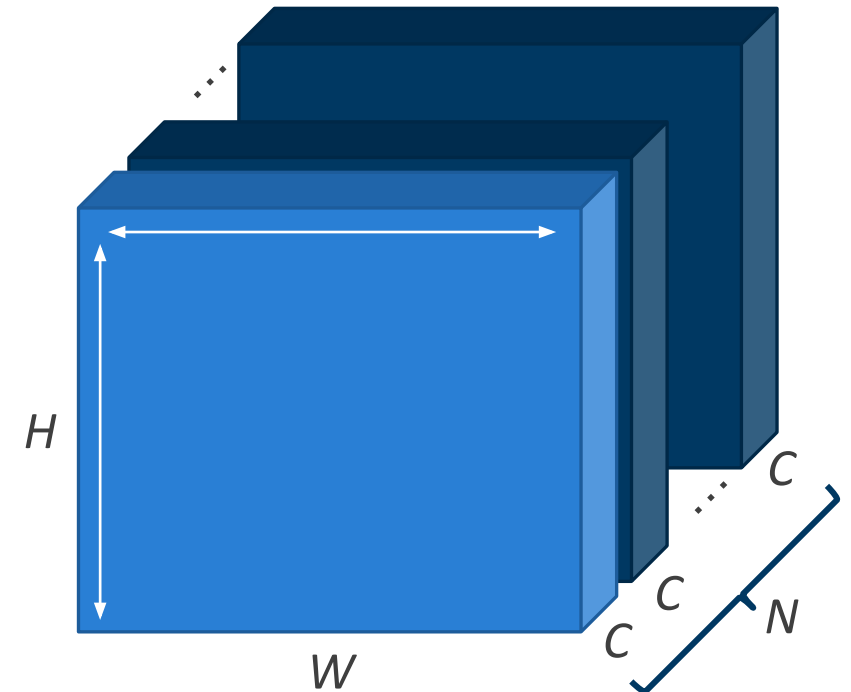
Instance normalization

Same behavior at training and test time!

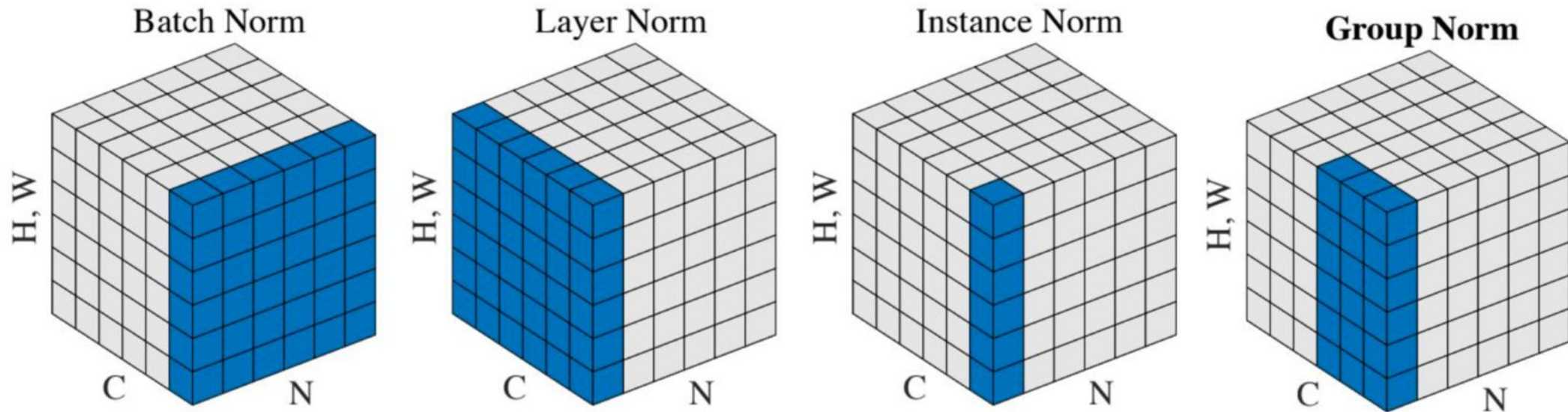
Batch norm
Input x $N \times H \times W \times C$
 $\downarrow \quad \downarrow \quad \downarrow$ Normalize
 $\mu, \sigma, \beta, \gamma$ $1 \times 1 \times 1 \times C$



Instance norm
 $N \times H \times W \times C$
 $\downarrow \quad \downarrow$ Normalize
 $N \times 1 \times 1 \times C$

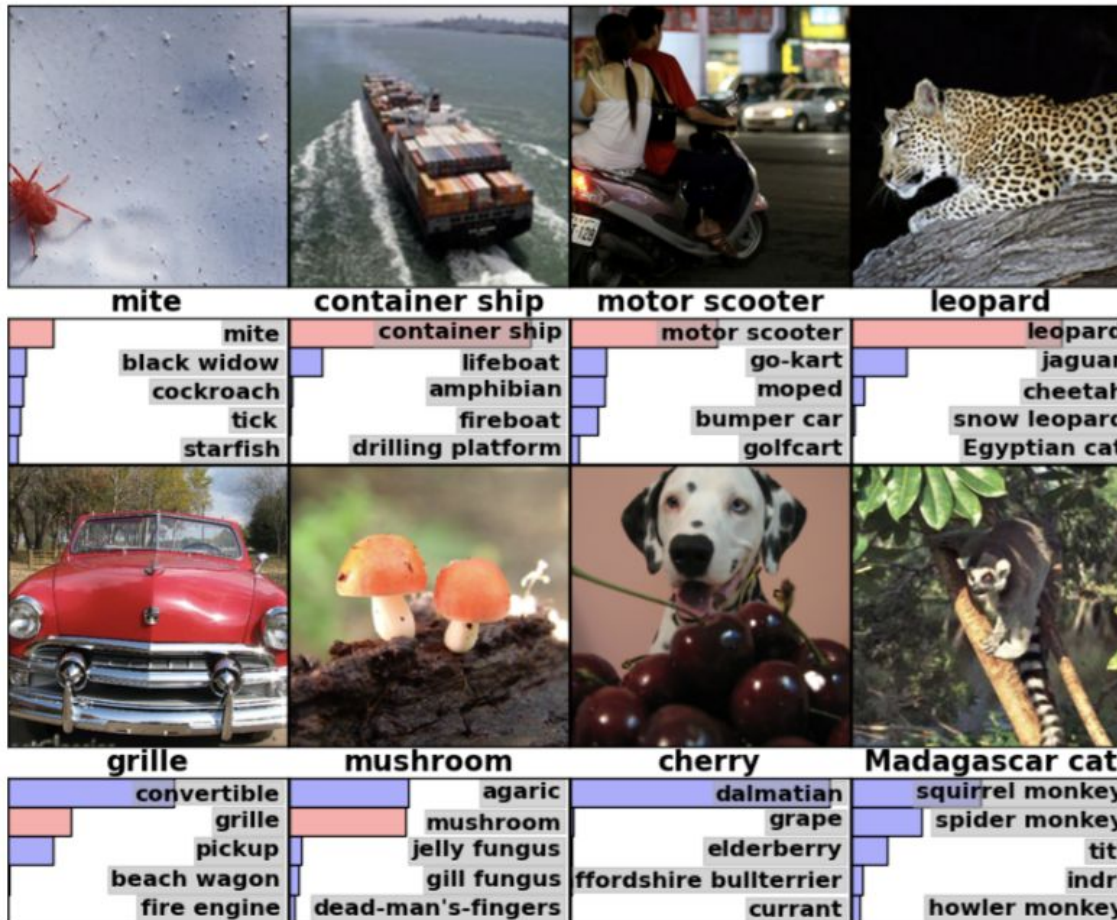


Comparison of normalization layers



CNN architectures

IMAGENET large-scale visual recognition challenge

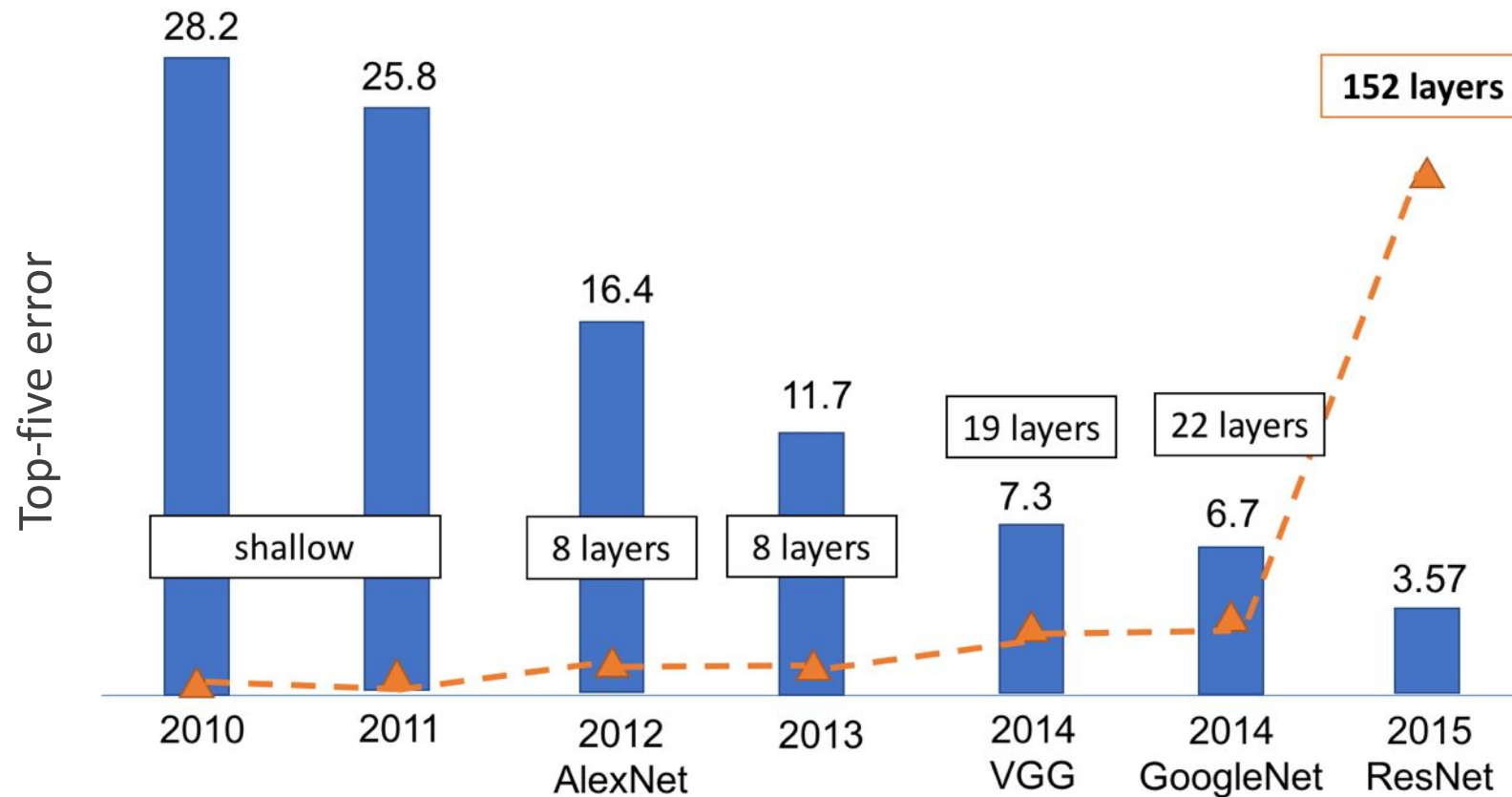


ca. 1 million images
1000 object categories

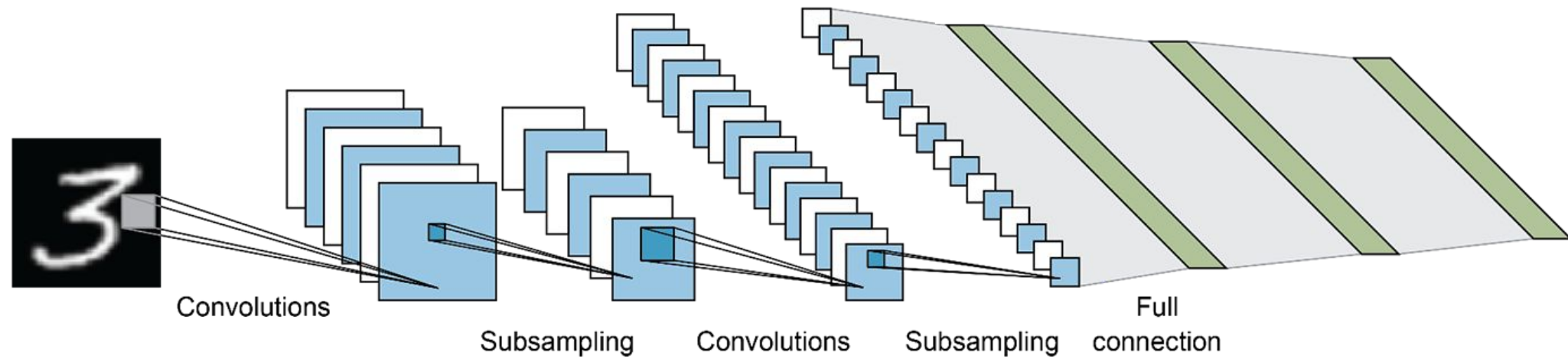
Performance measured as top-five error

More recently most people have switched to top-one error, since the task has become too "easy."

ImageNet performance over time



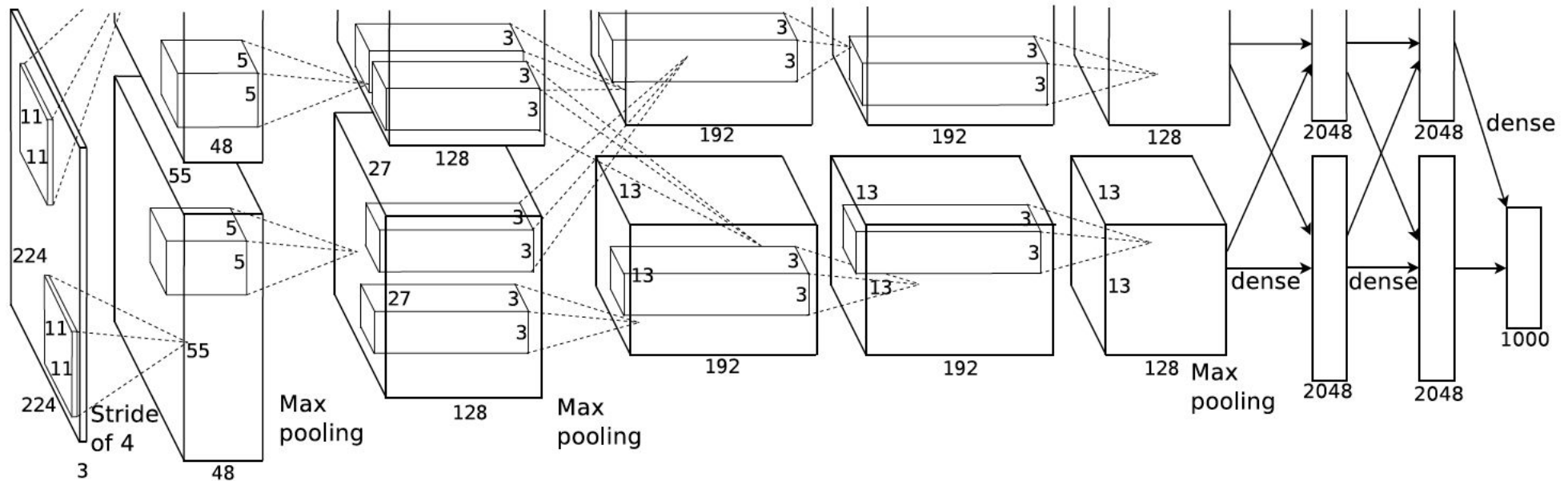
Convolutional neural networks (CNN)



Denker et al. 1989, LeCun et al. 1989, 1998

AlexNet (2012)

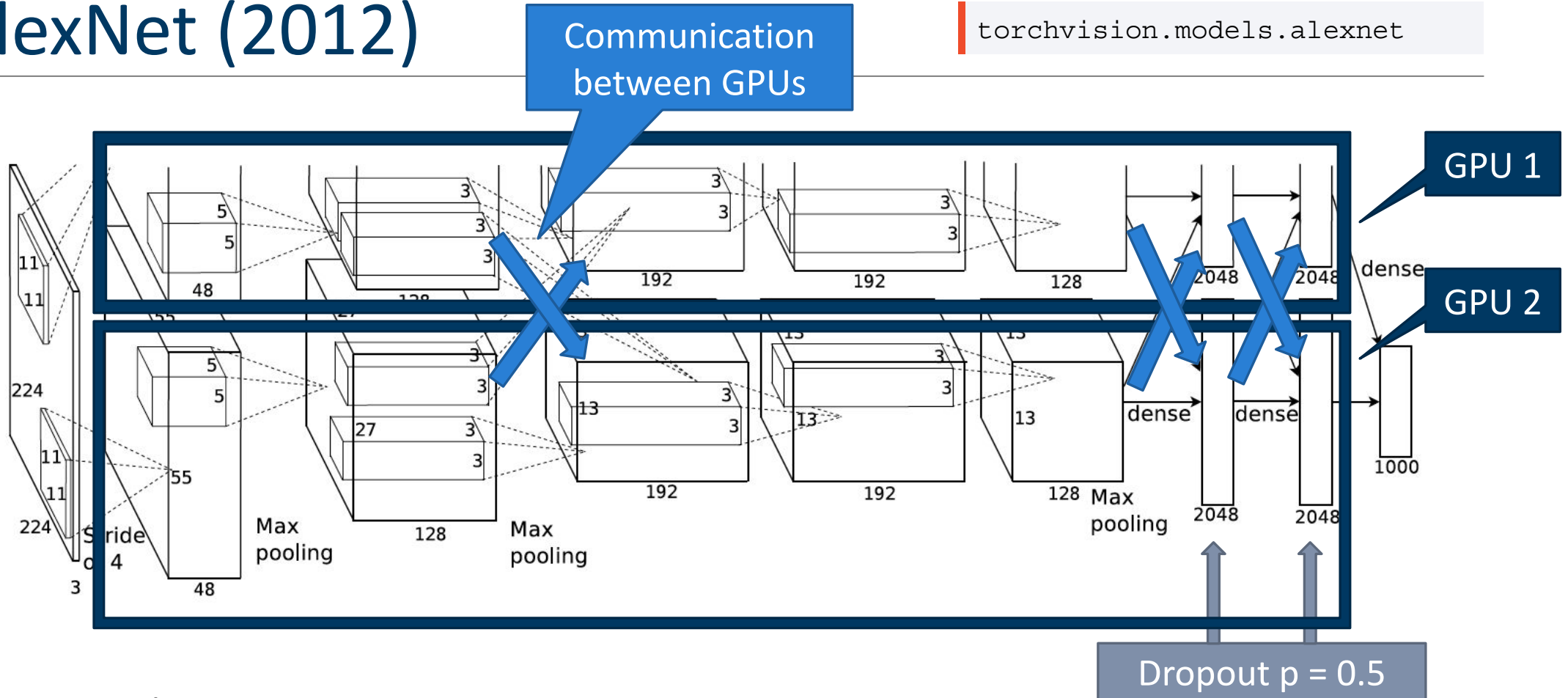
`torchvision.models.alexnet`



Training: 6 days on 2 NVIDIA GTX 580 3 GB GPUs

AlexNet (2012)

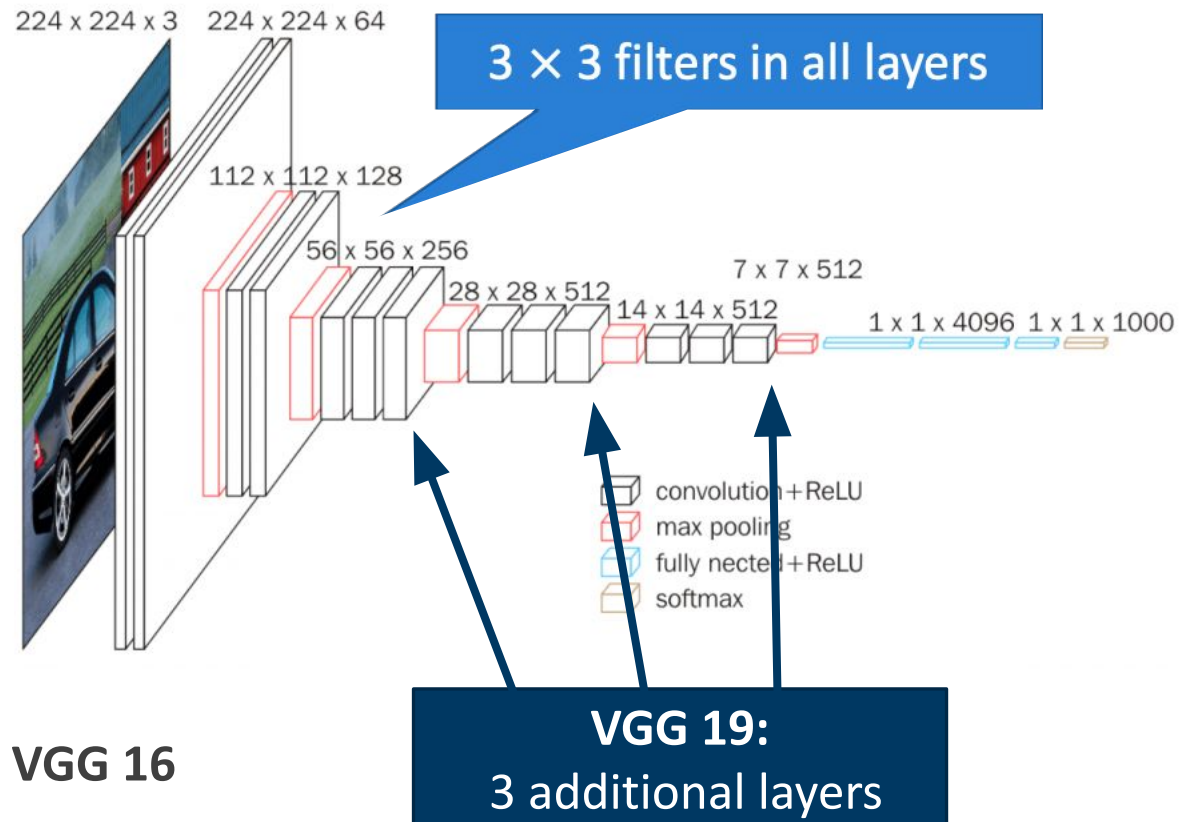
`torchvision.models.alexnet`



Training: 6 days on 2 NVIDIA GTX 580 3 GB GPUs
8 layers / 60 million parameters

VGG (2014)

```
torchvision.models.vgg16  
torchvision.models.vgg19  
torchvision.models.vgg19_bn  
...
```



Training:

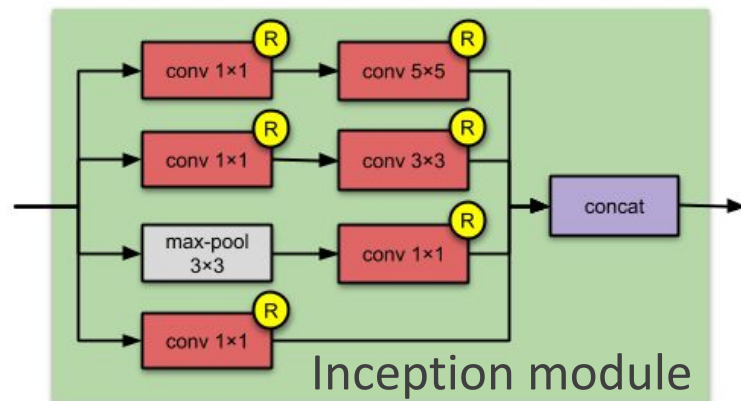
3 weeks on 4 NVIDIA Titan Black 6 GB GPUs

First train smaller configurations, then inject layers in between:
11, 13, 16, 19 layers

138 million parameters / 500 MB

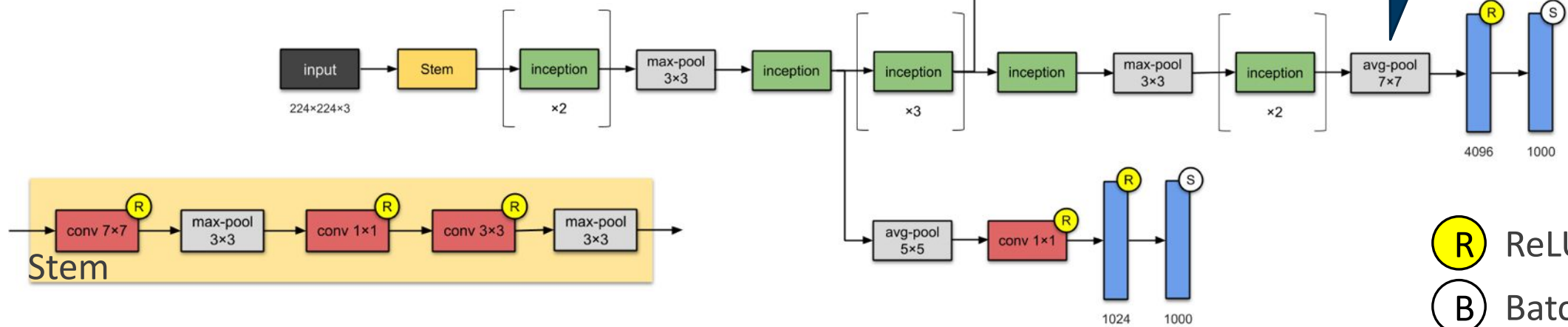
GoogLeNet/Inception-v1 (2014)

`torchvision.models.googlenet`



22 layers
5 million parameters
Trained on CPU

Replaced fully connected layer by global avg pool.
Now standard

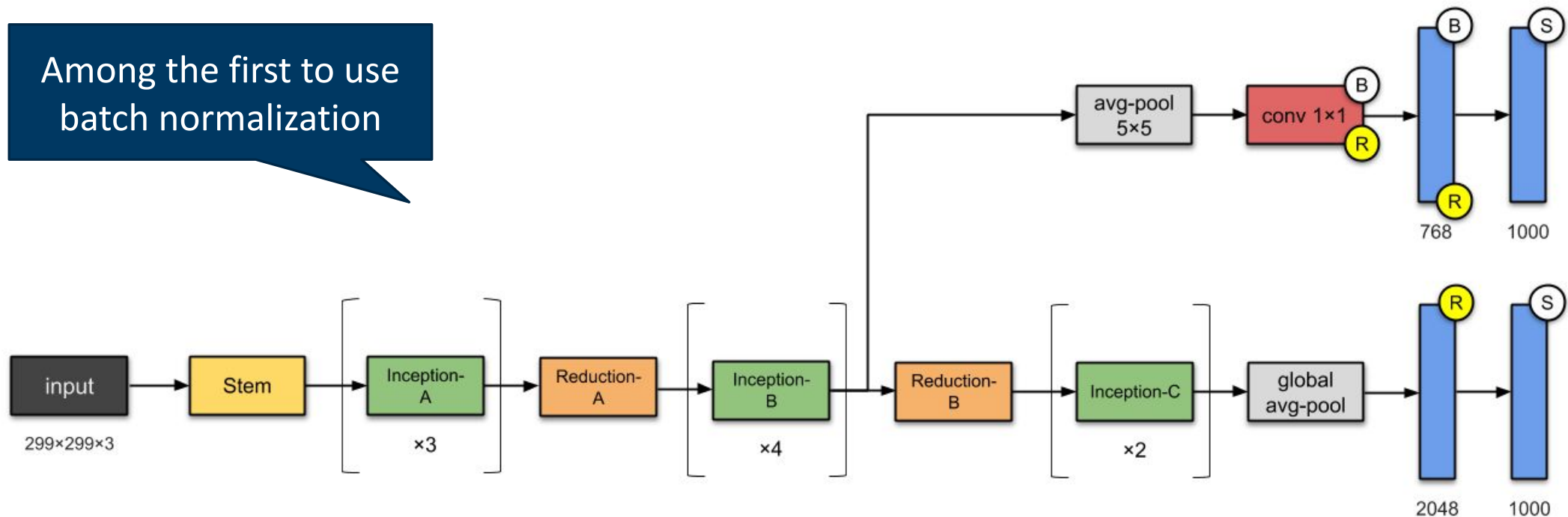


(R) ReLU
(B) Batch norm

Inception-v3 (2015)

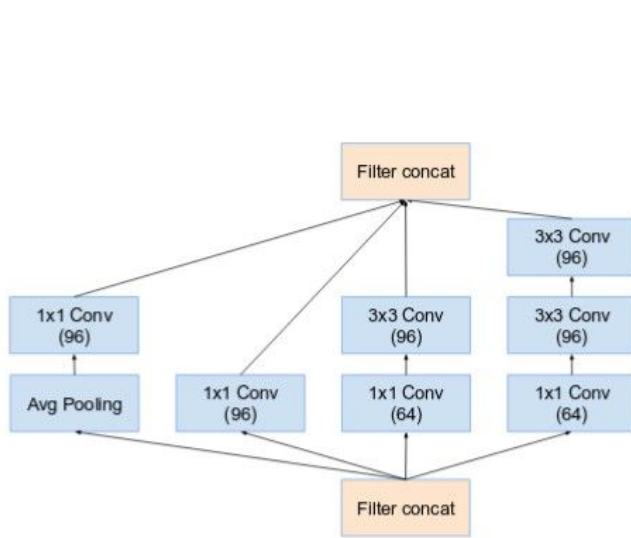
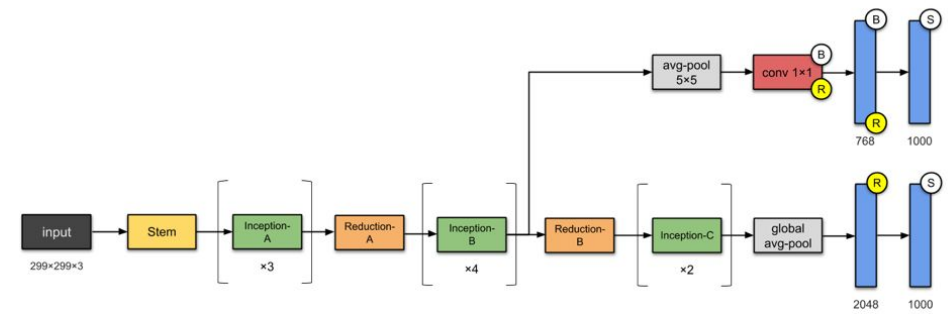
`torchvision.models.inception_v3`

Among the first to use
batch normalization

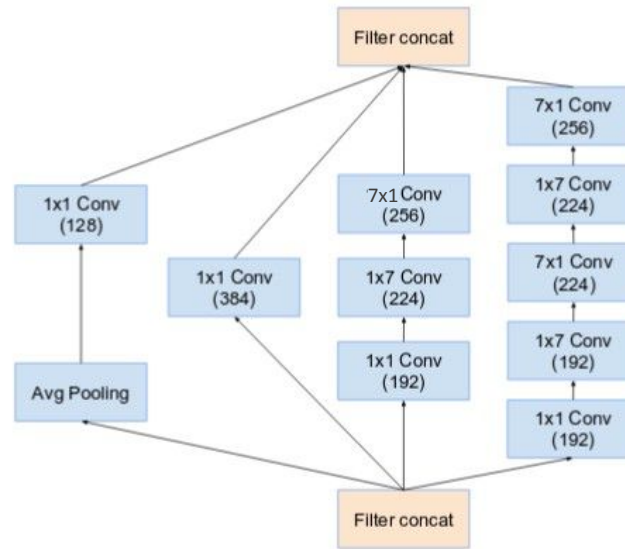


Trained on 50 NVIDIA K40 12 GB GPUs
24 million parameters

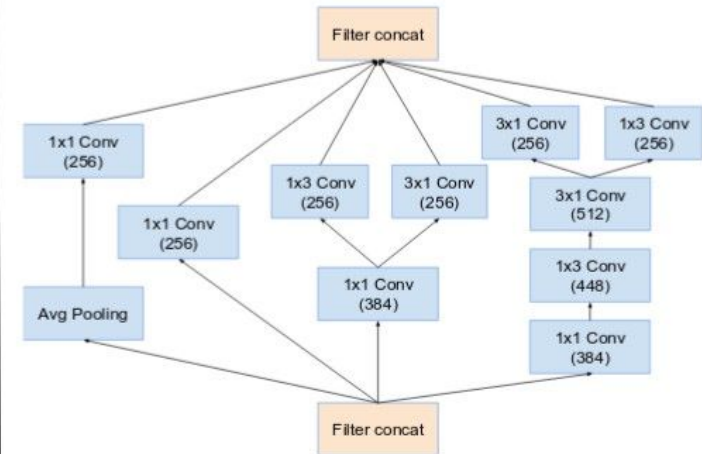
Inception-v3 (2015)



Inception A

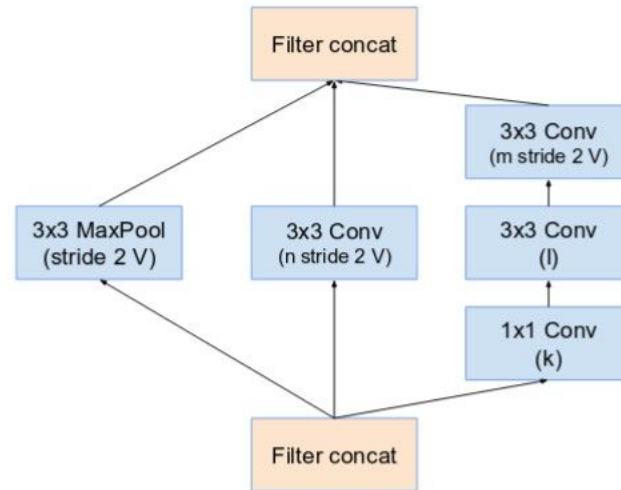
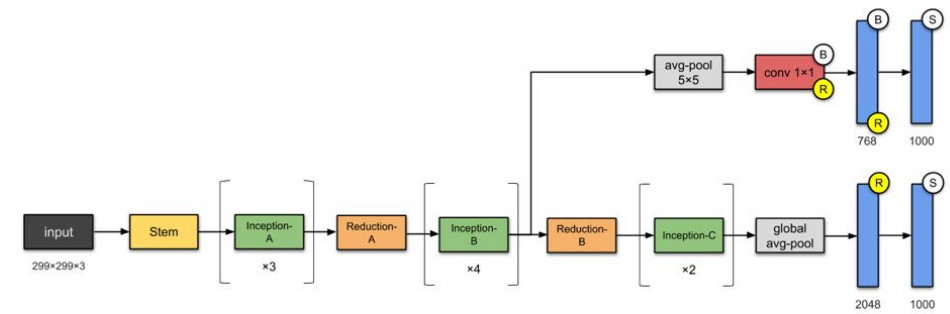


Inception B

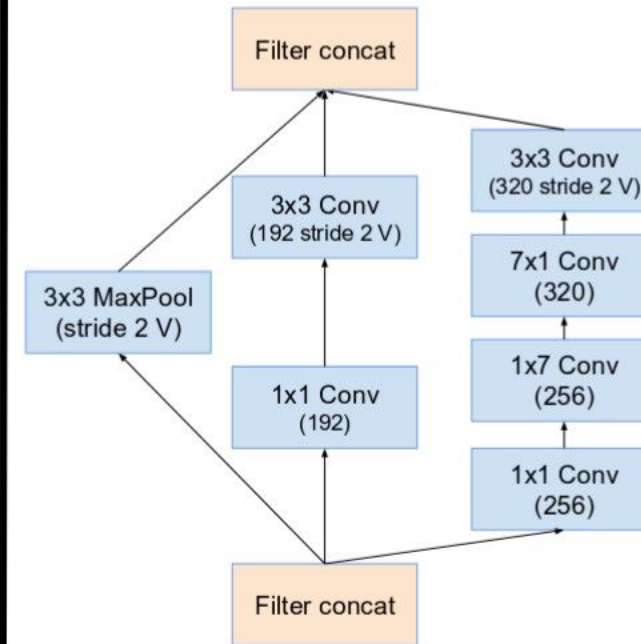


Inception C

Inception-v3 (2015)



Reduction A



Reduction B

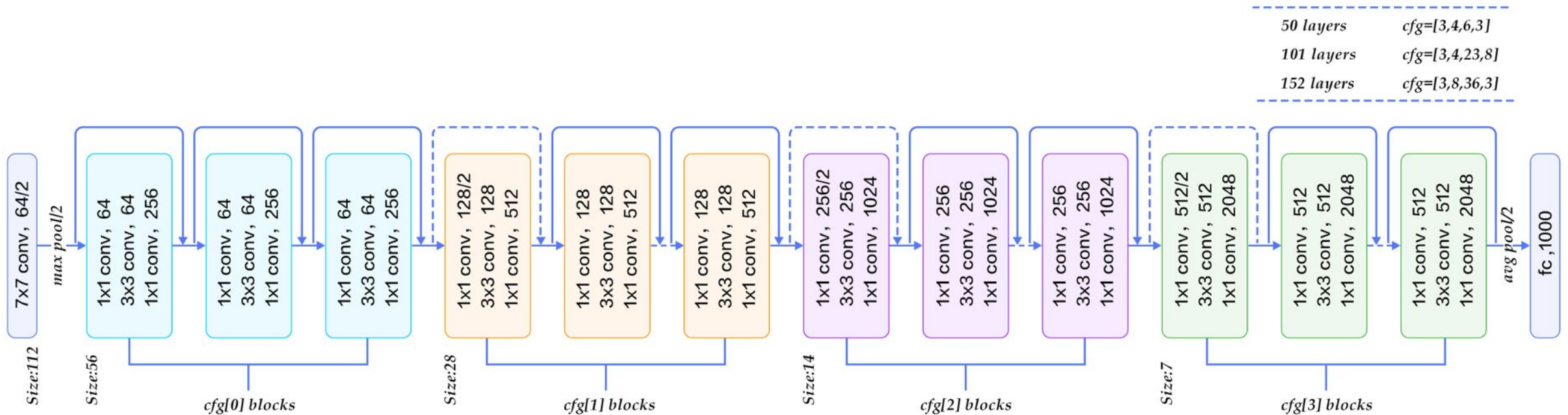
A close-up shot of Leonardo DiCaprio in a dark suit, white shirt, and patterned tie. He has a serious, intense expression and is looking slightly to his right. The lighting is warm and focused on his face. In the background, another person's head and shoulder are visible in profile, looking towards DiCaprio. The overall mood is dramatic and urgent.

WE NEED TO GO

DEEPER

ResNet (2015)

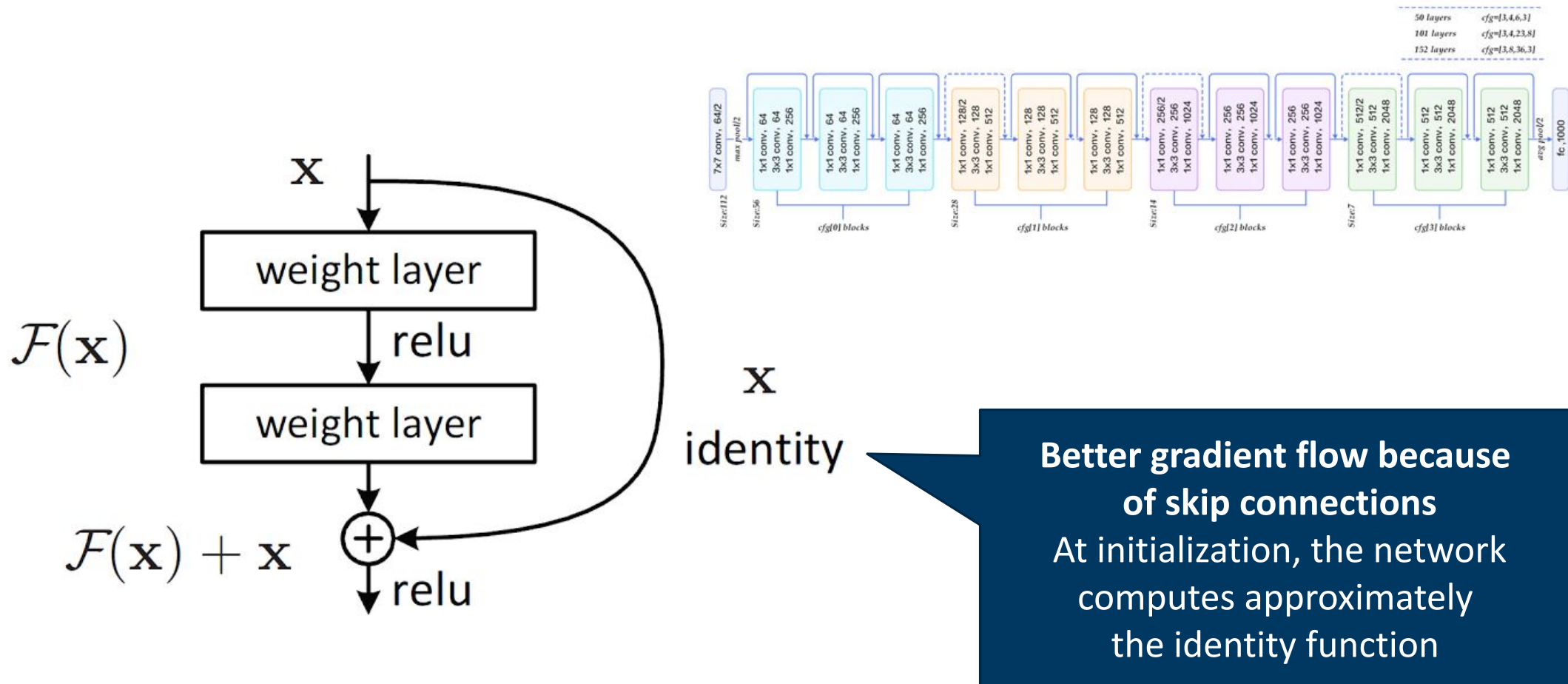
```
torchvision.models.resnet18()  
..  
torchvision.models.resnet152()
```



ResNet-50 (-18, -34, -101, -152 also exist)

26 million parameters

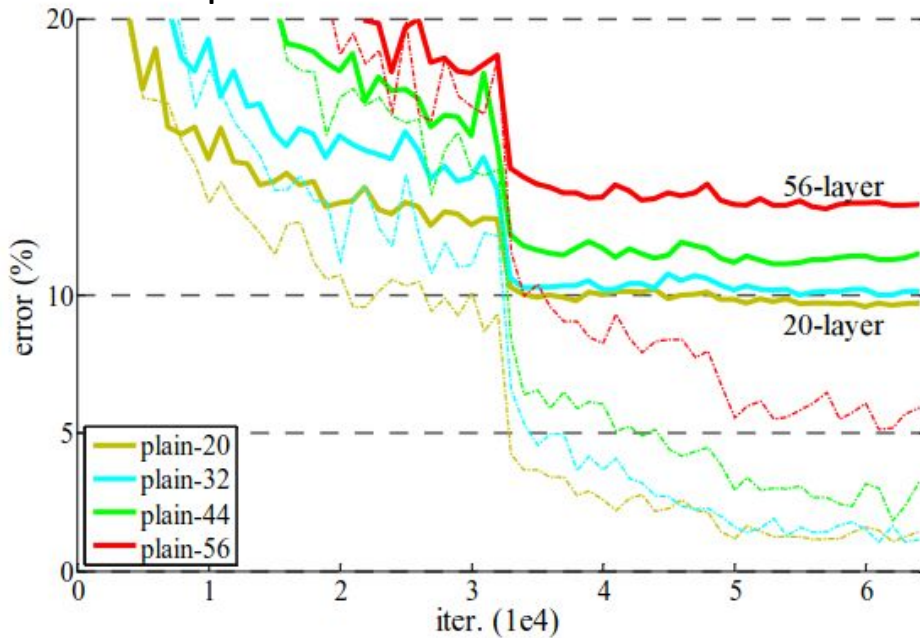
Residual blocks (“skip connections”)



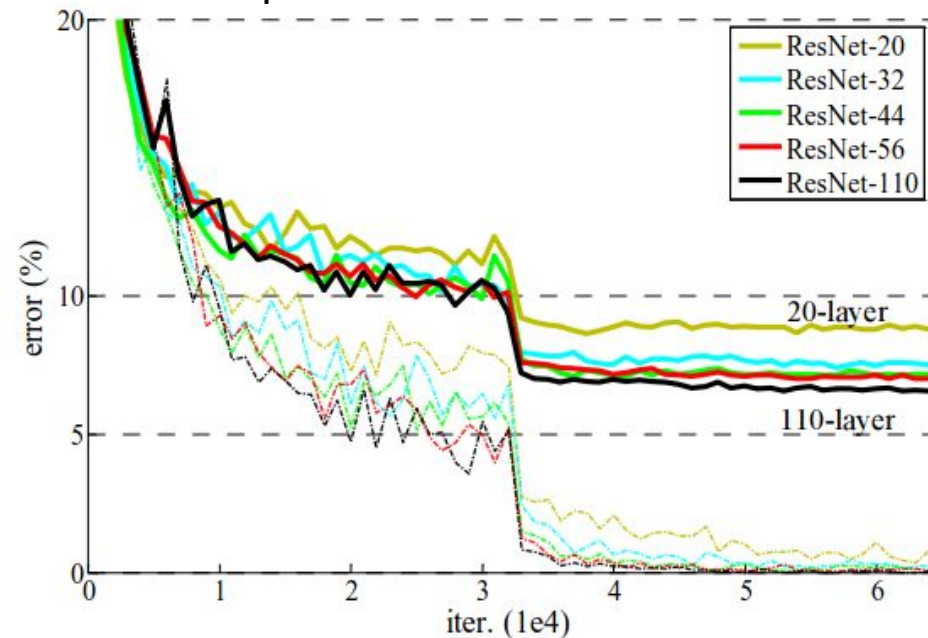
The effect of residual connections

<https://arxiv.org/pdf/1512.03385.pdf>

Plain networks:
no skip connections



ResNet:
with skip connections

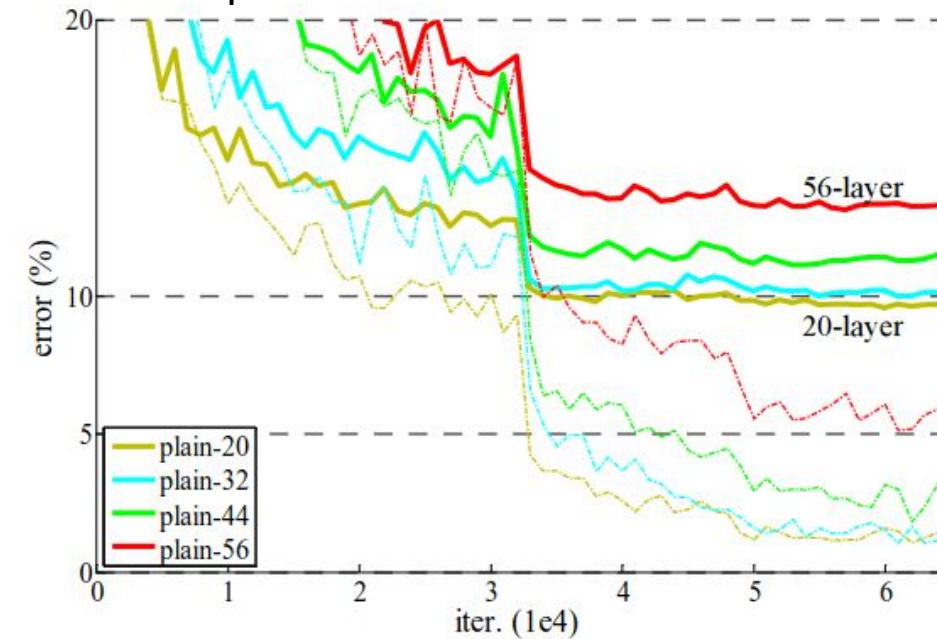


Experiments on CIFAR 10,
dashed lines=training error, bold lines=test error

The effect of residual connections

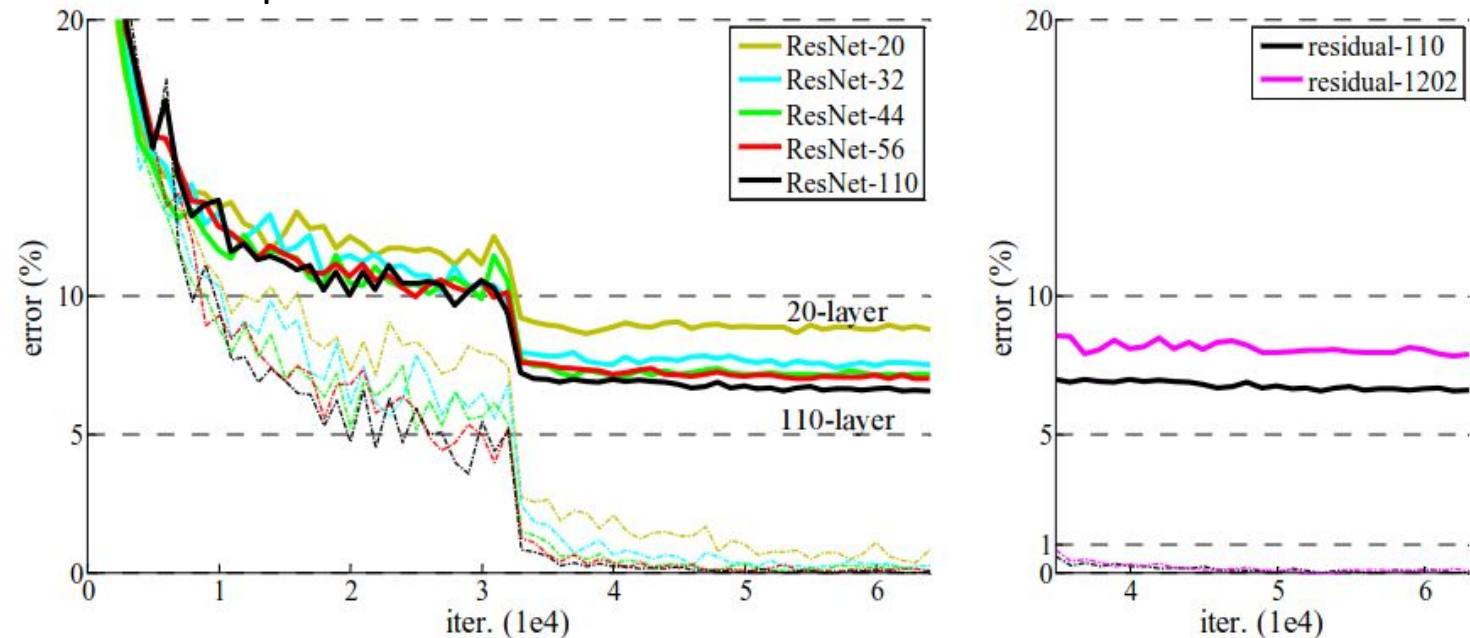
<https://arxiv.org/pdf/1512.03385.pdf>

Plain networks:
no skip connections

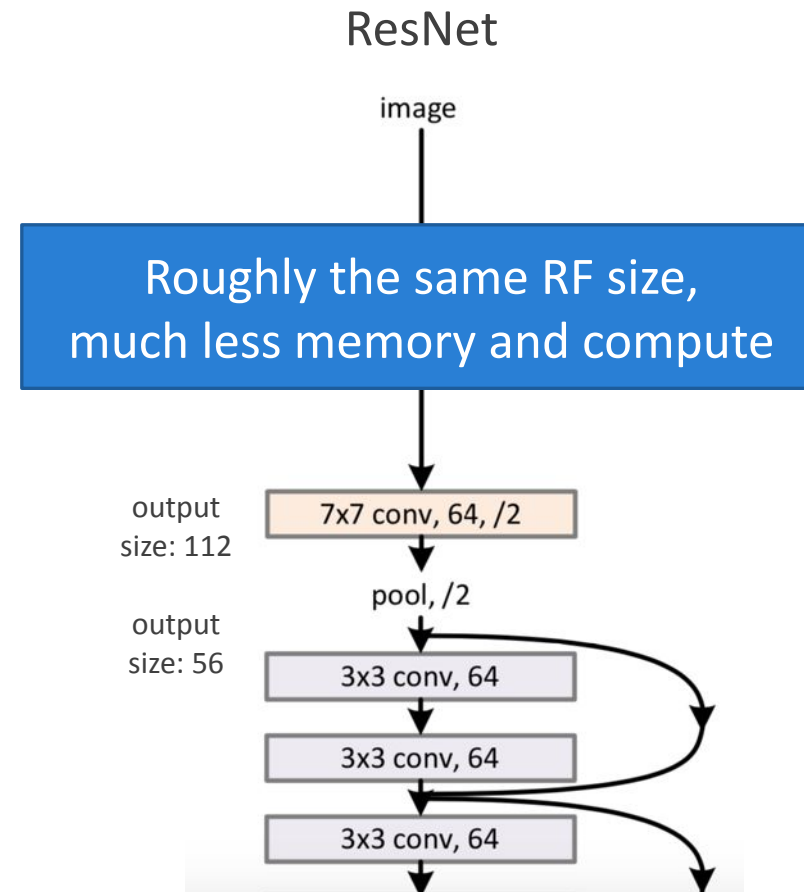
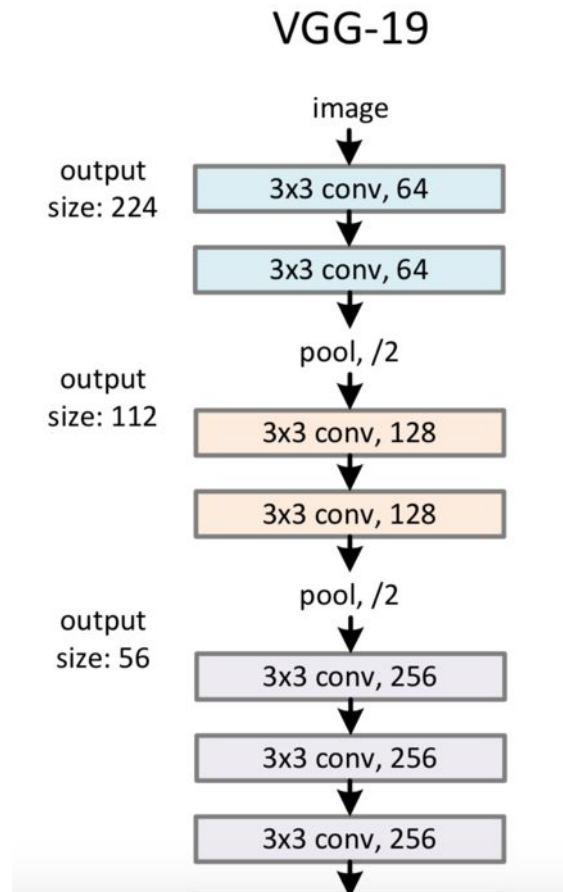


Experiments on CIFAR 10,
dashed lines=training error, bold lines=test error

ResNet:
with skip connections



ResNet (2015)

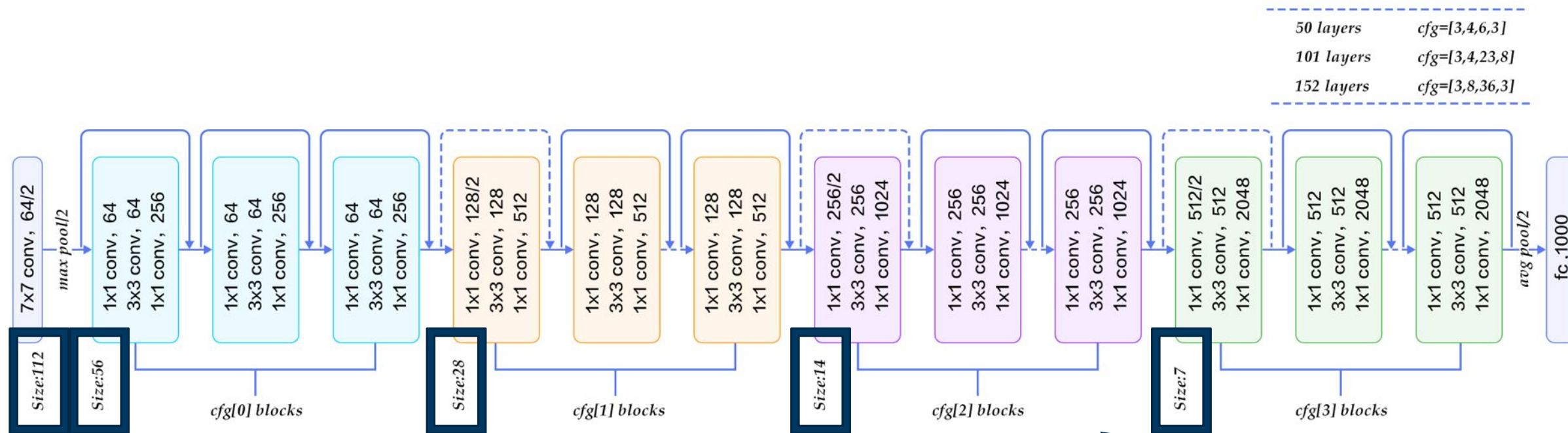


ResNet (2015)

```
torchvision.models.resnet18
```

```
...
```

```
torchvision.models.resnet152
```



Notice a pattern?

Input and feature map sizes

Starting with AlexNet, most networks trained on ImageNet used 224×224 as input

Reduced to $112 \times 112 \rightarrow 56 \times 56 \rightarrow \mathbf{28 \times 28} \rightarrow 14 \times 14 \rightarrow 7 \times 7$ in a cascade of five pooling steps

The unusual numbers probably originate from scaling up from MNIST (28×28) to larger images

More architectures

- ResNeXt
 - ResNet Architecture with parallel convolutional paths
 - Saves parameters, allows deeper networks
- DenseNet
 - All-to-all layer connectivity
 - Better gradient flow and less parameters

More architectures

- ResNeXt
 - ResNet Architecture with parallel convolutional paths
 - Saves parameters, allows deeper networks
- DenseNet
 - All-to-all layer connectivity
 - Better gradient flow and less parameters

ResNet is most commonly used
“off-the-shelf” architecture

More architectures

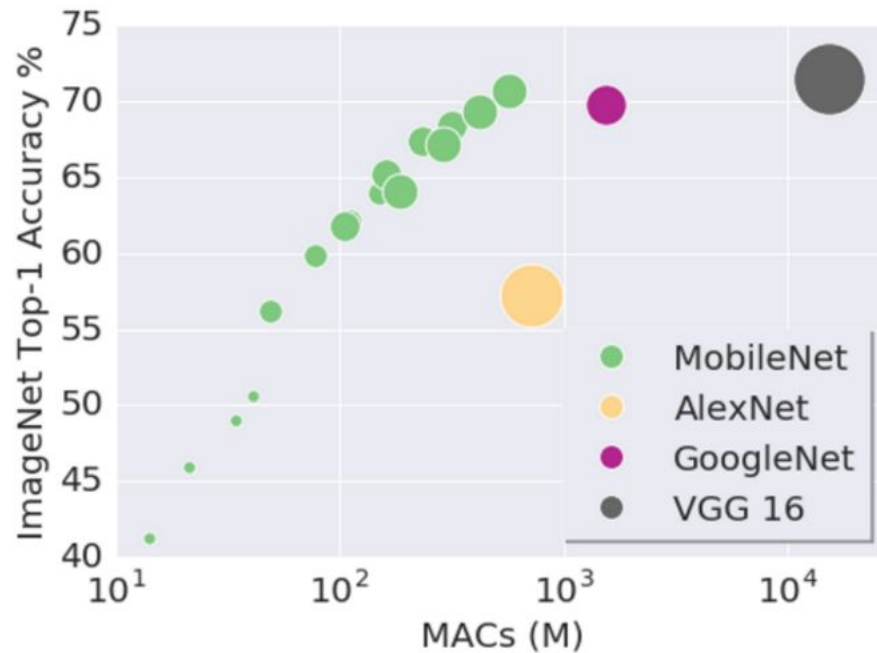
- ResNeXt
 - ResNet Architecture with parallel convolutional paths
 - Saves parameters, allows deeper networks
- DenseNet
 - All-to-all layer connectivity
 - Better gradient flow and less parameters

ResNet is most commonly used
“off-the-shelf” architecture

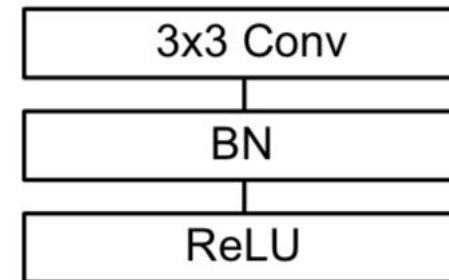
Other concern: inference with limited
computational resources (e.g. Mobile)

MobileNet (2017)

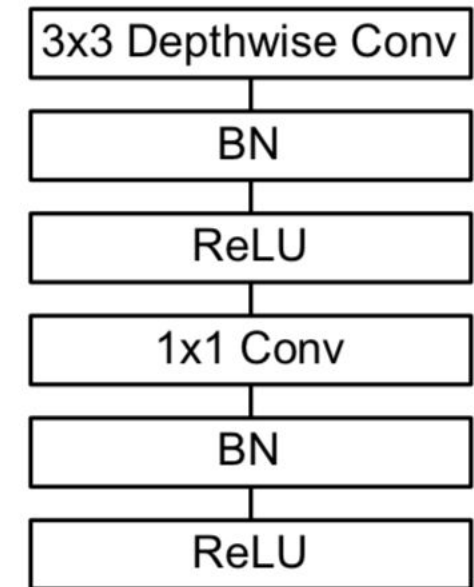
```
torchvision.models.mobilenet_v2
```



Regular CNN



MobileNet



Runs at > 100 frames/sec on an iPhone 7
at ~70% top-1 / 90% top-5 accuracy on ImageNet

Spatial Separable Convolutions

Spatially separable convolutions:

- Express KxK kernel as Kx1 and 1xK kernel
- Disadvantage: cannot express all possible KxK kernels!
- Used in inceptionV3

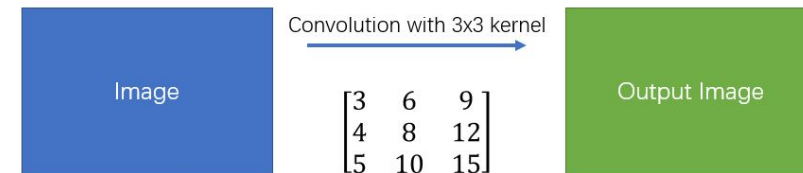
$$\begin{bmatrix} 3 & 6 & 9 \\ 4 & 8 & 12 \\ 5 & 10 & 15 \end{bmatrix} = \begin{bmatrix} 3 \\ 4 \\ 5 \end{bmatrix} \times [1 \quad 2 \quad 3]$$

Spatial Separable Convolutions

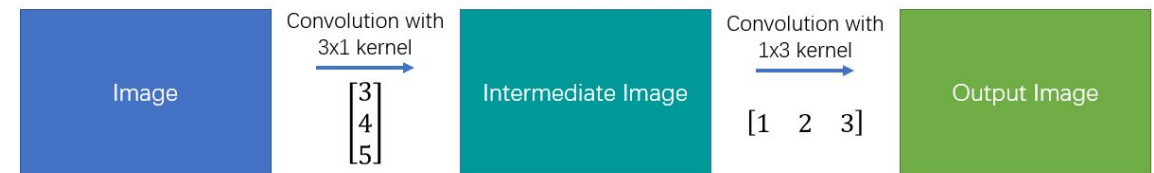
Spatially separable convolutions:

- Express $K \times K$ kernel as $K \times 1$ and $1 \times K$ kernel
- Disadvantage: cannot express all possible $K \times K$ kernels!
- Used in inceptionV3

Simple Convolution

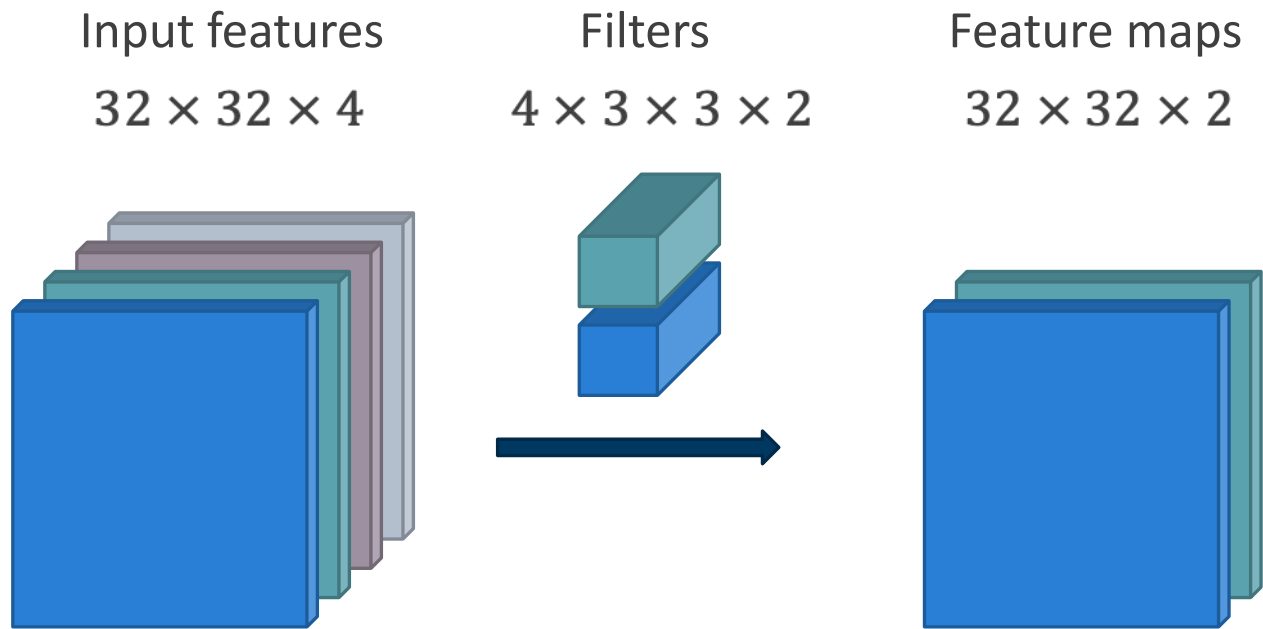


Spatial Separable Convolution



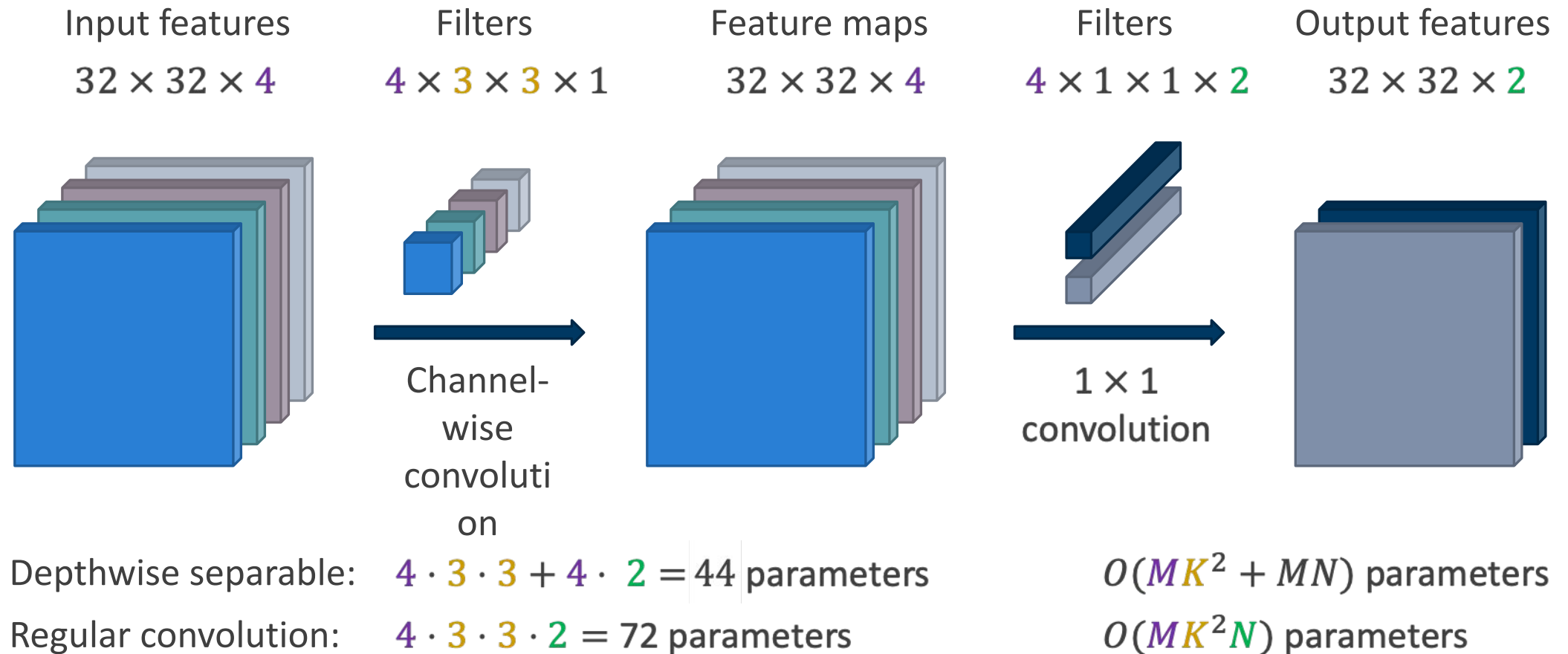
$$\begin{bmatrix} 3 & 6 & 9 \\ 4 & 8 & 12 \\ 5 & 10 & 15 \end{bmatrix} = \begin{bmatrix} 3 \\ 4 \\ 5 \end{bmatrix} \times \begin{bmatrix} 1 & 2 & 3 \end{bmatrix}$$

Convolution layer

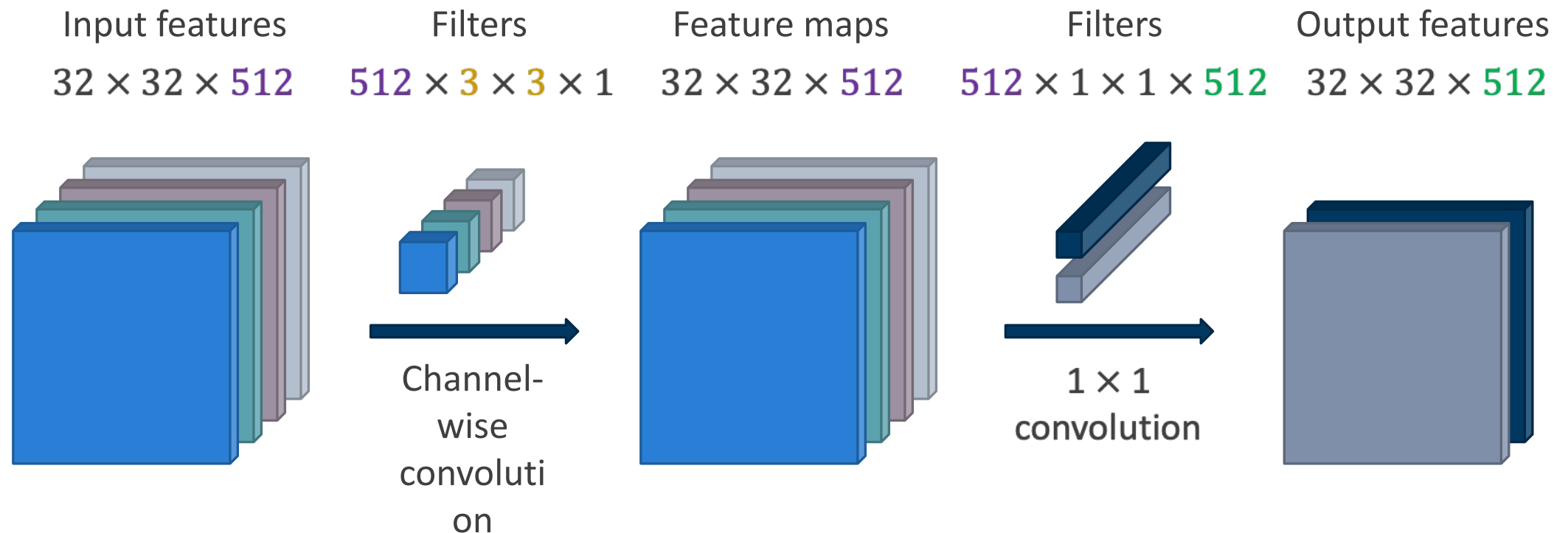


Regular convolution: $4 \cdot 3 \cdot 3 \cdot 2 = 72$ parameters

Depthwise separable convolution



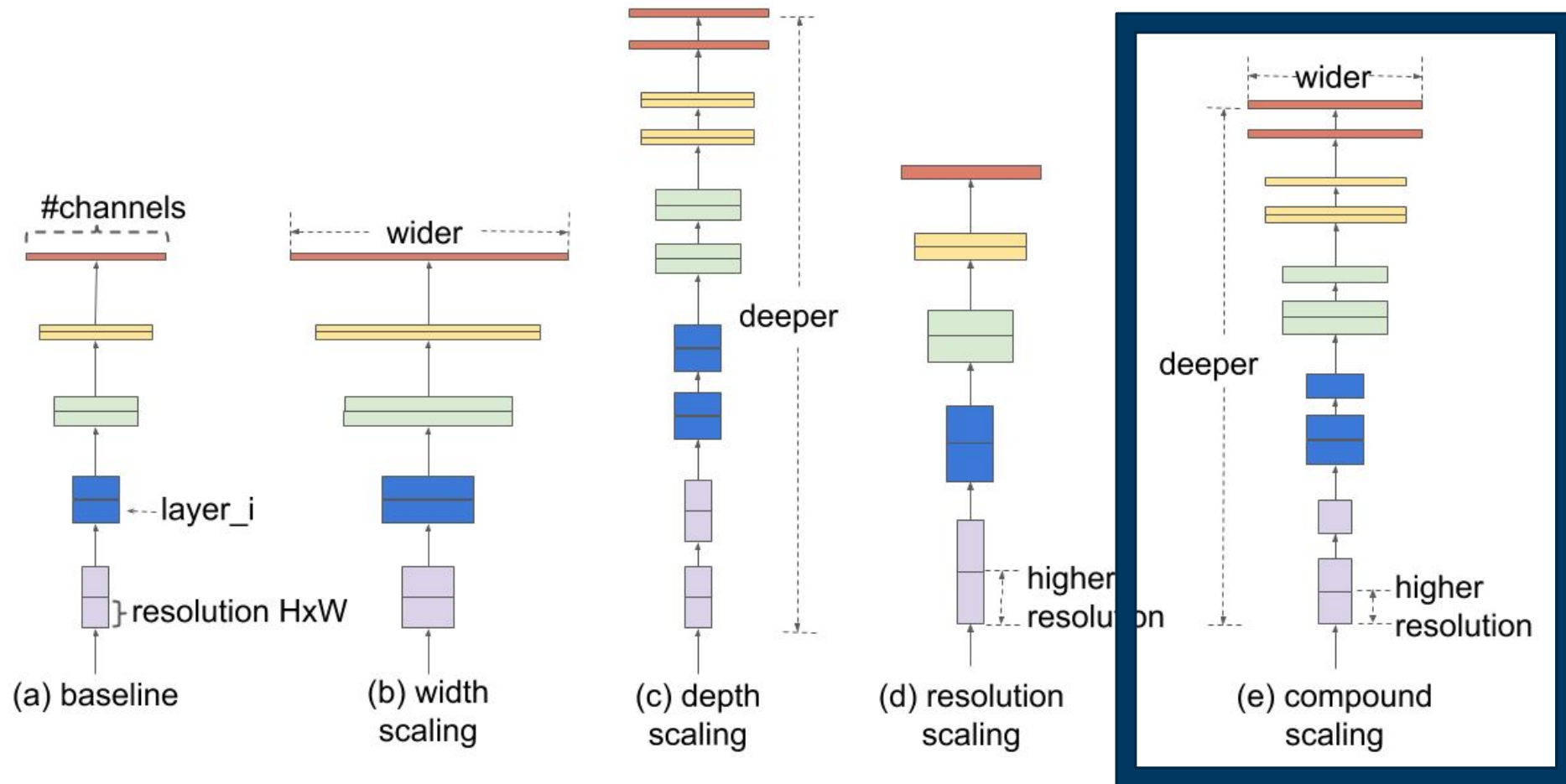
Depthwise separable convolution: real-world case



Depthwise separable: $512 \cdot 3 \cdot 3 + 512 \cdot 512 \approx 267\text{k parameters}$

Regular convolution: $512 \cdot 3 \cdot 3 \cdot 512 \approx 2.4\text{m parameters}$

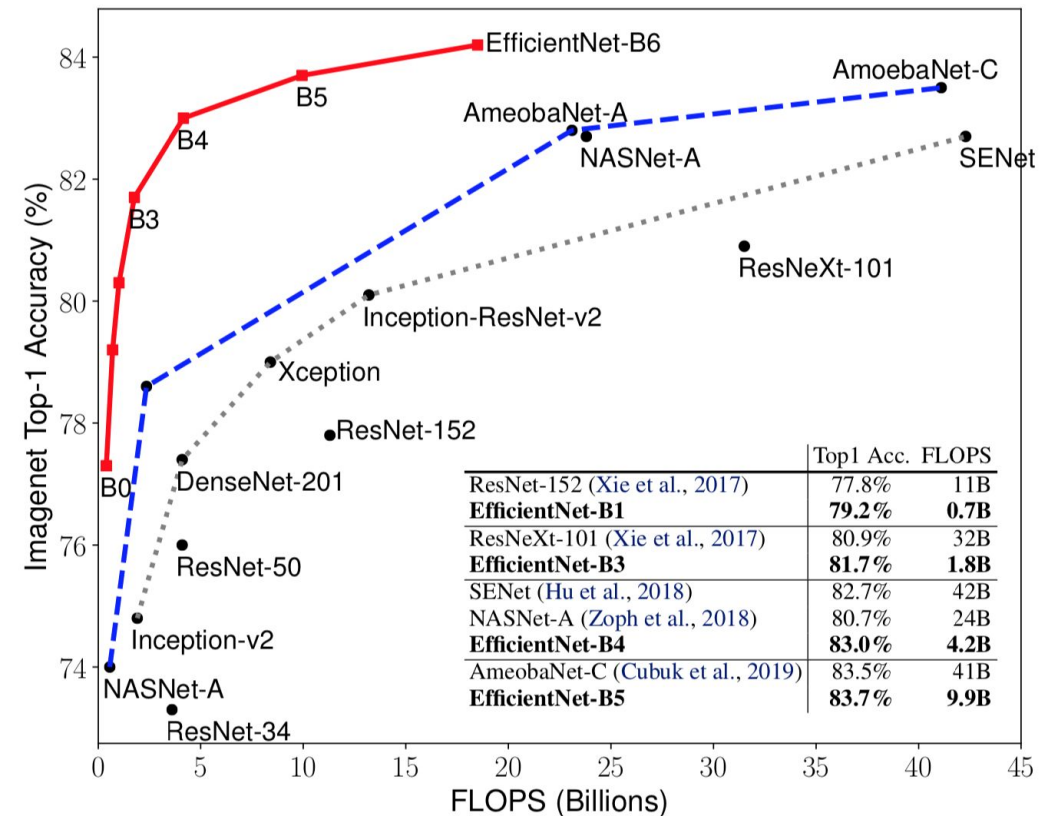
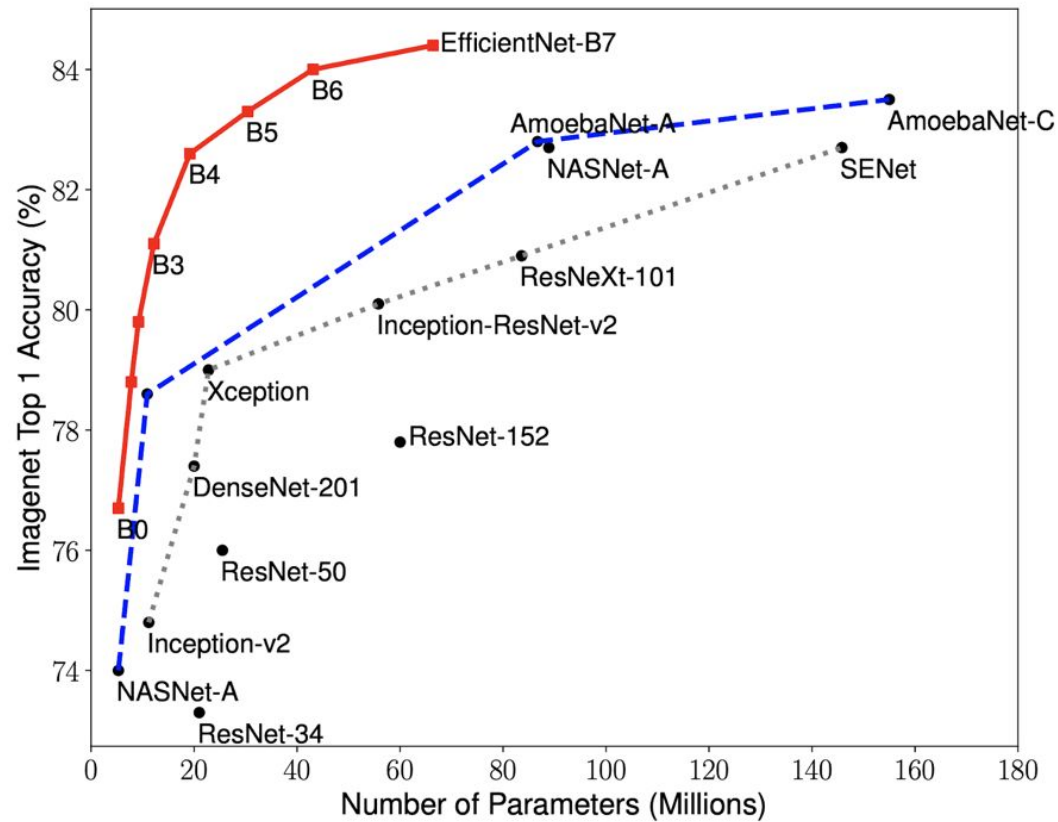
EfficientNet (2019)



EfficientNet

Tan & Le, *ICML* 2019

EfficientNet (2019)



	Top1 Acc.	FLOPS
ResNet-152 (Xie et al., 2017)	77.8%	11B
EfficientNet-B1	79.2%	0.7B
ResNeXt-101 (Xie et al., 2017)	80.9%	32B
EfficientNet-B3	81.7%	1.8B
SENet (Hu et al., 2018)	82.7%	42B
NASNet-A (Zoph et al., 2018)	80.7%	24B
EfficientNet-B4	83.0%	4.2B
AmoebaNet-C (Cubuk et al., 2019)	83.5%	41B
EfficientNet-B5	83.7%	9.9B

Tan & Le, *ICML* 2019

<https://ai.googleblog.com/2019/05/efficientnet-improving-accuracy-and.html>

After 2019

More convolutional architectures

- ResNeSt <https://arxiv.org/abs/2004.08955>
 - ResNet Architecture with channel-wise attention mechanism
- ConvNext <https://arxiv.org/abs/2201.03545>
 - Applying lessons from vision transformers to ResNet design
- ...

Vision Transformers: attention layer based architectures for computer vision
does not use convolutions!

Will cover in later lectures!

Student assistant positions

EKFZ: Student Assistant – IT Support for Regulatory Affairs

- Build up IT infrastructure for documenting gene therapy trials
- <https://ekfz.uni-goettingen.de/stellenausschreibungen/>



CryoET Facility: Scientific IT Infrastructure & DevOps Automation

- Build up IT infrastructure for cryogenic electron tomography
- <https://owncloud.gwdg.de/index.php/s/WtIA9RPAEaPnPuv>



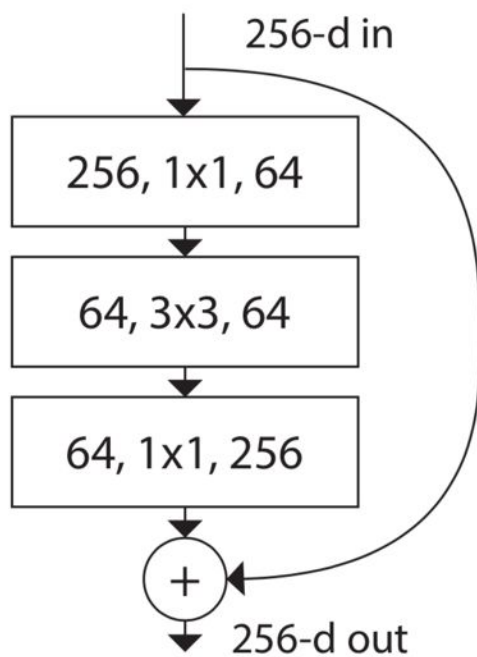
Questions?

Backup slides for other architectures

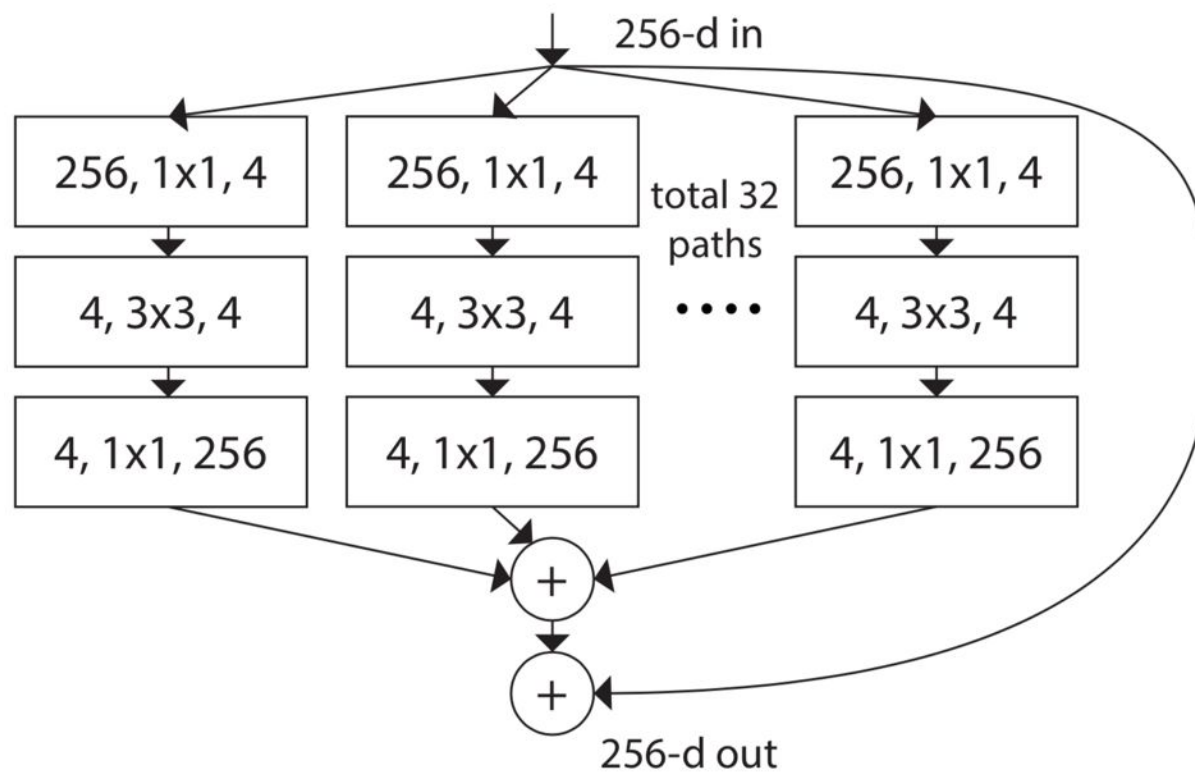
These are not relevant for the exam!

ResNeXt (2017)

`torchvision.models.resnext50_32x4d`



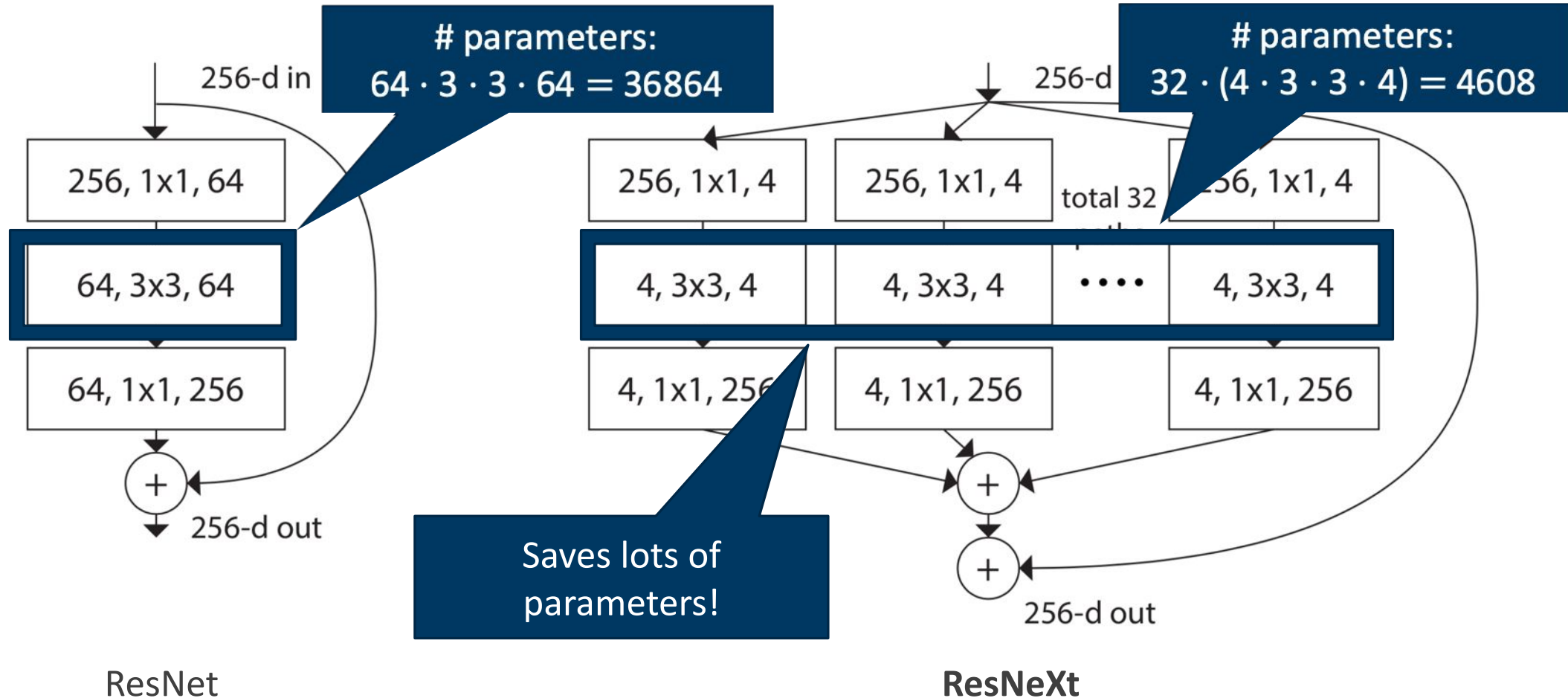
ResNet



ResNeXt

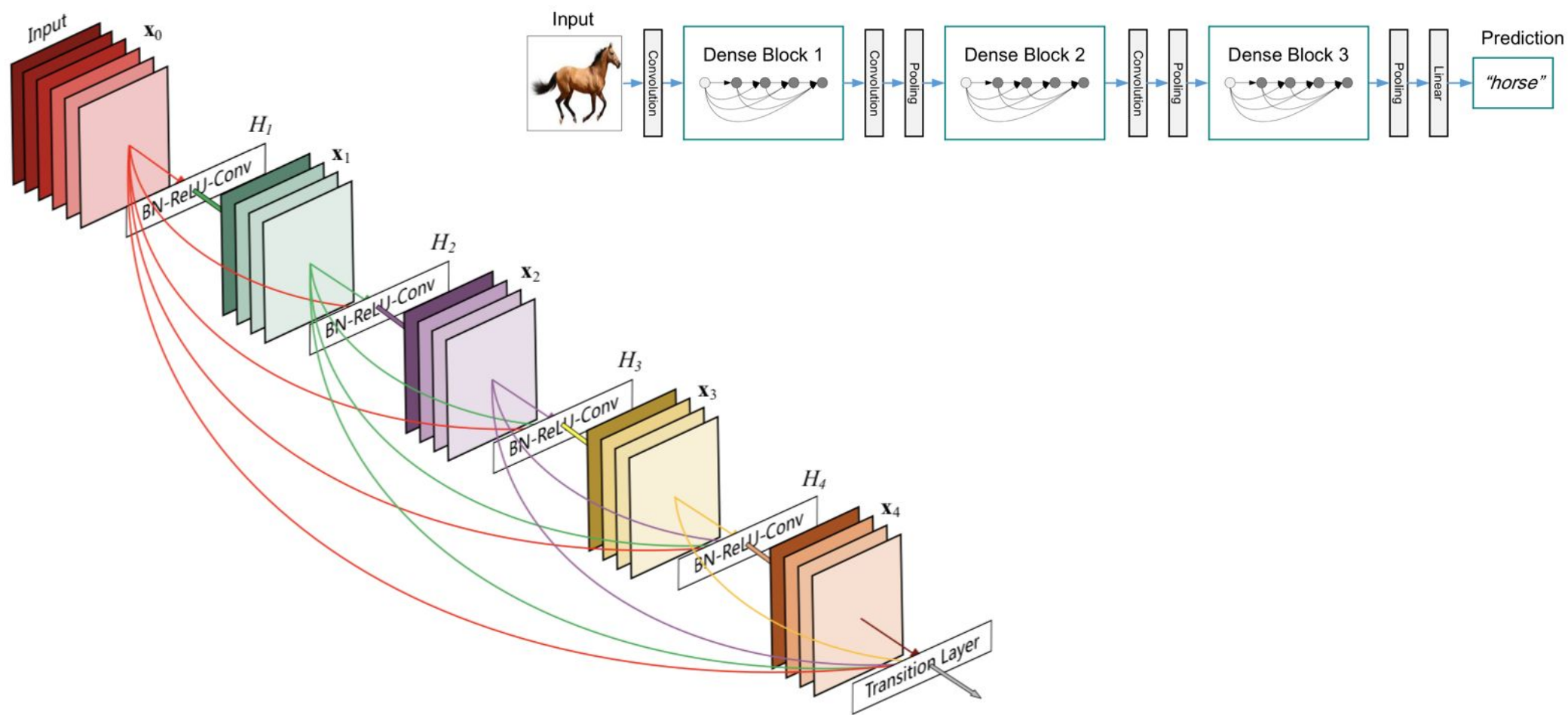
ResNeXt (2017)

`torchvision.models.resnext50_32x4d`



DenseNet

`torchvision.models.densenet161`



DenseNet

`torchvision.models.densenet161`

Layers	Output Size	DenseNet-121	DenseNet-169	DenseNet-201	DenseNet-264
Convolution	112×112	7×7 conv, stride 2			
Pooling	56×56	3×3 max pool, stride 2			
Dense Block (1)	56×56	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 6$
Transition Layer (1)	56×56	1×1 conv			
	28×28	2×2 average pool, stride 2			
Dense Block (2)	28×28	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 12$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 12$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 12$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 12$
Transition Layer (2)	28×28	1×1 conv			
	14×14	2×2 average pool, stride 2			
Dense Block (3)	14×14	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 24$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 32$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 48$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 64$
Transition Layer (3)	14×14	1×1 conv			
	7×7	2×2 average pool, stride 2			
Dense Block (4)	7×7	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 16$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 32$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 32$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 48$
Classification Layer	1×1	7×7 global average pool			
		1000D fully-connected, softmax			

Table 1: DenseNet architectures for ImageNet. The growth rate for all the networks is $k = 32$. Note that each “conv” layer shown in the table corresponds the sequence BN-ReLU-Conv.